

Marzo 2011

TÍTULO

**Seguridad funcional de los sistemas eléctricos/electrónicos/
electrónicos programables relacionados con la seguridad**

Parte 7: Presentación de técnicas y medidas

Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 7: Overview of techniques and measures.

Sécurité fonctionnelle des systèmes électriques/électroniques/électroniques programmables relatifs à la sécurité. Partie 7: Présentation de techniques et mesures.

CORRESPONDENCIA

Esta norma es la versión oficial, en español, de la Norma Europea EN 61508-7:2010, que a su vez adopta la Norma Internacional IEC 61508-7:2010.

OBSERVACIONES

Esta norma anulará y sustituirá a las Normas UNE-EN 61508-7:2004 y UNE-EN 61508-7:2004 Erratum antes de 2013-05-01.

ANTECEDENTES

Esta norma ha sido elaborada por el comité técnico AEN/CTN 203 *Equipamiento eléctrico y sistemas automáticos para la industria* cuya Secretaría desempeña SERCOBE.

Editada e impresa por AENOR
Depósito legal: M 14860:2011

© AENOR 2011
Reproducción prohibida

LAS OBSERVACIONES A ESTE DOCUMENTO HAN DE DIRIGIRSE A:

AENOR

Asociación Española de
Normalización y Certificación

Génova, 6
28004 MADRID-España

info@aenor.es
www.aenor.es

Tel.: 902 102 201
Fax: 913 104 032

153 Páginas

Grupo 87

Versión en español

**Seguridad funcional de los sistemas eléctricos/electrónicos/
electrónicos programables relacionados con la seguridad
Parte 7: Presentación de técnicas y medidas
(IEC 61508-7:2010)**

**Functional safety of electrical/electronic/
programmable electronic safety-related
systems.
Part 7: Overview of techniques and
measures.
(IEC 61508-7:2010).**

**Sécurité fonctionnelle des systèmes
électriques/électroniques/électroniques
programmables relatifs à la sécurité.
Partie 7: Présentation de techniques et
mesures.
(CEI 61508-7:2010).**

**Funktionale Sicherheit
sicherheitsbezogener elektrischer/
elektronischer/programmierbarer
elektronischer Systeme.
Teil 7: Überblick über Verfahren und
Maßnahmen.
(IEC 61508-7:2010).**

Esta norma europea ha sido aprobada por CENELEC el 2010-05-01. Los miembros de CENELEC están sometidos al Reglamento Interior de CEN/CENELEC que define las condiciones dentro de las cuales debe adoptarse, sin modificación, la norma europea como norma nacional.

Las correspondientes listas actualizadas y las referencias bibliográficas relativas a estas normas nacionales, pueden obtenerse en la Secretaría Central de CENELEC, o a través de sus miembros.

Esta norma europea existe en tres versiones oficiales (alemán, francés e inglés). Una versión en otra lengua realizada bajo la responsabilidad de un miembro de CENELEC en su idioma nacional, y notificada a la Secretaría Central, tiene el mismo rango que aquéllas.

Los miembros de CENELEC son los comités electrotécnicos nacionales de normalización de los países siguientes: Alemania, Austria, Bélgica, Bulgaria, Chipre, Croacia, Dinamarca, Eslovaquia, Eslovenia, España, Estonia, Finlandia, Francia, Grecia, Hungría, Irlanda, Islandia, Italia, Letonia, Lituania, Luxemburgo, Malta, Noruega, Países Bajos, Polonia, Portugal, Reino Unido, República Checa, Rumanía, Suecia y Suiza.

CENELEC
COMITÉ EUROPEO DE NORMALIZACIÓN ELECTROTÉCNICA
European Committee for Electrotechnical Standardization
Comité Européen de Normalisation Electrotechnique
Europäisches Komitee für Elektrotechnische Normung
SECRETARÍA CENTRAL: Avenue Marnix, 17-1000 Bruxelles

© 2010 CENELEC. Derechos de reproducción reservados a los Miembros de CENELEC.

PRÓLOGO

El texto del documento 65A/554/FDIS, futura edición 2 de la Norma IEC 61508-7, preparado por el Subcomité SC 65A, *Cuestiones relativas a los sistemas*, del Comité Técnico TC 65, *Medida, control y automatización de procesos industriales*, de IEC, fue sometido a voto paralelo IEC-CENELEC y fue aprobado por CENELEC como Norma EN 61508-7 el 2010-05-01.

Esta norma sustituye a la Norma EN 61508-7:2001.

Se llama la atención sobre la posibilidad de que algunos de los elementos de este documento estén sujetos a derechos de patente. CEN y CENELEC no son responsables de la identificación de dichos derechos de patente.

Se fijaron las siguientes fechas:

- | | | |
|---|-------|------------|
| – Fecha límite en la que la norma europea debe adoptarse a nivel nacional por publicación de una norma nacional idéntica o por ratificación | (dop) | 2011-02-01 |
| – Fecha límite en la que deben retirarse las normas nacionales divergentes con esta norma | (dow) | 2013-05-01 |

El anexo ZA ha sido añadido por CENELEC.

DECLARACIÓN

El texto de la Norma IEC 61508-7:2010 fue aprobado por CENELEC como norma europea sin ninguna modificación.

En la versión oficial, para la bibliografía, debe añadirse la siguiente nota para la norma indicada*:

- | | |
|-------------------------|--|
| [1] IEC 60068-1:1988 | NOTA Armonizada como Norma EN 60068-1:1994 (sin ninguna modificación). |
| [2] IEC 60529:1989 | NOTA Armonizada como Norma EN 60529:1991 (sin ninguna modificación). |
| [3] IEC 60812:2006 | NOTA Armonizada como Norma EN 60812:2006 (sin ninguna modificación). |
| [4] IEC 60880:2006 | NOTA Armonizada como Norma EN 60880:2009 (sin ninguna modificación). |
| [5] IEC 61000-4-1:2006 | NOTA Armonizada como Norma EN 61000-4-1:2007 (sin ninguna modificación). |
| [6] IEC 61000-4-5:2005 | NOTA Armonizada como Norma EN 61000-4-5:2006 (sin ninguna modificación). |
| [8] IEC 61025:2006 | NOTA Armonizada como Norma EN 61025:2007 (sin ninguna modificación). |
| [9] IEC 61069-5:1994 | NOTA Armonizada como Norma EN 61069-5:1995 (sin ninguna modificación). |
| [10] IEC 61078:2006 | NOTA Armonizada como Norma EN 61078:2006 (sin ninguna modificación). |
| [11] IEC 61131-3:2003 | NOTA Armonizada como Norma EN 61131-3:2003 (sin ninguna modificación). |
| [12] IEC 61160:2005 | NOTA Armonizada como Norma EN 61160:2005 (sin ninguna modificación). |
| [13] IEC 61163-1:2006 | NOTA Armonizada como Norma EN 61163-1:2006 (sin ninguna modificación). |
| [14] IEC 61164:2004 | NOTA Armonizada como Norma EN 61164:2004 (sin ninguna modificación). |
| [15] IEC 61165:2006 | NOTA Armonizada como Norma EN 61165:2006 (sin ninguna modificación). |
| [16] IEC 61326-3-1:2008 | NOTA Armonizada como Norma EN 61326-3-1:2008 (sin ninguna modificación). |

[17] IEC 61326-3-2:2008	NOTA	Armonizada como Norma EN 61326-3-2:2008 (sin ninguna modificación).
[18] IEC 81346-1:2009	NOTA	Armonizada como Norma EN 81346-1:2009 (sin ninguna modificación).
[21] IEC 61511 series	NOTA	Armonizada como serie de Normas EN 61511 (sin ninguna modificación).
[22] IEC 62061:2005	NOTA	Armonizada como Norma EN 62061:2005 (sin ninguna modificación).
[23] IEC 62308:2006	NOTA	Armonizada como Norma EN 62308:2006 (sin ninguna modificación).
[37] IEC 61800-5-2	NOTA	Armonizada como Norma EN 61800-5-2.
[38] IEC 60601 series	NOTA	Armonizada como serie de Normas EN 60601 (parcialmente modificada).
[39] IEC 60068-2-1	NOTA	Armonizada como Norma EN 60068-2-1.
[40] IEC 60068-2-2	NOTA	Armonizada como Norma EN 60068-2-2.
[41] ISO 9000	NOTA	Armonizada como Norma EN ISO 9000.
[42] IEC 61508-1:2010	NOTA	Armonizada como Norma EN 61508-1:2010 (sin ninguna modificación).
[43] IEC 61508-2:2010	NOTA	Armonizada como Norma EN 61508-2:2010 (sin ninguna modificación).
[44] IEC 61508-3:2010	NOTA	Armonizada como Norma EN 61508-3:2010 (sin ninguna modificación).
[45] IEC 61508-6:2010	NOTA	Armonizada como Norma EN 61508-6:2010 (sin ninguna modificación).

* Introducida en la norma indicándose con una línea vertical en el margen izquierdo del texto.

ÍNDICE

	Página
PRÓLOGO	8
INTRODUCCIÓN.....	10
1 OBJETO Y CAMPO DE APLICACIÓN.....	12
2 NORMAS PARA CONSULTA.....	14
3 DEFINICIONES Y ABREVIATURAS	14
ANEXO A (Informativo) PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA LOS SISTEMAS E/E/PE RELACIONADOS CON LA SEGURIDAD: CONTROL DE LAS DISFUNCIONES ALEATORIAS DEL HARDWARE (VÉASE LA NORMA IEC 61508-2)	15
ANEXO B (Informativo) PRESENTACIÓN DE LAS TÉCNICAS Y MEDIDAS PARA LOS SISTEMAS E/E/PE RELACIONADOS CON LA SEGURIDAD: PREVENCIÓN DE LAS DISFUNCIONES SISTEMÁTICAS (VÉANSE LAS NORMAS IEC 61508-2 E IEC 61508-3)	33
ANEXO C (Informativo) PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA LA OBTENCIÓN DE LA INTEGRIDAD DE SEGURIDAD DEL SOFTWARE (VÉASE LA NORMA IEC 61508-3).....	60
ANEXO D (Informativo) UNA APROXIMACIÓN PROBABILÍSTICA PARA DETERMINAR LA INTEGRIDAD DE SEGURIDAD DEL SOFTWARE PARA UN SOFTWARE PREDESARROLLADO	114
ANEXO E (Informativo) PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA EL DISEÑO DE ASIC	119
ANEXO F (Informativo) DEFINICIONES DE LAS PROPIEDADES DE LAS FASES DEL CICLO DE VIDA DEL SOFTWARE.....	134
ANEXO G (Informativo) GUÍA PARA EL DESARROLLO DE SOFTWARE RELACIONADO CON LA SEGURIDAD ORIENTADO A OBJETOS.....	141
BIBLIOGRAFÍA.....	143
ÍNDICE	146
Figura 1 – Estructura global de la serie de Normas IEC 61508.....	13
Tabla C.1 – Recomendaciones aplicables a los lenguajes de programación específicos	92
Tabla D.1 – Histórico necesario para asegurar los niveles de integridad de seguridad.....	114

Tabla D.2 – Probabilidad de disfunción en modo de funcionamiento de baja demanda.....	115
Tabla D.3 – Distancias medias de dos puntos de ensayo.....	116
Tabla D.4 – Probabilidad de disfunción en modo de funcionamiento de alta demanda o continuo.....	117
Tabla D.5 – Probabilidad de ensayo de todas las propiedades del programa	118
Tabla F.1 – Especificación de los requisitos de seguridad del software	134
Tabla F.2 – Diseño y desarrollo del software: diseño de la arquitectura del software.....	135
Tabla F.3 – Diseño y desarrollo del software: herramientas de soporte y lenguaje de programación	136
Tabla F.4 – Diseño y desarrollo del software: diseño detallado.....	136
Tabla F.5 – Diseño y desarrollo del software: ensayo e integración del módulo de software.....	137
Tabla F.6 – Integración de la electrónica programable (hardware y software).....	137
Tabla F.7 – Aspectos software de la validación de la seguridad del sistema	138
Tabla F.8 – Modificación del software	138
Tabla F.9 – Verificación del software.....	139
Tabla F.10 – Evaluación de la seguridad funcional	139
Tabla G.1 – Arquitectura del Software Orientado a objetos	141
Tabla G.2 – Diseño detallado orientado a objetos.....	142
Tabla G.3 – Algunos Términos Orientados a Objeto	142

COMISIÓN ELECTROTÉCNICA INTERNACIONAL

Seguridad funcional de los sistemas eléctricos/electrónicos/ electrónicos programables relacionados con la seguridad Parte 7: Presentación de técnicas y medidas

PRÓLOGO

- 1) IEC (Comisión Electrotécnica Internacional) es una organización mundial para la normalización, que comprende todos los comités electrotécnicos nacionales (Comités Nacionales de IEC). El objetivo de IEC es promover la cooperación internacional sobre todas las cuestiones relativas a la normalización en los campos eléctrico y electrónico. Para este fin y también para otras actividades, IEC publica Normas Internacionales, Especificaciones Técnicas, Informes Técnicos, Especificaciones Disponibles al Público (PAS) y Guías (de aquí en adelante "Publicaciones IEC"). Su elaboración se confía a los comités técnicos; cualquier Comité Nacional de IEC que esté interesado en el tema objeto de la norma puede participar en su elaboración. Organizaciones internacionales gubernamentales y no gubernamentales relacionadas con IEC también participan en la elaboración. IEC colabora estrechamente con la Organización Internacional de Normalización (ISO), de acuerdo con las condiciones determinadas por acuerdo entre ambas.
- 2) Las decisiones formales o acuerdos de IEC sobre materias técnicas, expresan en la medida de lo posible, un consenso internacional de opinión sobre los temas relativos a cada comité técnico en los que existe representación de todos los Comités Nacionales interesados.
- 3) Los documentos producidos tienen la forma de recomendaciones para uso internacional y se aceptan en este sentido por los Comités Nacionales mientras se hacen todos los esfuerzos razonables para asegurar que el contenido técnico de las publicaciones IEC es preciso, IEC no puede ser responsable de la manera en que se usan o de cualquier mal interpretación por parte del usuario.
- 4) Con el fin de promover la unificación internacional, los Comités Nacionales de IEC se comprometen a aplicar de forma transparente las Publicaciones IEC, en la medida de lo posible en sus publicaciones nacionales y regionales. Cualquier divergencia entre la Publicación IEC y la correspondiente publicación nacional o regional debe indicarse de forma clara en esta última.
- 5) IEC no establece ningún procedimiento de marcado para indicar su aprobación y no se le puede hacer responsable de cualquier equipo declarado conforme con una de sus publicaciones.
- 6) Todos los usuarios deberían asegurarse de que tienen la última edición de esta publicación.
- 7) No se debe adjudicar responsabilidad a IEC o sus directores, empleados, auxiliares o agentes, incluyendo expertos individuales y miembros de sus comités técnicos y comités nacionales de IEC por cualquier daño personal, daño a la propiedad u otro daño de cualquier naturaleza, directo o indirecto, o por costes (incluyendo costes legales) y gastos derivados de la publicación, uso o confianza de esta publicación IEC o cualquier otra publicación IEC.
- 8) Se debe prestar atención a las normas para consulta citadas en esta publicación. La utilización de las publicaciones referenciadas es indispensable para la correcta aplicación de esta publicación.
- 9) Se debe prestar atención a la posibilidad de que algunos de los elementos de esta Publicación IEC puedan ser objeto de derechos de patente. No se podrá hacer responsable a IEC de identificar alguno o todos esos derechos de patente.

La Norma IEC 61508-7 ha sido elaborada por el subcomité 65A: Cuestiones relativas a los sistemas, del comité técnico 65 de IEC: Medida, control y automatización de procesos industriales.

Esta segunda edición anula y sustituye a la primera edición publicada en 2000. Esta edición constituye una revisión técnica.

Esta edición se ha sometido a una revisión integral e incorpora muchos comentarios recibidos en las distintas fases de revisión.

El texto de esta norma se basa en los documentos siguientes:

FDIS	Informe de voto
65A/554/FDIS	65A/578/RVD

El informe de voto indicado en la tabla anterior ofrece toda la información sobre la votación para la aprobación de esta norma.

Esta norma ha sido elaborada de acuerdo con las Directivas ISO/IEC, Parte 2.

En la página web de IEC puede encontrarse una lista de todas las partes de la serie de Normas IEC 61508, bajo el título general *Seguridad funcional de los sistemas eléctricos/electrónicos/electrónicos programables relacionados con la seguridad*.

El comité ha decidido que el contenido de esta norma (la norma base y sus modificaciones) permanezca vigente hasta la fecha de mantenimiento indicada en la página web de IEC "<http://webstore.iec.ch>" en los datos relativos a la norma específica. En esa fecha, la norma será

- confirmada;
- anulada;
- reemplazada por una edición revisada; o
- modificada.

INTRODUCCIÓN

Los sistemas que incluyen elementos eléctricos y/o electrónicos se han utilizado durante muchos años para realizar funciones de seguridad en la mayoría de los sectores de aplicación. Los sistemas basados en procesadores electrónicos (generalmente conocidos como Sistemas Electrónicos Programables) se utilizan en todos los sectores de aplicación para realizar funciones no relacionadas con la seguridad, y cada día más se están utilizando para funciones de seguridad. Si se quiere explotar de forma eficaz y segura la tecnología de los sistemas basados en procesadores electrónicos, es imprescindible que los responsables de tomar decisiones tengan suficiente orientación en los aspectos de seguridad en los cuales van a tomar las decisiones.

Esta norma internacional establece una aproximación genérica para todas las actividades relacionadas con el ciclo de vida de seguridad de los sistemas que incluyan elementos eléctricos y/o electrónicos y/o electrónicos programables (E/E/PE) que se utilizan para realizar las funciones de seguridad. Esta propuesta unificada ha sido adoptada con el fin de desarrollar una política técnica racional y consistente relativa a todos los sistemas eléctricos relacionados con la seguridad. Uno de los principales objetivos perseguidos es el de facilitar el desarrollo de normas internacionales de producto y de aplicación sectorial basadas en la serie de Normas IEC 61508.

NOTA 1 En la Bibliografía se dan ejemplos de normas internacionales de producto y de aplicación sectorial basadas en la serie de Normas IEC 61508 (véanse las referencias [21], [22] y [37]).

En la mayoría de los casos, la seguridad se obtiene gracias a un cierto número de sistemas basados en distintas tecnologías (por ejemplo, mecánica, hidráulica, neumática, eléctrica, electrónica, electrónica programable). Por lo tanto, toda estrategia de seguridad debe tener en cuenta no solamente todos los elementos de un sistema de seguridad individual (por ejemplo, sensores, dispositivos de control y actuadores), sino que también debe tener en cuenta todos los sistemas relacionados con la seguridad que forman la combinación completa. Es por ello que esta norma internacional, mientras trata los sistemas eléctricos/electrónicos/electrónicos programables E/E/PE relacionados con la seguridad, también puede proporcionar un marco en el cual pueden considerarse los sistemas relacionados con la seguridad basados en otras tecnologías.

Se reconoce que existe una gran variedad de aplicaciones que utilizan sistemas E/E/PE relacionados con la seguridad en una variedad de sectores de aplicación y que cubren un gran número de grados de complejidad y de posibilidades de peligros y riesgos. Para cada aplicación, las medidas de seguridad requeridas dependerán de muchos factores propios de la aplicación. Esta norma internacional, por ser genérica, permitirá que tales medidas sean trasladadas a las futuras normas internacionales de producto y de aplicación sectorial y a las revisiones de aquellas que ya existan.

Esta norma internacional

- considera globalmente todas las fases relevantes del ciclo de vida de la seguridad de los sistemas E/E/PE y del software (por ejemplo, desde la concepción inicial, pasando por el diseño, la implementación, la explotación y el mantenimiento, hasta la finalización del servicio) donde los E/E/PE realizan funciones de seguridad;
- ha sido elaborada teniendo en cuenta la rápida evolución de la tecnología; el marco que comprende esta norma internacional es suficientemente sólido y extenso como para prever las evoluciones futuras;
- permite la elaboración de normas internacionales de producto y de aplicación sectorial concernientes a los sistemas E/E/PE relacionados con la seguridad; el desarrollo de normas internacionales de producto y de aplicación sectorial a partir de esta norma internacional debería conducir a un alto nivel de coherencia (por ejemplo, principios subyacentes, terminología, etc.) tanto en el seno de cada sector de aplicación, como de un sector a otro; esto proporcionará un beneficio tanto en términos de seguridad como económicos;
- proporciona un método para el desarrollo de la especificación de los requisitos de seguridad necesarios para lograr la seguridad funcional requerida para los sistemas E/E/PE relacionados con la seguridad;
- adopta un planteamiento basado en el riesgo para determinar los requisitos de los niveles de integridad de seguridad;

- introduce los niveles de integridad de seguridad para especificar el nivel a conseguir de integridad de seguridad para las funciones de seguridad que van a realizar los sistemas E/E/PE relacionados con la seguridad;

NOTA 2 Esta norma no especifica los requisitos de nivel de integridad de la seguridad para ninguna función de seguridad, ni establece cómo se determina el nivel de integridad de la seguridad. En vez de ello proporciona un marco conceptual basado en el análisis del riesgo y da técnicas de ejemplo.

- establece tasas de disfunción a alcanzar para las funciones de seguridad realizadas por los sistemas E/E/PE relacionados con la seguridad, que tienen relación con los niveles de integridad de seguridad;
- fija un límite inferior para las tasas de disfunción a alcanzar para una función de seguridad realizada por un único sistema E/E/PE relacionado con la seguridad. Para el caso de sistemas E/E/PE relacionados con la seguridad funcionando:
 - en un modo de baja demanda, el límite inferior se fija en una probabilidad media de disfunción peligrosa bajo demanda de 10^{-5} ;
 - en un modo de funcionamiento continuo o de alta demanda, el límite inferior se fija en una frecuencia media de disfunción peligrosa de $10^{-9} [h^{-1}]$;

NOTA 3 Un sistema E/E/PE único relacionado con la seguridad no implica necesariamente una arquitectura de un solo canal.

NOTA 4 Puede ser posible conseguir diseños de sistemas relacionados con la seguridad con valores más bajos del objetivo de integridad de seguridad en sistemas no complejos, pero estos límites se considera que representan lo que se puede conseguir para sistemas relativamente complejos (por ejemplo sistemas relacionados con la seguridad de electrónica programable) en la actualidad.

- fija requisitos para evitar y controlar fallos sistemáticos, que se basan en la experiencia y juicio obtenidos de la experiencia práctica en la industria. Incluso aunque la probabilidad de ocurrencia no puede ser, en general, cuantificada, la norma permite sin embargo realizar una declaración para una función de seguridad específica que haga que la tasa de disfunción a alcanzar asociada con la función de seguridad se pueda considerar conseguida si se cumplen todos los requisitos de la norma;
- introduce la capacidad sistemática que se aplica a un elemento en relación a su confianza en que la integridad sistemática de seguridad cumpla los requisitos del nivel de integridad de seguridad especificado;
- adopta una amplia gama de principios, técnicas y medidas para conseguir la seguridad funcional de los sistemas E/E/PE relacionados con la seguridad, pero no utiliza explícitamente el concepto de seguro ante fallos. Sin embargo, los conceptos de los principios de “seguridad ante fallos” y de “seguridad inherente” pueden ser aplicables y la adopción de tales conceptos se acepta siempre que se cumplan los apartados pertinentes de la norma.

**Seguridad funcional de los sistemas eléctricos/electrónicos/
electrónicos programables relacionados con la seguridad
Parte 7: Presentación de técnicas y medidas**

1 OBJETO Y CAMPO DE APLICACIÓN

1.1 Esta parte de la Norma IEC 61508 contiene una presentación de diferentes técnicas y medidas de seguridad pertinentes para las Normas IEC 61508-2 e IEC 61508-3.

Las referencias citadas se deberían considerar como referencias básicas de métodos y herramientas, o como ejemplos; pueden no representar el estado de arte.

1.2 Las Normas IEC 61508-1, IEC 61508-2, IEC 61508-3 e IEC 61508-4 son publicaciones básicas de seguridad, aunque esta categoría no sea aplicable en el contexto de los sistemas E/E/PE de baja complejidad relacionados con la seguridad (véase 3.4.3 de la Norma IEC 61508-4). Como publicaciones básicas de seguridad, estas publicaciones están previstas para utilizarse por los comités técnicos para la preparación de normas de acuerdo con los principios contenidos en la Guía IEC 104 y la Guía ISO/IEC 51. Las Normas IEC 61508-1, IEC 61508-2, IEC 61508-3 e IEC 61508-4 también están destinadas a utilizarse como publicaciones autónomas. La función horizontal de seguridad de esta norma internacional no se aplica a los equipos médicos que cumplen la serie de Normas IEC 60601.

1.3 Una de las responsabilidades de un comité técnico es, en la medida de lo posible, utilizar las publicaciones básicas de seguridad para la preparación de sus publicaciones. En este contexto, los requisitos, los métodos de ensayo o las condiciones de ensayo de esta publicación básica de seguridad sólo se aplican si se indican específicamente o se incluyen en las publicaciones preparadas por estos comités técnicos.

1.4 La figura 1 muestra la estructura general de las partes 1 a 7 de la Norma IEC 61508 e indica el papel que la Norma IEC 61508-7 juega en el logro de la seguridad funcional de los sistemas E/E/PE relacionados con la seguridad.

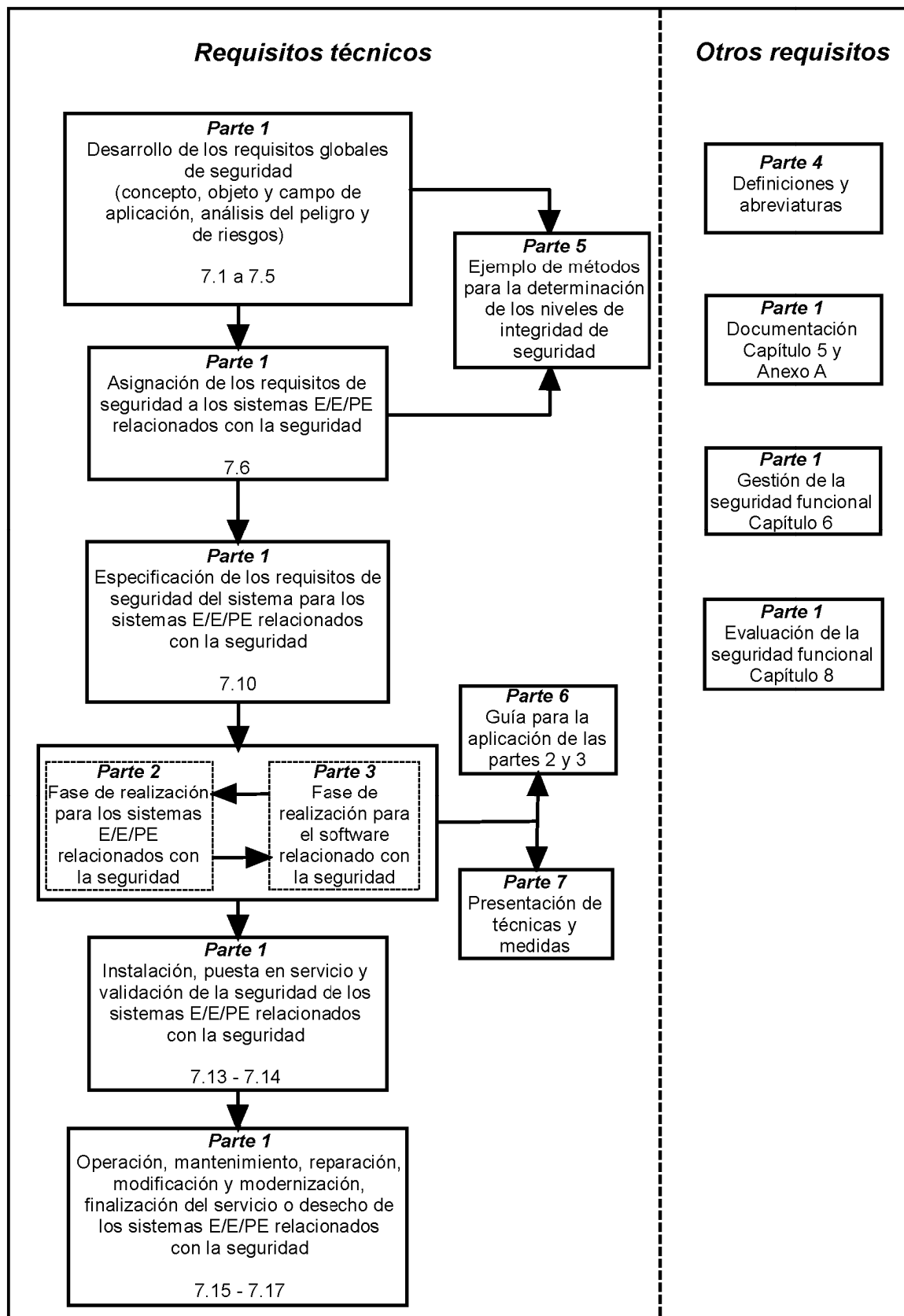


Figura 1 – Estructura global de la serie de Normas IEC 61508

2 NORMAS PARA CONSULTA

Las normas que a continuación se indican son indispensables para la aplicación de esta norma. Para las referencias con fecha, sólo se aplica la edición citada. Para las referencias sin fecha se aplica la última edición de la norma (incluyendo cualquier modificación de ésta).

IEC 61508-4:2010 *Seguridad funcional de los sistemas eléctricos/electrónicos/electrónicos programables relacionados con la seguridad. Parte 4: Definiciones y abreviaturas.*

3 DEFINICIONES Y ABREVIATURAS

Para los fines de este documento, se aplican las definiciones y abreviaturas incluidas en la Norma IEC 61508-4.

ANEXO A (Informativo)**PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA LOS SISTEMAS E/E/PE RELACIONADOS CON LA SEGURIDAD: CONTROL DE LAS DISFUNCIONES ALEATORIAS DEL HARDWARE (VÉASE LA NORMA IEC 61508-2)****A.1 Eléctricos**

Objetivo general: Controlar las disfunciones de los componentes electromecánicos.

A.1.1 Detección de disfunciones por supervisión en línea

NOTA Esta técnica/medida se menciona en las tablas A.2, A.3, A.7 y A.13 a A.18 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones supervisando el comportamiento del sistema E/E/PE relacionado con la seguridad en respuesta a la operación normal (en línea) del equipo bajo control (EUC).

Descripción: En ciertas condiciones, las disfunciones se pueden detectar gracias a la información sobre, por ejemplo, el comportamiento temporal del EUC. Por ejemplo, si un conmutador, que forma parte del sistema E/E/PE relacionado con la seguridad, está normalmente controlado por el EUC y no cambia de estado en el momento dado, se ha detectado una disfunción. Generalmente no es posible localizar la disfunción.

A.1.2 Supervisión de los contactos de relés

NOTA Esta técnica/medida se menciona en las tablas A.2 y A.14 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones, de soldadura por ejemplo, de contactos de relés.

Descripción: Los relés de contactos forzados (o de contacto dirigido positivamente) se diseñan de forma que sus contactos estén rígidamente unidos. Suponiendo que hay dos conjuntos de contactos inversores *a* y *b*, si el contacto normalmente abierto, *a*, se suelda, el contacto normalmente cerrado, *b*, no se puede cerrar cuando la bobina del relé ya no está controlada. En consecuencia, la supervisión del cierre del contacto *b* normalmente cerrado cuando la bobina del relé ya no está controlada puede servir para probar que el contacto *a* normalmente abierto está abierto. La disfunción del cierre del contacto *b* normalmente cerrado indica una disfunción del contacto *a*; el circuito de supervisión debería garantizar una parada de seguridad o que la parada de seguridad se mantenga para toda máquina controlada por el contacto *a*.

Referencias:

Zusammenstellung und Bewertung elektromechanischer Sicherheitsschaltungen für Verriegelungseinrichtungen. F. Kreutzkampff, W. Hertel, Sicherheitstechnisches Informations- und Arbeitsblatt 330212, BIA-Handbuch. 17. Lfg. X/91, Erich Schmidt Verlag, Bielefeld.
www.BGIA-HANDBUCHdigital.de/330212

A.1.3 Comparadores

NOTA Esta técnica/medida se menciona en las tablas A.2, A.3 y A.4 de la Norma IEC 61508-2.

Objetivo: Detectar, lo antes posible, las disfunciones (no simultáneas) en una unidad de procesamiento independiente o en el comparador.

Descripción: Las señales de unidades de procesamiento independientes se comparan cíclicamente o continuamente con un comparador hardware. El comparador puede ensayarse externamente o utilizar una tecnología de automonitorización. Las diferencias detectadas en el comportamiento de los procesadores generan un mensaje de disfunción.

A.1.4 Dispositivo de voto mayoritario

NOTA Esta técnica/medida se menciona en las tablas A.2, A.3 y A.4 de la Norma IEC 61508-2.

Objetivo: Detectar y enmascarar las disfunciones en uno de al menos tres canales del hardware.

Descripción: Un dispositivo de voto mayoritario (2 sobre 3 ó 3 sobre 3, o m sobre n) se utiliza para detectar y enmascarar las disfunciones. El dispositivo de voto puede ensayarse externamente o utilizar una tecnología de auto-monitorización.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

A.1.5 Principio de corriente en reposo (sin energía para iniciar la activación)

NOTA Esta técnica/medida se menciona en la tabla A.16 de la Norma IEC 61508-2.

Objetivo: Ejecutar la función de seguridad cuando la alimentación se corta o se pierde.

Descripción: La función de seguridad se ejecuta si los contactos están abiertos y la corriente no pasa. Por ejemplo, si los frenos se utilizan para parar un movimiento peligroso de un motor, los frenos se abren por el cierre de los contactos en el sistema relacionado con la seguridad y se cierran al abrir los contactos en el sistema relacionado con la seguridad.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

A.2 Electrónicos

Objetivo general: Controlar las disfunciones en los componentes semiconductores.

A.2.1 Ensayos por el hardware redundante

NOTA Esta técnica/medida se menciona en las tablas A.3, A.15, A.16 y A.18 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones con la ayuda del hardware redundante, es decir, utilizando un hardware suplementario no necesario para la ejecución de las funciones del proceso.

Descripción: El hardware redundante se puede utilizar para ensayar a una frecuencia adecuada las funciones de seguridad especificadas. Esta aproximación es normalmente necesaria para realizar los apartados A.1.1 o A.2.2.

A.2.2 Principios dinámicos

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones estáticas con un procesamiento dinámico de las señales.

Descripción: Un cambio forzado de señales normalmente estáticas (generadas en el interior o en el exterior) ayuda a la detección de disfunciones estáticas de componentes. Esta técnica se asocia a menudo con componentes electromecánicos.

Referencia:

Elektronik in der Sicherheitstechnik. H. Jürs, D. Reinert, Sicherheitstechnisches Informationsund Arbeitsblatt 330220, BIA-Handbuch, Erich-Schmidt Verlag, Bielefeld, 1993.
<http://www.bgia-handbuchdigital.de/330220>

A.2.3 Puerto de acceso de ensayo normalizado y arquitectura de ensayo de barrido del conjunto de las uniones

NOTA Esta técnica/medida se menciona en las tablas A.3, A.15 y A.18 de la Norma IEC 61508-2.

Objetivo: Controlar y observar qué sucede en cada patilla de un circuito integrado (CI):

Descripción: El ensayo de barrido del conjunto de las uniones es una técnica de diseño de CI que aumenta la posibilidad de ensayar el CI resolviendo el problema de acceso a los puntos de ensayo situados en el interior de él. En un CI típico con barrido del conjunto de las uniones, compuesto de un núcleo lógico y de registros de entrada/salida, se coloca un registro de desplazamiento entre el núcleo lógico y los registros de entrada/salida adyacentes a cada patilla del CI. Cada registro de desplazamiento se sitúa en una celda de barrido. La celda de barrido puede controlar y observar lo que pasa en cada patilla de entrada y de salida de un CI, a través del puerto de acceso para los ensayos normalizados. El ensayo interno del núcleo lógico del CI se realiza aislando el núcleo lógico de los estímulos recibidos de los componentes de alrededor y realizando a continuación un ensayo automático interno. Estos ensayos pueden utilizarse para detectar las disfunciones en el CI.

Referencia:

IEEE 1149-1:2001, *IEEE standard test access port and boundary-scan architecture*, IEEE Computer Society, 2001, ISBN: 0-7381-2944-5.

A.2.4 (No utilizado)**A.2.5 Redundancia supervisada**

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones, proporcionando varias unidades funcionales, supervisando el comportamiento de cada una de ellas para detectar las disfunciones e iniciar una transición hacia un estado seguro si se detecta una discrepancia en su comportamiento.

Descripción: La función de seguridad se ejecuta por al menos dos canales del hardware. Las salidas de estos canales están supervisadas y se inicia un estado seguro si se detecta un fallo (es decir, si las señales de salida de todos los canales no son las mismas).

Referencias:

Elektronik in der Sicherheitstechnik. H. Jürs, D. Reinert, Sicherheitstechnisches Informationsund Arbeitsblatt 330220, BIA-Handbuch, Erich-Schmidt Verlag, Bielefeld, 1993.
<http://www.bgia-handbuchdigital.de/330220>

Dependability of Critical Computer Systems 1. F. J. Redmill, Elsevier Applied Science, 1988, ISBN 1-85166-203-0.

A.2.6 Componentes eléctricos/electrónicos con chequeo automático

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-2.

Objetivo: Detectar los fallos con controles periódicos de las funciones de seguridad.

Descripción: El hardware se ensaya antes de empezar el proceso y se ensaya de forma repetida en intervalos adecuados. El EUC continúa funcionando solo si cada ensayo es satisfactorio.

Referencias:

Elektronik in der Sicherheitstechnik. H. Jürs, D. Reinert, Sicherheitstechnisches Informationsund Arbeitsblatt 330220, BIA-Handbuch, Erich-Schmidt Verlag, Bielefeld, 1993.
<http://www.bgia-handbuchdigital.de/330220>

Dependability of Critical Computer Systems 1. F. J. Redmill, Elsevier Applied Science, 1988, ISBN 1-85166-203-0.

A.2.7 Supervisión de la señal analógica

NOTA Esta técnica/medida se menciona en las tablas A.3 y A.13 de la Norma IEC 61508-2.

Objetivo: Mejorar la fiabilidad de las señales medidas.

Descripción: Siempre que haya una opción, las señales analógicas se utilizan preferentemente a los estados digitales de todo o nada. Por ejemplo, las maniobras o los estados seguros se representan por los niveles de las señales analógicas, generalmente con una supervisión de la tolerancia del nivel de la señal. La técnica proporciona la continuidad de la supervisión y una mejora del nivel de confianza en los transmisores, reduciendo la frecuencia del ensayo periódico de la función del transmisor. Las interfaces externas, por ejemplo las líneas de impulsos, necesitan también un ensayo.

A.2.8 Devaluación

NOTA Esta técnica/medida se menciona en el apartado 7.4.2.13 de la Norma IEC 61508-2.

Objetivo: Aumentar la fiabilidad de los componentes del hardware.

Descripción: Los componentes del hardware funcionan en niveles garantizados por el diseño del sistema, que están bastante por debajo de sus características máximas de especificación. La devaluación es la práctica que garantiza que, en todas las condiciones de funcionamiento normales, los componentes funcionan bastante por debajo de sus niveles de limitación máxima.

A.3 Unidades de procesamiento

Objetivo general: Reconocer las disfunciones que son el origen de los resultados incorrectos en las unidades de procesamiento.

A.3.1 Autotests mediante el software: Número limitado de configuraciones (un canal)

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-2.

Objetivo: Detectar lo antes posible las disfunciones de la unidad de procesamiento.

Descripción: El hardware se construye con las técnicas estándar que no tienen en cuenta ninguna exigencia especial relacionada con la seguridad. La detección de las disfunciones se realiza completamente con las funciones del software suplementarias que realizan autotests utilizando al menos dos configuraciones de datos complementarias (por ejemplo, 55hex y AAhex).

A.3.2 Autotests mediante el software: bit deslizante (un canal)

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-2.

Objetivo: Detectar lo antes posible las disfunciones de la memoria física (por ejemplo los registros) y del decodificador de instrucciones de la unidad de procesamiento.

Descripción: La detección de disfunciones se realiza completamente con las funciones del software suplementarias que realizan autotests utilizando una configuración de datos (por ejemplo la configuración de bit deslizante) que ensaya el almacenamiento físico (registros de datos y direcciones) y el decodificador de instrucciones. De todos modos, la cobertura del diagnóstico solo es del 90%.

A.3.3 Autotest soportado por el hardware (un canal)

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-2.

Objetivo: Detectar lo antes posible las disfunciones de la unidad de procesamiento con un hardware especial que aumente la velocidad y el campo de detección de las disfunciones.

Descripción: Las instalaciones de hardware especiales suplementarias soportan las funciones de autotest para la detección de disfunciones. Esto podría ser, por ejemplo, una unidad de hardware que vigile de forma cíclica la salida de una cierta configuración de bits según el principio del perro guardián.

A.3.4 Procesamiento codificado (un canal)

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-2.

Objetivo: Detectar lo antes posible las disfunciones de la unidad de procesamiento.

Descripción: Las unidades de procesamiento se pueden diseñar con técnicas de circuitos especiales que reconocen o corrigen las disfunciones. A día de hoy, estas técnicas solo se aplican a los circuitos relativamente simples y no están muy extendidas; de todos modos no se deberían excluir los desarrollos futuros.

Referencias:

Le processeur codé: un nouveau concept appliqué à la sécurité des systèmes de transports. Gabriel, Martin, Wartski, Revue Générale des chemins de fer, No. 6, June 1990.

Vital Coded Microprocessor Principles and Application for Various Transit Systems. P. Forin, IFAC Control Computers Communications in Transportation, 79-84, 1989.

A.3.5 Comparación recíproca mediante el software

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-2.

Objetivo: Detectar lo antes posible las disfunciones de la unidad de procesamiento por comparación dinámica mediante el software.

Descripción: Dos unidades de procesamiento se transmiten recíprocamente los datos (incluyendo los resultados, los resultados intermedios y los datos de ensayo). Se realiza una comparación de los datos con la ayuda de un software en cada unidad y las diferencias detectadas generan un mensaje de disfunción.

A.4 Rangos de memoria invariable

Objetivo general: La detección de las modificaciones de las informaciones en la memoria invariable.

A.4.1 Redundancia multibit con protección de palabra (por ejemplo, supervisión de la ROM con un código de Hamming modificado)

NOTA 1 Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-2.

NOTA 2 Véase también el apartado A.5.6 “Supervisión de la RAM con un código de Hamming modificado, o detección de las disfunciones de los datos con los códigos de detección/corrección de errores (EDC)” y el apartado C.3.2 “Detección de errores y códigos de corrección”.

Objetivo: Detectar todas las disfunciones de un solo bit, todas las disfunciones de dos bits, algunas disfunciones de tres bits y algunas disfunciones de todos los bits en una palabra de 16 bits.

Descripción: Cada palabra de memoria se extiende en varios bits redundantes para producir un código de Hamming modificado con una distancia de Hamming de al menos 4. Cada vez que una palabra se lee, la verificación de los bits redundantes puede determinar si una corrupción ha tenido lugar o no. Si se detecta una diferencia, se produce un mensaje de disfunción. El procedimiento puede también servir para detectar las disfunciones de dirección calculando los bits redundantes para la concatenación de la palabra de datos y de su dirección.

Referencias:

Prüfbar und korrigierbare Codes. W. W. Peterson, München, Oldenburg, 1967.

Error detecting and error correcting codes. R. W. Hamming, The Bell System Technical Journal 29 (2), 147-160, 1950.

A.4.2 Suma de control modificada

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma 61508-2.

Objetivo: Detectar todas las disfunciones de los bits impares, es decir, aproximadamente el 50% de todas las disfunciones de bits posibles.

Descripción: Se crea una suma de control con un algoritmo adecuado que utiliza todas las palabras de un bloque de memoria. La suma de control puede almacenarse como una palabra suplementaria en la ROM o puede añadirse una palabra suplementaria al bloque de memoria para asegurarse de que el algoritmo de la suma de control produce un valor predeterminado. En un ensayo de memoria posterior, se crea una suma de control de nuevo con el mismo algoritmo y el resultado se compara con el valor almacenado o definido. Si se detecta una diferencia, se produce un mensaje de disfunción.

A.4.3 Firma de una sola palabra (8 bits)

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-2.

Objetivo: Detectar todas las disfunciones de un bit y todas las disfunciones multibits en una palabra, así como aproximadamente el 99,6% de todas las disfunciones de bits posibles.

Descripción: El contenido de un bloque de memoria se comprime (bien con el hardware o bien con el software) con la ayuda de un algoritmo de control cíclico redundante (CRC) en una sola palabra de memoria. Un algoritmo CRC típico trata todo el contenido del bloque como un flujo de datos en serie de octetos o de bits sobre el cual se realiza una división polinomial continua con la ayuda de un generador polinomial. El resto de la división representa el contenido comprimido de la memoria - es la firma de la memoria - y se almacena. La firma se recalcula en los ensayos posteriores y se compara con la almacenada. Si existe una diferencia se produce un mensaje de disfunción.

A.4.4 Firma de una palabra doble (16 bits)

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma 61508-2.

Objetivo: Detectar todas las disfunciones de un bit y todas las disfunciones multibits en una palabra, así como aproximadamente el 99,998% de todas las disfunciones de bits posibles.

Descripción: Este procedimiento calcula una firma con la ayuda de un algoritmo de control cíclico redundante (CRC), pero el valor obtenido tiene un tamaño de al menos dos palabras. La firma extendida se almacena, se recalcula y se compara como en el caso de una sola palabra. Si existe una diferencia entre las firmas almacenadas y calculadas, se produce un mensaje de disfunción.

A.4.5 Réplica de bloque (por ejemplo, doble ROM con comparación por hardware o software)

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-2.

Objetivo: Detectar todas las disfunciones de bit.

Descripción: El espacio de direcciones se duplica en dos memorias. La primera memoria se gestiona de la manera normal. La segunda memoria contiene las mismas informaciones y está accesible en paralelo con la primera. Las salidas se comparan y si se detecta una diferencia se produce un mensaje de disfunción. Para detectar ciertos tipos de errores de bit, los datos se deben almacenar invertidos en una de las dos memorias y se invierten de nuevo cuando se leen.

A.5 Rangos de memoria variable

Objetivo general: Detectar las disfunciones durante el direccionamiento, la escritura, el almacenamiento y la lectura.

NOTA Los errores intermitentes se listan en la tabla A.1 de la Norma IEC 61508-2 como los fallos a detectar durante la operación o a ser analizados en la obtención de la fracción de disfunción segura. Las causas de errores intermitentes son: (1) partículas alfa del decaimiento del empaquetado, (2) neutrones, (3) ruido externo de interferencia electromagnética, (4) diafonía interna. El ruido externo de interferencia electromagnética está cubierto por otros requisitos de esta norma internacional.

El efecto de las partículas alfa y de los neutrones debería controlarse mediante medidas de integridad de seguridad durante el tiempo de ejecución. Las medidas eficaces de integridad de seguridad para los errores de hardware pueden no ser eficaces para los errores intermitentes, por ejemplo los ensayos de RAM como walk-path o galpat no son eficaces, mientras que las técnicas de supervisión como la Paridad y ECC con lectura recurrente de las celdas de memoria lo son.

Los errores intermitentes se producen cuando un evento de radiación causa la suficiente perturbación de carga como para invertir el estado de los datos de una celda de memoria de semiconductor de baja energía, registros, latch o biestable. El error se llama "intermitente" porque el propio circuito no queda permanentemente dañado por la radiación. Los errores intermitentes se clasifican en perturbaciones de un solo bit (SBU, Single Bit Upsets) o perturbaciones aisladas (SEU, Single Event Upsets) y perturbaciones multibit (MBU, Multi-Bit Upsets).

Si el circuito perturbado es un elemento de almacenamiento como una celda de memoria o un biestable, el estado se almacena hasta la siguiente operación (intencionada) de escritura. Los nuevos datos serán almacenados correctamente. En un circuito combinado el efecto es un régimen transitorio en el sistema puesto que hay un flujo continuo de energía desde el componente que alimenta dicho nodo. Sobre los cables de conexión y las líneas de comunicación el efecto también podría ser un régimen transitorio. Sin embargo, debido a la mayor capacitancia el efecto por partículas alfa y neutrones se considera despreciable.

Los errores intermitentes pueden ser relevantes para una memoria variable de cualquier clase, es decir, para DRAM, SRAM, bancos de registros en microprocesadores (μP), caché, pipelines, registros de configuración de dispositivos tales como ADC, DMA, MMU, controlador de interrupción, temporizadores complejos. La sensibilidad a las partículas alfa y neutrones es una función de la tensión del núcleo y de la geometría. Las geometrías más pequeñas a tensiones de núcleo de 2,5 V y especialmente por debajo de 1,8 V exigirían más evaluación y más medidas eficaces de protección.

Se sabe que la tasa de errores intermitentes (véanse los puntos a) e i) a continuación) está en el rango de 700 Fit/Mbit a 1 200 Fit/Mbit para memorias (embebidas). Este es un valor de referencia a comparar con los datos procedentes del proceso del silicio con el que se implementa el dispositivo. Hasta hace poco los SBU se consideraron dominantes, pero la última previsión (véase el punto a) a continuación) informa de un porcentaje creciente de MBU en la tasa global de errores intermitentes (SER) para tecnologías desde los 65 nm e inferiores.

La siguiente literatura y fuentes dan detalles acerca de los errores intermitentes:

- Altitude SEE Test European Platform (ASTEP) and First Results in CMOS 130 nm SRAM. J-L. Autran, P. Roche, C. Sudre et al. Nuclear Science, IEEE Transactions on Volume 54, Issue 4, Aug. 2007 Page(s):1002 - 1009.
- Radiation-Induced Soft Errors in Advanced Semiconductor Technologies, Robert C. Baumann, Fellow, IEEE, IEEE TRANSACTIONS ON DEVICE AND MATERIALS RELIABILITY, VOL. 5, NO. 3, SEPTEMBER 2005.
- Soft errors' impact on system reliability, Ritesh Mastipuram and Edwin C Wee, Cypress Semiconductor, 2004.
- Trends And Challenges In VLSI Circuit Reliability, C. Costantinescu, Intel, 2003, IEEE Computer Society.

- e) Basic mechanisms and modeling of single-event upset in digital microelectronics, P. E. Dodd and L. W. Massengill, IEEE Trans. Nucl. Sci., vol. 50, no. 3, pp. 583–602, Jun. 2003.
- f) Destructive single-event effects in semiconductor devices and ICs, F. W. Sexton, IEEE Trans. Nucl. Sci., vol. 50, no. 3, pp. 603–621, Jun. 2003.
- g) Coming Challenges in Microarchitecture and Architecture, Ronen, Mendelson, Proceedings of the IEEE, Volume 89, Issue 3, Mar 2001 Page(s):325 – 340.
- h) Scaling and Technology Issues for Soft Error Rates, A Johnston, 4th Annual Research Conference on Reliability Stanford University, October 2000.
- i) International Technology Roadmap for Semiconductors (ITRS), several paper.

A.5.1 Ensayo de RAM “damero” o “marcha”

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones de bit predominantemente estáticas.

Descripción: Se escribe un patrón de tipo "damero" de 0 y de 1 en las celdas de una memoria de bits. Las celdas se inspeccionan después por pares para asegurar que los contenidos son los mismos y correctos. La dirección de la primera celda del par es variable y la dirección de la segunda celda está formada por la inversión bit a bit de la primera dirección. En la primera pasada, el rango de direcciones de la memoria se incrementa hacia las direcciones superiores a partir de la dirección variable y, posteriormente, se disminuye hacia las direcciones inferiores. Las dos pasadas se repiten con la distribución previa invertida. Si existe alguna diferencia se produce un mensaje de disfunción.

En el ensayo de “marcha” de una RAM, las celdas de una memoria de bits se inicializan con una cadena binaria uniforme. En la primera pasada, las celdas se inspeccionan en orden ascendente: el contenido de cada celda se comprueba y se invierte. Este contenido de base, que se crea durante la primera pasada, se trata en la segunda pasada en orden descendente y de la misma forma. Las dos primeras pasadas se repiten con una distribución previa invertida en una tercera o cuarta pasada. Si existe una diferencia se produce un mensaje de disfunción.

A.5.2 Ensayo de RAM “walkpath”

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones estáticas y dinámicas de bits y las interacciones entre las celdas de memoria.

Descripción: El rango de memoria a ensayar se inicializa con una cadena binaria uniforme. La primera celda se invierte y se inspecciona la zona de memoria restante para asegurar que su contenido es correcto. Después, la primera celda se invierte de nuevo para volver a su valor inicial y todo el procedimiento se repite para la celda siguiente. Se realiza una segunda pasada del “modelo del bit errante” con una distribución invertida del contenido previo. Si existe una diferencia se produce un mensaje de disfunción.

A.5.3 Ensayo de RAM “galpat” o “galpat transparente”

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones estáticas de bit y una gran parte de los acoplamientos dinámicos

Descripción: En el ensayo de RAM “galpat”, el rango de memoria elegido se inicializa primero uniformemente (es decir, todo a 0 o todo a 1). La primera celda de memoria a ensayar se invierte y todas las celdas restantes se inspeccionan para asegurar que su contenido es correcto. Después de cada acceso de lectura a las celdas restantes, la celda invertida también se comprueba. Este procedimiento se repite para cada celda en el rango de memoria elegido. Se realiza una segunda pasada con la inicialización opuesta. Toda diferencia genera un mensaje de disfunción.

El ensayo “galpat transparente” es una variante del procedimiento anterior: en vez de inicializar todas las celdas en el rango de memoria elegido, los contenidos existentes permanecen invariables y se utilizan las firmas para comparar los contenidos de los conjuntos de celdas. Se selecciona la primera celda a ensayar en el rango elegido y se calcula y se registra la firma S1 de todas las celdas restantes en el rango. La celda a ensayar se invierte y se recalcula la firma S2 de todas las celdas restantes. (Después de cada acceso de lectura a las celdas restantes, la celda invertida también se comprueba). S2 se compara con S1 y toda diferencia genera un mensaje de disfunción. La celda ensayada se invierte de nuevo para restablecer el contenido de partida y se recalcula la firma S3 de todas las celdas restantes y se compara con S1. Cualquier diferencia produce un mensaje de disfunción. Todas las celdas de memoria en el rango elegido se inspeccionan de la misma forma.

A.5.4 Ensayo de RAM “Abraham”

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar todas las disfunciones de unión y acoplamiento entre las celdas de memoria.

Descripción: La proporción de fallos detectados excede la del ensayo de RAM “galpat”. El número de operaciones requerido para realizar el ensayo de toda la memoria es aproximadamente $30n$, siendo n el número de celdas de la memoria. El ensayo se puede hacer de forma transparente para utilizarlo durante el ciclo de operación partiendo la memoria y ensayando cada parte en segmentos de tiempo diferentes.

Referencia:

Efficient Algorithms for Testing Semiconductor Random-Access Memories. R. Nair, S. M. Thatte, J. A. Abraham, IEEE Trans. Comput. C-27 (6), 572-576, 1978.

A.5.5 Redundancia de un bit (por ejemplo, supervisión de la RAM con un bit de paridad)

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar el 50% de todas las disfunciones de bit posibles en el rango de memoria ensayado.

Descripción: Cada palabra de la memoria se extiende en un bit (el bit de paridad) que completa cada palabra con un número par o impar de niveles lógicos “1”. La paridad de la palabra de datos se comprueba cada vez que se lee. Si se encuentra un número erróneo de 1, se produce un mensaje de disfunción. La elección de una paridad par o impar debería ser tal que la más desfavorable de las palabras cero (únicamente 0) y de las palabras uno (únicamente 1) en caso de disfunción no sea un código válido. La paridad también puede servir para detectar las disfunciones de direccionamiento cuando la paridad se calcula para la concatenación de la palabra de datos y de su dirección.

A.5.6 Supervisión de la RAM con un código de Hamming modificado, o detección de las disfunciones de los datos con los códigos de detección/corrección de errores (EDC)

NOTA 1 Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

NOTA 2 Véase también el apartado A.4.1 “Redundancia multibit con protección de palabra (por ejemplo, supervisión de la ROM con un código de Hamming modificado)” y el apartado C.3.2 “Detección de errores y códigos de corrección”.

Objetivo: Detectar todas las disfunciones impares de bit, todas las disfunciones de dos bits, algunas disfunciones de tres bits y multibits.

Descripción: Cada acceso a la memoria se extiende con varios bits redundantes para obtener un código de Hamming modificado con una distancia de Hamming de al menos cuatro. Cada vez que se leen los datos, se puede determinar si ha ocurrido una alteración comprobando los bits redundantes. Si existe una diferencia, se produce un mensaje de disfunción. El procedimiento también puede servir para detectar una disfunción de direccionamiento cuando los bits redundantes se calculan para la concatenación de los datos y de su dirección.

Referencias:

Prüfbare und korrigierbare Codes. W. W. Peterson, München, Oldenburg, 1967.

Error detecting and error correcting codes. R. W. Hamming, The Bell System Technical Journal 29 (2), 147-160, 1950.

A.5.7 Doble RAM con comparación por hardware o software y ensayo de lectura/escritura

NOTA Esta técnica/medida se menciona en la tabla A.6 de la Norma IEC 61508-2.

Objetivo: Detectar todas las disfunciones de bit.

Descripción: El espacio de direcciones se duplica en dos memorias. La primera memoria se gestiona normalmente. La segunda memoria contiene las mismas informaciones y se accede en paralelo con la primera. Las salidas se comparan y se produce un mensaje de disfunción si se detecta una diferencia. Para detectar ciertos tipos de errores de bit, los datos se deben almacenar invertidos en una de las dos memorias y se invierten de nuevo cuando se leen.

A.6 Unidades e interfaces de E/S (comunicación externa)

Objetivo general: Detectar las disfunciones de las unidades de entrada y de salida (digitales, analógicas, en serie o en paralelo) y prevenir el envío a los procesos de salidas inadmisibles.

A.6.1 Patrón de ensayo

NOTA Esta técnica/medida se menciona en las tablas A.7, A.13 y A.14 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones estáticas (disfunciones debidas a un bloqueo) y las diafonías.

Descripción: Se trata de un ensayo cíclico independiente del flujo de datos de las unidades de entrada y de salida. Utiliza un patrón de ensayo definido para comparar las observaciones con los valores previstos correspondientes. Las informaciones del patrón de ensayo, la recepción del patrón y la evaluación del patrón de ensayo deben ser independientes las unas de las otras. El EUC no debería verse influenciado por el patrón de ensayo de forma inadmisibile.

A.6.2 Protección por código

NOTA Esta técnica/medida se menciona en las tablas A.7, A.15, A.16 y A.18 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones sistemáticas y aleatorias del hardware en los flujos de datos de entrada/salida.

Descripción: Este procedimiento protege las informaciones de entrada y de salida contra las disfunciones sistemáticas y aleatorias del hardware. La protección por código permite una detección de las disfunciones dependiente del flujo de datos en las unidades de entrada y de salida, basada en la redundancia de las informaciones y/o en la redundancia temporal. Típicamente, la información redundante se superpone a los datos de entrada y de salida. Esto proporciona un medio de supervisión del funcionamiento correcto de los circuitos de entrada o de salida. Numerosas técnicas son posibles, por ejemplo se puede superponer una frecuencia portadora a la señal de salida de un sensor. La unidad de procesamiento lógico puede verificar la presencia de la frecuencia portadora o se puede añadir un código binario redundante a un canal de salida con el fin de permitir la supervisión de la validez de una señal que pasa entre la unidad lógica y el accionador final.

A.6.3 Salida paralela multicanal

NOTA Esta técnica/medida se menciona en la tabla A.7 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones aleatorias del hardware (disfunciones debidas a un bloqueo), las disfunciones debidas a las influencias externas, las disfunciones de sincronización, las disfunciones de direccionamiento, las disfunciones debidas a una deriva y las disfunciones transitorias.

Descripción: Es una salida paralela de varios canales que depende del flujo de datos, con salidas independientes para la detección de las disfunciones aleatorias del hardware. La detección de las disfunciones se realiza con comparadores externos. Si se produce una disfunción, el EUC se para directamente. Esta medida solo es eficaz si el flujo de datos cambia durante el intervalo del ensayo de diagnóstico.

A.6.4 Salidas supervisadas

NOTA Esta técnica/medida se menciona en la tabla A.7 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones individuales, las disfunciones debidas a las influencias externas, las disfunciones de sincronización, las disfunciones de direccionamiento, las disfunciones debidas a una deriva (para las señales analógicas) y las disfunciones transitorias.

Descripción: Es una comparación dependiente del flujo de datos de las salidas, con entradas independientes para garantizar la conformidad con un rango de tolerancias definido (tiempo, valores). No siempre se puede relacionar una disfunción detectada con una salida defectuosa. Esta medida solo es eficaz si el flujo de datos cambia durante el intervalo del ensayo de diagnóstico.

A.6.5 Comparación/voto mayoritario sobre las entradas

NOTA Esta técnica/medida se menciona en las tablas A.7 y A.13 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones individuales, las disfunciones debidas a las influencias externas, las disfunciones de sincronización, las disfunciones de direccionamiento, las disfunciones debidas a una deriva (para las señales analógicas) y las disfunciones transitorias.

Descripción: Es una comparación dependiente del flujo de datos de las entradas independientes, para asegurar la conformidad con un rango de tolerancias definido (tiempo, valores). Habrá redundancias de 1 entre 2, 2 entre 3 o más. Esta medida solo es eficaz si el flujo de datos cambia durante el intervalo del ensayo de diagnóstico.

A.7 Rutas de datos (comunicación interna)

Objetivo general: Detectar las disfunciones debidas a un defecto en la transferencia de información.

A.7.1 Redundancia del hardware sobre un bit

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar todas las disfunciones impares de bit, es decir, el 50% de todas las disfunciones de bit posibles en el flujo de datos.

Descripción: El bus se completa con una línea (bit) y esta línea suplementaria (bit) se utiliza para detectar las disfunciones por un control de paridad.

A.7.2 Redundancia del hardware sobre varios bits

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones durante la comunicación por el bus y en los enlaces de transmisiones en serie.

Descripción: El bus se completa con dos o más líneas (bits) y estas líneas suplementarias (bits) se utilizan para detectar las disfunciones con las técnicas del código de Hamming.

A.7.3 Redundancia completa del hardware

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones durante la comunicación comparando las señales de dos buses.

Descripción: El bus se dobla y las líneas suplementarias (bits) se utilizan para detectar las disfunciones.

A.7.4 Inspección utilizando los patrones de ensayo

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones estáticas (disfunciones debidas a un bloqueo) y las diafonías.

Descripción: Se trata de un ensayo cíclico, independiente del flujo de datos, de las rutas de datos. Utiliza un patrón de ensayo definido para comparar las observaciones con los valores previstos correspondientes.

El contenido del patrón de ensayo, la recepción del patrón de ensayo y la evaluación del patrón de ensayo deben ser independientes los unos de los otros. El EUC no debería verse influenciado de forma inadmisibile por el patrón de ensayo.

A.7.5 Redundancia de transmisión

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones transitorias en la comunicación por el bus.

Descripción: Las informaciones se transfieren de forma secuencial varias veces. La repetición solo es eficaz contra las disfunciones transitorias.

A.7.6 Redundancia de información

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones en la comunicación por el bus.

Descripción: Los datos se transmiten en paquetes, acompañados de una suma de control calculada por cada uno de los paquetes. El receptor vuelve a calcular la suma de control correspondiente a los datos recibidos y compara los resultados con las sumas de control recibidas.

A.8 Alimentación

Objetivo general: Detectar o tolerar las disfunciones debidas a un defecto en la alimentación.

A.8.1 Protección contra las sobretensiones con parada de seguridad

NOTA Esta técnica/medida se menciona en la tabla A.9 de la Norma IEC 61508-2.

Objetivo: Proteger el sistema relacionado con la seguridad contra las sobretensiones.

Descripción: Una sobretensión se detecta suficientemente pronto como para que todas las salidas se conmuten a un estado seguro por el subprograma de puesta fuera de tensión o se conmuten una segunda unidad de alimentación.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

A.8.2 Supervisión de la tensión (secundaria)

NOTA Esta técnica/medida se menciona en la tabla A.9 de la Norma IEC 61508-2.

Objetivo: Supervisar las tensiones secundarias e iniciar un estado seguro si la tensión no está en el rango especificado.

Descripción: La tensión secundaria se supervisa y se inicia una desconexión o una conmutación a una segunda unidad de alimentación si la tensión no está en el rango especificado.

A.8.3 Puesta fuera de tensión con parada de seguridad

NOTA Esta técnica/medida se menciona en la tabla A.9 de la Norma IEC 61508-2.

Objetivo: Cortar la alimentación registrando todas las informaciones críticas relacionadas con la seguridad.

Descripción: La sobretensión o la subtensión se detecta lo suficientemente pronto como para que el estado interno se pueda guardar en una memoria no volátil (si fuera necesario), y para que todas las salidas se puedan poner en estado seguro por el subprograma de puesta fuera de tensión, o para que todas las salidas se puedan conmutar a un estado seguro por el subprograma de puesta fuera de tensión, o que se produzca una conmutación a una segunda unidad de alimentación.

A.9 Supervisión temporal y lógica de la secuencia del programa

NOTA Este grupo de técnicas y medidas se menciona en las tablas A.15, A.16 y A.18 de la Norma IEC 61508-2.

Objetivo general: Detectar una secuencia de programa defectuosa. Una secuencia de programa es defectuosa si los elementos individuales de un programa (por ejemplo módulos del software, subprogramas o comandos) se procesan en una mala secuencia o periodo de tiempo o si el reloj del procesador presenta un fallo.

A.9.1 Perro guardián con base de tiempo separada sin ventana temporal

NOTA Esta técnica/medida se menciona en las tablas A.10 y A.11 de la Norma IEC 61508-2.

Objetivo: Supervisar el comportamiento y la verosimilitud de la secuencia del programa.

Descripción: Los elementos de temporización externos con una base de tiempos diferente (por ejemplo, los perros guardianes) se lanzan periódicamente para supervisar el comportamiento del ordenador y la verosimilitud de la secuencia del programa. Es importante que los puntos de lanzamiento estén colocados correctamente en el programa. El perro guardián no se lanza en periodos fijos, pero está especificado un intervalo máximo.

A.9.2 Perro guardián con base de tiempo separada y ventana temporal

NOTA Esta técnica/medida se menciona en las tablas A.10 y A.11 de la Norma IEC 61508-2.

Objetivo: Supervisar el comportamiento y la verosimilitud de la secuencia del programa.

Descripción: Los elementos de temporización externos con una base de tiempos diferente (por ejemplo, los perros guardianes) se lanzan periódicamente para supervisar el comportamiento del ordenador y la verosimilitud de la secuencia del programa. Es importante que los puntos de lanzamiento estén colocados correctamente en el programa. Para el temporizador perro guardián se fijan un límite inferior y un límite superior. Si la secuencia del programa tarda más o menos tiempo del esperado, se lanza una acción de emergencia.

A.9.3 Supervisión lógica de la secuencia del programa

NOTA Esta técnica/medida se menciona en las tablas A.10 y A.11 de la Norma IEC 61508-2.

Objetivo: Supervisar la secuencia correcta de las secciones individuales del programa.

Descripción: La secuencia correcta de las secciones individuales del programa se supervisa con la ayuda de un software (procedimiento de contador, procedimiento adaptado) o con la ayuda de elementos de supervisión externos. Es importante que los puntos de supervisión se coloquen correctamente en el programa.

A.9.4 Combinación de supervisión temporal y lógica de las secuencias del programa

NOTA Esta técnica/medida se menciona en las tablas A.10 y A.11 de la Norma IEC 61508-2.

Objetivo: Supervisar el comportamiento y la secuencia correcta de las secciones individuales del programa.

Descripción: Un medio temporal (por ejemplo un temporizador perro guardián) que supervisa la secuencia del programa solo es relanzado si la secuencia de las secciones del programa se ejecuta correctamente.

A.9.5 Supervisión temporal con control en línea

NOTA Esta técnica/medida se menciona en las tablas A.10 y A.11 de la Norma IEC 61508-2.

Objetivo: Detectar los fallos de la supervisión temporal.

Descripción: La supervisión temporal se comprueba en el arranque que solo es posible si la supervisión temporal funciona correctamente. Por ejemplo, un sensor de temperatura se podría controlar en el arranque con una resistencia de calentamiento.

A.10 Ventilación y calentamiento

NOTA Este grupo de técnicas y medidas se menciona en las tablas A.16 y A.18 de la Norma IEC 61508-2.

Objetivo general: Controlar las disfunciones de ventilación o de calentamiento y/o la supervisión de la ventilación o calentamiento si está relacionada con la seguridad.

A.10.1 Sensor de temperatura

Objetivo: Detectar una temperatura demasiado alta o demasiado baja antes de que el sistema empiece a funcionar fuera de las especificaciones.

Descripción: Un sensor de temperatura supervisa la temperatura en los puntos más críticos de los sistemas E/E/PE relacionados con la seguridad. Antes de que la temperatura salga del rango especificado, se lanza una acción de emergencia.

A.10.2 Control de los ventiladores

Objetivo: Detectar un mal funcionamiento de los ventiladores.

Descripción: Se supervisa el buen funcionamiento de los ventiladores. Si un ventilador no funciona correctamente, se lanza una acción de mantenimiento (o como último recurso, de emergencia).

A.10.3 Accionamiento de la parada de seguridad por medio de un fusible térmico

Objetivo: Parar el sistema relacionado con la seguridad antes de que el sistema funcione fuera de su especificación térmica.

Descripción: Se utiliza un fusible térmico para parar el sistema relacionado con la seguridad. Para un PES, la parada se produce por un programa de puesta fuera de tensión que registra todas las informaciones necesarias para una acción de emergencia.

A.10.4 Mensaje escalonado de los sensores térmicos y alarma condicional

Objetivo: Indicar que el sistema relacionado con la seguridad funciona fuera de su especificación térmica.

Descripción: La temperatura se supervisa y se lanza una alarma si la temperatura está fuera del rango especificado.

A.10.5 Conexión de la refrigeración por aire forzado e indicación de estado

Objetivo: Prevenir el sobrecalentamiento con refrigeración por aire forzado.

Descripción: La temperatura se supervisa y la refrigeración por aire forzado se introduce si la temperatura es más alta que el límite especificado. El usuario es informado del estado.

A.11 Comunicación y almacenamiento masivo

Objetivo general: Controlar las disfunciones durante la comunicación con las fuentes externas y el almacenamiento masivo.

A.11.1 Separación entre las líneas de alimentación eléctrica y las líneas de información

NOTA Esta técnica/medida se menciona en la tabla A.16 de la Norma IEC 61508-2.

Objetivo: Minimizar las interferencias causadas en las líneas de información por las corrientes elevadas.

Descripción: Las líneas de alimentación se separan de las líneas de información. El campo eléctrico que podría provocar picos de tensión en las líneas de información disminuye con la distancia.

A.11.2 Separación espacial de las líneas múltiples

NOTA Esta técnica/medida se menciona en la tabla A.16 de la Norma IEC 61508-2.

Objetivo: Minimizar las interferencias causadas en las líneas múltiples por las corrientes elevadas.

Descripción: Las líneas que transportan las señales duplicadas se separan las unas de las otras. El campo eléctrico que podría provocar picos de tensión en las líneas múltiples disminuye con la distancia. Esta medida también reduce las disfunciones de causa común.

A.11.3 Aumento de la inmunidad a las interferencias

NOTA Esta técnica/medida se menciona en las tablas A.16 y A.18 de la Norma IEC 61508-2.

Objetivo: Minimizar las interferencias electromagnéticas sobre el sistema relacionado con la seguridad.

Descripción: Las técnicas de diseño tales como el blindaje y el filtrado se utilizan para aumentar la inmunidad del sistema relacionado con la seguridad a las perturbaciones electromagnéticas que se pueden radiar o conducir por las líneas de alimentación o de señal, o como resultado de descargas electrostáticas.

NOTA Véanse las referencias [16] y [17] para los requisitos de inmunidad de los sistemas relacionados con la seguridad y del equipamiento previsto para realizar funciones relacionadas con la seguridad (seguridad funcional) en aplicaciones industriales.

Referencias:

IEC/TR 61000-5-2:1997 *Compatibilidad electromagnética (CEM). Parte 5: Guías de instalación y atenuación. Sección 2: Puesta a tierra y cableado.*

Principles and Techniques of Electromagnetic Compatibility, Second Edition, C. Christopoulos, CRC Press, 2007, ISBN-10: 0849370353, ISBN-13: 978-0849370359.

Noise Reduction Techniques in Electronic Systems. H. W. Ott, John Wiley Interscience, 2nd Edition, 1988.

EMC for Product Designers. T. Williams, Newnes, 2007, ISBN 0750681705.

Grounding and Shielding Techniques in Instrumentation, 3rd edition, R. Morrison . Wiley-Interscience, New York, 1986, ISBN-10: 0471838055, ISBN-13: 978-0471838050.

A.11.4 Transmisión de señales complementarias

NOTA Esta técnica/medida se menciona en las tablas A.7 y A.16 de la Norma IEC 61508-2.

Objetivo: Detectar las mismas tensiones inducidas en las líneas múltiples de transmisión de señales.

Descripción: Todas las informaciones duplicadas se transmiten con las señales complementarias (por ejemplo, en lógica positiva o negativa). Las disfunciones de causa común (debidas, por ejemplo, a una emisión electromagnética) se pueden detectar con un comparador de las señales complementarias.

Referencia:

Elektronik in der Sicherheitstechnik. H. Jürs, D. Reinert, Sicherheitstechnisches Informationsund Arbeitsblatt 330220, BIA-Handbuch. 20. Lfg. V/93, Erich Schmidt Verlag, Bielefeld.
<http://www.bgia-handbuchdigital.de/330220>

A.12 Sensores

Objetivo general: Controlar las disfunciones de los sensores del sistema relacionado con la seguridad.

A.12.1 Sensor de referencia

NOTA Esta técnica/medida se menciona en la tabla A.13 de la Norma IEC 61508-2.

Objetivo: Detectar el mal funcionamiento de un sensor.

Descripción: Un sensor de referencia independiente se utiliza para supervisar el funcionamiento de un sensor de proceso. Todas las señales de entrada se comprueban a intervalos adecuados con el sensor de referencia para detectar las disfunciones del sensor de proceso.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

A.12.2 Conmutador de acción directa

NOTA Esta técnica/medida se menciona en la tabla A.13 de la Norma IEC 61508-2.

Objetivo: Abrir un contacto con una conexión mecánica directa entre la leva y el contacto de conmutación.

Descripción: Un conmutador de acción directa abre sus contactos normalmente cerrados con una conexión mecánica directa entre la leva y el contacto de conmutación. Esto garantiza que cada vez que la leva de conmutación está en posición de funcionamiento, los contactos de conmutación se abren.

Referencia:

Verriegelung beweglicher Schutzeinrichtungen. F. Kreutzkamp, K. Becker, Sicherheitstechnisches Informations- und Arbeitsblatt 330210, BIA-Handbuch. 1. Lfg. IX/85, Erich Schmidt Verlag, Bielefeld.

A.13 Elementos finales (accionadores)

Objetivo general: Controlar las disfunciones de los elementos finales del sistema relacionado con la seguridad.

A.13.1 Supervisión

NOTA Esta técnica/medida se menciona en la tabla A.14 de la Norma IEC 61508-2.

Objetivo: Detectar el mal funcionamiento de un accionador.

Descripción: El funcionamiento de un accionador se supervisa (por ejemplo, por los contactos de acción directa de un relé; véase supervisión de los contactos de relés en el apartado A.1.2). La redundancia introducida por esta supervisión puede servir para lanzar una acción de emergencia.

Referencias:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

Zusammenstellung und Bewertung elektromechanischer Sicherheitsschaltungen für Verriegelungseinrichtungen. F. Kreutzkamp, W. Hertel, Sicherheitstechnisches Informations- und Arbeitsblatt 330212, BIA-Handbuch. 17. Lfg. X/91, Erich Schmidt Verlag, Bielefeld.

A.13.2 Supervisión cruzada de varios accionadores

NOTA Esta técnica/medida se menciona en la tabla A.14 de la Norma IEC 61508-2.

Objetivo: Detectar los fallos de los accionadores comparando los resultados.

Descripción: Cada accionador múltiple se supervisa por un canal del hardware diferente. Si existe una diferencia se lanza una acción de emergencia.

A.14 Medidas contra el entorno físico

NOTA Esta técnica/medida se menciona en las tablas A.16 y A.18 de la Norma IEC 61508-2.

Objetivo: Prevenir las influencias del entorno físico (agua, polvo, productos corrosivos) que causan disfunciones.

Descripción: La envolvente del equipo se diseña para resistir al entorno previsto.

Referencia:

IEC 60529:1989 *Grados de protección proporcionados por las envolventes (código IP)*.

ANEXO B (Informativo)**PRESENTACIÓN DE LAS TÉCNICAS Y MEDIDAS PARA LOS SISTEMAS E/E/PE RELACIONADOS
CON LA SEGURIDAD: PREVENCIÓN DE LAS DISFUNCIONES SISTEMÁTICAS
(VÉANSE LAS NORMAS IEC 61508-2 E IEC 61508-3)**

NOTA Muchas de las técnicas de este anexo se aplican al software pero no se han repetido en el anexo C.

B.1 Medidas y técnicas generales**B.1.1 Gestión de proyecto**

NOTA Esta técnica/medida se menciona en las tablas B.1 a B.6 de la Norma IEC 61508-2.

Objetivo: Evitar las disfunciones adoptando un modelo de organización así como las reglas y medidas para el desarrollo y el ensayo de los sistemas relacionados con la seguridad.

Descripción: Las medidas más importantes y mejores son:

- la creación de un modelo de organización, en particular para asegurar la calidad, que se pone por escrito en un manual sobre el aseguramiento de la calidad; y
- el establecimiento de las regulaciones y medidas para la creación y la validación de los sistemas relacionados con la seguridad en recomendaciones para los proyectos cruzados y para los proyectos específicos.

A continuación se exponen varios principios básicos importantes:

- definición de una organización de diseño:
 - tareas y responsabilidades de las unidades organizativas;
 - autoridad de los departamentos de aseguramiento de la calidad;
 - independencia del aseguramiento de la calidad (inspección interna) con respecto al desarrollo;
- definición de un plan secuencial (modelos de actividad):
 - determinación de todas las actividades pertinentes durante la ejecución del proyecto, incluyendo las inspecciones internas y su planificación;
 - actualización del proyecto;
- definición de una secuencia normalizada para una inspección interna:
 - planificación, ejecución y control de la inspección (teoría de la inspección);
 - mecanismos de entrega para los subproductos;
 - aseguramiento de las inspecciones repetitivas;

- gestión de la configuración:
 - administración y control de las versiones;
 - detección de los efectos de las modificaciones;
 - inspección de coherencia después de las modificaciones;
- introducción de una evaluación cuantitativa de las medidas de aseguramiento de la calidad:
 - identificación de los requisitos;
 - estadísticas de las disfunciones;
- introducción de los métodos universales asistidos por ordenador, incluyendo las herramientas y la formación del personal.

Referencias:

ISO 9001:2008 *Sistemas de gestión de la calidad. Requisitos.*

ISO/IEC 15504 (todas las partes) *Tecnología de la información. Evaluación del proceso.*

CMMI: Guidelines for Process Integration and Product Improvement, 2nd Edition. M. B. Chrissis, M. Konrad, S. Shrum, Addison-Wesley Professional, 2006, ISBN-10: 0-3212-7967-0, ISBN-13: 978-0-3212-7967-5.

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

Dependability of Critical Computer Systems 1. F. J. Redmill, Elsevier Applied Science, 1988, ISBN 1-85166-203-0.

B.1.2 Documentación

NOTA 1 Esta técnica/medida se menciona en las tablas B.1 a B.6 de la Norma IEC 61508-2.

NOTA 2 Véanse también el capítulo 5 y el anexo A de la Norma IEC 61508-1.

Objetivo: Evitar las disfunciones y facilitar la evaluación de la seguridad del sistema documentando cada etapa del desarrollo.

Descripción: La aptitud y la seguridad operativa así como el cuidado aportado en el desarrollo por todas las partes que intervienen se deben demostrar durante la evaluación. Para poder demostrar el cuidado en el desarrollo y para garantizar la verificación de la prueba de seguridad en todo momento, se da una importancia particular a la documentación.

Las medidas comunes importantes son la introducción de directrices y de una ayuda por ordenador, es decir

- directrices que
 - especifican un plan de agrupamiento;
 - demandan una lista de control para los contenidos, y
 - determinan la forma de la documentación;
- administración de la documentación con la ayuda de una biblioteca de proyecto organizada y asistida por ordenador.

Las medidas individuales son:

- la separación en la documentación
 - de los requisitos;
 - del sistema (documentación para el usuario); y
 - del desarrollo (incluyendo la inspección interna);
- el agrupamiento de la documentación sobre el desarrollo de acuerdo con el ciclo de vida de la seguridad;
- la definición de los módulos normalizados de la documentación a partir de los que se puedan compilar los documentos;
- la identificación clara de las partes constitutivas de la documentación;
- actualización formal de las revisiones;
- la elección de los medios de descripción claros e inteligibles:
 - notación formal para las resoluciones;
 - lenguaje natural para las introducciones, las justificaciones y las representaciones de las intenciones;
 - representación gráfica para los ejemplos;
 - definición semántica de los elementos gráficos; y
 - glosario de palabras especializadas.

Referencias:

IEC 61506:1997 *Medida y control de procesos industriales. Documentación del software de aplicación.*

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.1.3 Separación de las funciones de seguridad de las funciones de no-seguridad del sistema E/E/PE

NOTA Esta técnica/medida se menciona en las tablas B.1 y B.6 de la Norma IEC 61508-2.

Objetivo: Evitar que la parte del sistema no relacionada con la seguridad tenga una influencia negativa sobre la parte relacionada con la seguridad.

Descripción: En la especificación, se debería decidir si es posible la separación de los sistemas relacionados con la seguridad de los sistemas no relacionados con la seguridad. Se deberían dar especificaciones claras para la interfaz entre las dos partes. Una separación clara reduce el esfuerzo del ensayo de los sistemas relacionados con la seguridad.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.1.4 Diversidad del hardware

NOTA Esta técnica/medida se menciona en las tablas A.15, A.16 y A.18 de la Norma IEC 61508-2.

Objetivo: Detectar las disfunciones sistemáticas durante el funcionamiento del EUC con la ayuda de diversos componentes que tienen tasas de disfunción y tipos de disfunciones diferentes.

Descripción: Los diferentes tipos de componentes se utilizan para los distintos canales de un sistema relacionado con la seguridad. Esto reduce la probabilidad de las disfunciones debidas a causas comunes (por ejemplo, sobretensión, perturbación electromagnética) y aumenta la probabilidad de detección de las disfunciones.

La existencia de medios diferentes para realizar la función deseada, por ejemplo principios físicos diferentes, ofrece distintas formas de resolver el mismo problema. Existen varios tipos de diversidad. La diversidad funcional utiliza diferentes aproximaciones para obtener el mismo resultado.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.2 Especificación de los requisitos de diseño del sistema E/E/PE

Objetivo general: Producir una especificación que sea, en la medida de lo posible, completa, sin errores, sin contradicciones y sencilla de verificar.

B.2.1 Especificación estructurada

NOTA Esta técnica/medida se menciona en las tablas B.1 y B.6 de la Norma IEC 61508-2.

Objetivo: Reducir la complejidad creando una estructura jerárquica de los requisitos parciales. Evitar las disfunciones de interfaz entre requisitos.

Descripción: Esta técnica estructura la especificación funcional en requisitos parciales de forma que existan relaciones visibles lo más simples posibles entre los requisitos. Este análisis se afina sucesivamente hasta que se distingan pequeños requisitos parciales claramente definidos. El resultado final es una estructura jerárquica de requisitos parciales que proporciona un marco para la especificación de los requisitos completos. Este método enfatiza las interfaces de los requisitos parciales y es particularmente eficaz para evitar las disfunciones en las interfaces.

Referencias:

ESA PSS 05-02, *Guide to the user requirements definition phase*, Issue 1, Revision 1, ESA Board for Software Standardisation and Control (BSSC), ESA, Paris, March 1995, <ftp://ftp.estec.esa.nl/pub/wm/wme/bssc/PSS0502.pdf>.

Structured Analysis and System Specification. T. De Marco, Yourdon Press, Englewood Cliffs, 1979, ISBN-10: 0138543801, ISBN-13: 978-0138543808.

B.2.2 Métodos formales

NOTA 1 Véase el apartado C.2.4 para los detalles sobre los métodos formales específicos.

NOTA 2 Esta técnica/medida se menciona en las tablas B.1, B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Los métodos formales transfieren los principios del razonamiento matemático a la especificación y a la implementación de los sistemas técnicos y en consecuencia incrementan la completitud, consistencia o corrección de una especificación o implementación.

Descripción: Los métodos formales ofrecen un medio para desarrollar una descripción de un sistema durante la fase de especificación o implementación. Estas descripciones formales son modelos matemáticos de la función y/o estructura del sistema.

En consecuencia, puede lograrse la descripción no ambigua del sistema (por ejemplo, todo estado de un autómata se describe mediante su estado inicial, entradas y las ecuaciones de transición del autómata) que aumenta la comprensión del sistema subyacente.

La elección de un método formal adecuado es una tarea difícil que exige un entendimiento total del sistema, de su proceso de desarrollo y de la gama de modelos matemáticos que podrían utilizarse (véanse las siguientes notas):

NOTA 3 Los teoremas de interés del modelo (propiedades) representan las garantías acerca del sistema que suministran bastante más confianza que la simulación, es decir, la observación de las acciones seleccionadas del sistema.

NOTA 4 Las desventajas de los métodos formales pueden ser:

- nivel fijo de abstracción;
- limitaciones para capturar toda la funcionalidad que es pertinente en una etapa dada;
- dificultad que los ingenieros de implementación tengan para entender el modelo;
- grandes esfuerzos para desarrollar, analizar y mantener el modelo a lo largo del ciclo de vida del sistema;
- disponibilidad de herramientas eficientes que soporten la construcción y análisis del modelo;
- disponibilidad de personal capaz de desarrollar y analizar el modelo.

NOTA 5 El foco de la comunidad de los métodos formales ha sido claramente modelar la función objetivo del sistema desenfatiando frecuentemente la robustez frente al fallo de un sistema. En consecuencia, se han seleccionado métodos formales que incluyen la robustez del sistema.

Referencia:

Formal Specification: Techniques and Applications. N.Nissanke, Springer-Verlag Telos, 1999, ISBN-10: 1852330023.

B.2.3 Métodos semiformales

NOTA 1 La tabla B.7 de la Norma IEC 61508-3 extiende esta lista del anexo B con otras técnicas semiformales relativas al software. Son las siguientes:

- diagramas de bloques lógicos/funcionales; descritos en la Norma IEC 61131-3;
- diagramas de secuencias: descritos en la Norma IEC 61131-3;
- diagramas de flujo de datos: véase el apartado C.2.2;
- autómatas de estados finitos/diagramas de cambios de estado: véase el apartado B.2.3.2;
- redes de Petri temporales: véase el apartado B.2.3.3;
- modelos de datos entidad-relación-atributo: véase el apartado B.2.4.4;
- mapas de secuencias de mensaje: véase el apartado C.2.14;
- tablas de decisión/verdad: véase el apartado C.6.1.

Objetivo: Enunciar las partes de una especificación de forma no ambigua y coherente para que los errores, omisiones y comportamiento erróneo se puedan detectar.

NOTA 2 Esta técnica/medida se menciona en las tablas B.1, B.2 y B.6 de la Norma IEC 61508-2 y en las tablas A.1, A.2, A.4, B.7, C.1, C.2, C.4 y C.17 de la Norma IEC 61508-3.

B.2.3.1 Generalidades

Objetivo: Probar que el diseño cumple con su especificación.

Descripción: Los métodos semiformales ofrecen un medio para desarrollar una descripción un sistema en un estado de su desarrollo, es decir, la especificación, el diseño o la codificación. La descripción puede, en ciertos casos, ser analizada por una máquina o animarse para mostrar diferentes aspectos del comportamiento del sistema. La animación puede reforzar la confianza en que el sistema satisface los requisitos reales así como los requisitos especificados.

En los apartados siguientes se describen dos métodos semiformales.

B.2.3.2 Máquinas de estados finitos/diagramas de cambios de estado

NOTA Esta técnica/medida se menciona en las tablas B.5, B.7, C.15 y C.17 de la Norma IEC 61508-3.

Objetivo: Modelizar, verificar, especificar o establecer la estructura de control de un sistema.

Descripción: Muchos de los sistemas se pueden describir en términos de estados, de entradas y de acciones. Así, en el estado S1, al recibir una entrada I, un sistema puede ejecutar la acción A y pasar al estado S2. Describiendo las acciones de un sistema para cada entrada en cada estado, es posible describir completamente un sistema. El modelo resultante del sistema se llama máquina de estados finitos (o autómata de estados finitos). A menudo se representa bajo la forma de diagrama de cambios de estado, que muestra como el sistema se desplaza de un estado a otro o bajo la forma de matriz en donde las dimensiones son los estados y las entradas. Las celdas de la matriz contienen la acción y el nuevo estado que es el resultado de la recepción de la entrada en el estado dado.

Cuando un sistema es complicado o tiene una estructura natural, se puede reflejar por una máquina de estados finitos en capas. Un mapa de estados es un tipo de diagrama de transición de estados en el que se permiten estados anidados (el estado objeto se divide en dos o más subestados que pueden evolucionar en paralelo, y posiblemente recombinarse en un estado sencillo en cierto punto); esto se añade a la potencia expresiva de la notación de la transición de estados pero puede añadir complejidad extra que puede ser indeseable en un sistema relacionado con la seguridad. Los mapas de estados tienen una especificación formal (matemática). Los diagramas de transición de estados pueden aplicarse al sistema completo o a algún objeto o elemento en él.

Una especificación o diseño expresados como máquina de estados finitos se puede comprobar en cuanto a:

- completitud (es necesario que el sistema u objeto tenga una acción y un nuevo estado para cada entrada en cada estado);
- coherencia (sólo es posible una transición de estado para cada par de estado/entrada); y
- accesibilidad (si es posible o no pasar de un estado a otro con alguna secuencia de entradas);
- ausencia de bucles infinitos o estados sin salida; etc.

Estas propiedades son importantes para los sistemas críticos. Las herramientas para facilitar estos controles se desarrollan fácilmente y pueden utilizarse varios modelos basados en el autómata de estados finitos (lenguajes formales, redes de Petri, gráficos de Markov, etc.). Existen algoritmos que permiten la generación automática de casos de ensayo para verificar la implementación de una máquina de estados finitos o para animar un modelo de máquina de estados finitos. Los diagramas de transición de estados y los mapas de estados están ampliamente soportados por herramientas que permiten dibujar los diagramas y chequearlos y que generarán un código para implementar la máquina de estados descrita.

Pueden utilizarse también para los cálculos de probabilidad de disfunción, véanse los capítulos B.6 y C.6.

Referencias:

Introduction to Automata Theory, Languages, and Computation (3rd Edition). J. Hopcroft, R. Motwani, J. Ullman, Addison-Wesley Longman Publishing Co, 2006, ISBN: 0321462254.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.2.3.3 Redes de Petri temporales

NOTA Esta técnica/medida se menciona en las tablas B.5, B.7, C.15 y C.17 de la Norma IEC 61508-3.

Objetivo: Modelizar los aspectos pertinentes del comportamiento del sistema, evaluar y posiblemente mejorar los requisitos de operación y de seguridad mediante el análisis y el rediseño.

Descripción: Las redes de Petri son un caso particular del autómata de estados finitos. Pertenecen a una categoría de modelos teóricos gráficos que son adecuados para representar el flujo de información y de control en los sistemas que tienen concurrencia y un comportamiento asíncrono.

Una red de Petri es una red de nodos y de transiciones. Los nodos pueden estar “marcados” o no. Se “permite” una transición cuando todos los nodos de entrada están marcados. Cuando está permitida, puede (pero no es obligatorio) “lanzarse”. Si se lanza, los nodos de entrada a la transición se vuelven no marcados y, en cambio, cada nodo de salida de la transición se marca.

Los peligros potenciales pueden representarse como estados particulares (marcas) en el modelo. El modelo de red de Petri se puede extender para permitir las características temporales del sistema. Aunque las redes de Petri “clásicas” se concentran sobre los aspectos de flujo de control, se han propuesto varias extensiones para incorporar el flujo de datos en el modelo.

También suministran un soporte muy eficaz para realizar la simulación de Monte Carlo con el fin de realizar los cálculos de probabilidad de disfunción, véase el apartado B.6.6.8.

Referencias:

Timed Petri Nets: Theory and Application. Jiacun Wang, Springer, 1998, ISBN 0792382706.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.2.4 Herramientas de especificación asistidas por ordenador

NOTA Esta técnica/medida se menciona en las tablas B.1 y B.6 de la Norma IEC 61508-2 y en las tablas A.1, A.2, C.1 y C.2 de la Norma IEC 61508-3.

B.2.4.1 Generalidades

Objetivo: Utilizar las técnicas de especificación formales para facilitar la detección automática de la ambigüedad y de la completitud.

Descripción: Esta técnica proporciona una especificación en forma de una base de datos que se puede inspeccionar automáticamente para evaluar la coherencia y la completitud. La herramienta de especificación puede animar diversos aspectos del sistema especificado para el usuario. En general, esta técnica facilita no solo la creación de la especificación sino también el diseño y otras fases del ciclo de vida del proyecto. Las herramientas de especificación se pueden clasificar de acuerdo con los apartados siguientes.

B.2.4.2 Herramientas no orientadas a ningún método específico

Objetivo: Ayudar al usuario a escribir una buena especificación proporcionando atajos y uniones entre las partes relacionadas.

Descripción: La herramienta de especificación realiza una parte del trabajo rutinario del usuario y facilita la gestión del proyecto. No aplica ninguna metodología de especificación en particular. La independencia relativa en relación al método ofrece a los usuarios una gran libertad pero también les aporta poco soporte especializado necesario en la creación de las especificaciones. Esto hace más difícil la familiarización con el sistema.

B.2.4.3 Procedimiento orientado al modelo con un análisis jerárquico

Objetivo: Evitar la no-completitud, ambigüedad y la contradicción en la especificación, por ejemplo, apoyando al usuario para escribir una buena especificación asegurando la coherencia entre la descripción de las acciones y los datos en diferentes niveles de abstracción.

Descripción: Este método proporciona una representación funcional del sistema deseado (análisis estructurado) en diferentes niveles de abstracción (grado de precisión). Hay un arsenal enorme de tales modelos: los autómatas finitos son una clase de estos modelos, ampliamente utilizados para describir la evolución de sistemas discretos/digitales. Las ecuaciones diferenciales son similares en espíritu y objetivo para los sistemas continuos/analógicos. El análisis a diferentes niveles de abstracción actúa sobre las acciones y los datos. La evaluación de la ambigüedad y de la completitud es posible entre los niveles jerárquicos y también entre dos unidades funcionales (módulos) en el mismo nivel (por ejemplo, todo estado de un modelo del sistema se describe mediante su estado inicial, sus entradas y las ecuaciones de transición del autómata).

NOTA Las desventajas de las descripciones basadas en modelos pueden ser el nivel de abstracción, limitaciones para capturar toda la funcionalidad que es pertinente en una etapa dada, dificultad que los ingenieros de implementación tengan para entender el modelo (desde la lectura de la sintaxis hasta la comprensión), grandes esfuerzos para desarrollar, analizar y mantener el modelo a lo largo del ciclo de vida del sistema, disponibilidad de herramientas eficientes que soporten la construcción y análisis del modelo (el desarrollo de tales herramientas es ciertamente una tarea de gran esfuerzo) y la disponibilidad de personal capaz de desarrollar y analizar los modelos.

Referencia:

System requirements analysis. Jeffrey O. Grady, Academic Press, 2006, ISBN 012088514X, 9780120885145.

B.2.4.4 Modelos de datos de entidad-relación-atributo

Objetivo: Ayudar al usuario a escribir una buena especificación centrándose en entidades del sistema y en las relaciones entre estas entidades.

Descripción: El sistema deseado se describe como un conjunto de objetos y sus relaciones. La herramienta permite determinar qué relaciones se pueden interpretar por el sistema. En general, las relaciones permiten una descripción de la estructura jerárquica de los objetos, de los flujos de datos, de las relaciones entre los datos y de los datos que se someten a ciertos procesos de fabricación. El procedimiento clásico se ha extendido a las aplicaciones de control de procesos. Las capacidades de inspección y de asistencia al usuario dependen de la variedad de las relaciones ilustradas. Por otro lado, el gran número de posibilidades de representación hace que la aplicación de esta técnica sea complicada.

Referencia:

Software Requirements: Practical Techniques for Gathering and Managing Requirements Throughout the Product Development Cycle. Karl Eugene Wiegers, Microsoft Press, 2003, ISBN 0735618798, 9780735618794.

B.2.4.5 Estímulo y respuesta

Objetivo: Ayudar al usuario a escribir una buena especificación identificando las relaciones entre los estímulos y las respuestas.

Descripción: Las relaciones entre los objetos del sistema se especifican con una notación de estímulos y de respuestas. Se utiliza un lenguaje simple y fácil de extender, que contiene elementos de lenguaje que representan los objetos, las relaciones, las características y las estructuras.

B.2.5 Listas de control

NOTA Esta técnica/medida se menciona en las tablas B.1, B.2 y B.6 de la Norma IEC 61508-2 y en las tablas A.10, B.8, C.10 y C.18 de la Norma IEC 61508-3.

Objetivo: Atraer la atención y gestionar la evaluación crítica de todos los aspectos importantes del sistema en cada fase del ciclo de vida de la seguridad, asegurando una cobertura completa sin establecer los requisitos exactos.

Descripción: Conjunto de preguntas a las que la persona que realiza la lista de control debe responder. Muchas de las cuestiones son de carácter general y la persona encargada de la evaluación las debe interpretar de la forma que sea la más apropiada para el sistema particular evaluado. Las listas de control se pueden utilizar en todas las fases del ciclo de vida de seguridad global del software y de los sistemas E/E/PE y son particularmente útiles como herramienta para evaluar la seguridad funcional.

Para adaptarse a una gran variedad de sistemas a validar, la mayor parte de las listas de control contienen preguntas aplicables a varios tipos de sistemas. En consecuencia, puede haber preguntas en la lista de control utilizada inadaptadas al sistema considerado y que se deberían ignorar. Igualmente, puede haber una necesidad, para un sistema en particular, de adjuntar a la lista de control estándar las cuestiones específicas relativas a ese sistema.

En cualquier caso, debería quedar claro que la utilización de la lista de control depende de forma crítica de la competencia y del juicio del ingeniero que elige y aplica la lista de control. En consecuencia, deberían justificarse y documentarse las decisiones tomadas por el ingeniero relativas a la o a las listas de control elegidas, así como todas las cuestiones suplementarias o superfluas. El objetivo es asegurar que la aplicación de las listas de control se puede examinar y que se obtendrán los mismos resultados, excepto si se utilizan criterios diferentes.

Cuando se rellena una lista de control hay que ser tan conciso como sea posible. Cuando sea necesaria una justificación larga, debería hacerse con una referencia a la documentación suplementaria. Deberían utilizarse respuestas del tipo “pasa”, “falla” o “no concluyente” o un conjunto limitado similar de respuestas para documentar los resultados de cada cuestión. Esta precisión simplifica mucho el procedimiento para obtener una conclusión general de los resultados de evaluación de la lista de control.

Referencias:

IEC 60880:2006 *Centrales nucleares. Instrumentación y sistemas de control importantes para la seguridad. Aspectos lógicos (software) de los sistemas informáticos que realizan funciones de categoría A.*

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Software Quality Assurance: From Theory to Implementation. Daniel Galin, Pearson Education, 2004, ISBN 0201709457, 9780201709452.

IEC 61346 (todas las partes), *Industrial systems, installations and equipment and industrial products – Structuring principles and reference designation.*

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

Risk Assessment and Risk Management for the Chemical Process Industry. H.R. Greenberg, J.J. Cramer, John Wiley and Sons, 1991, ISBN 0471288829, 9780471288824.

B.2.6 Inspección de la especificación

NOTA Esta técnica/medida se menciona en las tablas B.1 y B.6 de la Norma IEC 61508-2.

Objetivo: Evitar la no completitud y la contradicción en la especificación.

Descripción: La inspección es una técnica general con la que se evalúan, por un equipo independiente, varias cualidades de un documento de especificación. El equipo de inspección formula preguntas al creador, que debe responderla de forma satisfactoria. El examen debería realizarlo (si es posible) un equipo que no haya estado implicado en la creación de la especificación. El grado de independencia requerido se determina con los niveles de integridad de seguridad exigidos del sistema. Los inspectores independientes deberían ser capaces de reconstruir la función operacional del sistema de forma indiscutible sin referirse a otras especificaciones. También deben verificar que están cubiertos todos los aspectos técnicos y de seguridad pertinentes en las medidas de organización y de operación. Este procedimiento se ha probado que es muy efectivo en la práctica.

Referencias:

IEC 61160:2005 *Revisión de diseño.*

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Software Quality Assurance: From Theory to Implementation. D. Galin, Pearson Education, 2004, ISBN 0201709457, 9780201709452.

B.3 Diseño y desarrollo de los sistemas E/E/PE

Objetivo general: Producir un diseño estable del sistema relacionado con la seguridad, conforme a la especificación.

B.3.1 Seguimiento de las directrices y de las normas

NOTA Esta técnica/medida se menciona en la tabla B.2 de la Norma IEC 61508-2.

Objetivo: Respetar las normas del sector de aplicación (no precisadas en esta norma).

Descripción: Deberían seguirse las directrices durante el diseño del sistema relacionado con la seguridad. Las directrices deberían conducir primero a sistemas relacionados con la seguridad prácticamente sin disfunciones y después deberían facilitar la validación de la seguridad. Pueden ser universales, específicas para un proyecto o específicas para una sola fase.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.3.2 Diseño estructurado

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Reducir la complejidad creando una estructura jerárquica de requisitos parciales. Evitar las disfunciones de interfaz entre los requisitos. Simplificar la verificación.

Descripción: Durante el diseño del hardware, deberían utilizarse los criterios o los métodos específicos. Por ejemplo, se podría requerir lo siguiente:

- un diseño de circuito jerárquicamente estructurado;
- la utilización de componentes del circuito fabricados y ensayados.

Igualmente, durante el diseño del software, la utilización de organigramas estructurados permite crear una estructura no ambigua de los módulos del software. Esta estructura presenta como se relacionan los diferentes módulos entre ellos, los datos precisos que se intercambian entre los módulos y los controles precisos entre los módulos.

Referencias:

IEC 61346 (todas las partes) *Sistemas industriales, instalaciones y equipos y productos industriales. Principios de estructuración y designación de referencia*.

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Software Design. D. Budgen, Pearson Education, 2003, ISBN 0201722194, 9780201722192.

An Overview of JSD, J. R. Cameron, IEEE Trans SE-12 No. 2, February 1986.

Structured Development for Real-Time Systems (3 Volumes). P. T. Yourdon, P. T. Yourdon Press, 1985.

Structured Development for Real-Time Systems (3 Volumes). P. T. Ward, S. J. Mellor, Yourdon Press, 1985.

Applications and Extensions of SADT. D. T. Ross, Computer, 25-34, April 1985.

Essential Systems Analysis. St. M. McMenamin, F. Palmer, Yourdon Inc, 1984.

Structured Analysis (SA): A language for communicating ideas. D. T. Ross, IEEE Trans. Software Eng, Vol. SE-3 (1), 16-34.

B.3.3 Utilización de componentes probados

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Reducir el riesgo de numerosos fallos nuevos y no detectados utilizando componentes que tienen unas características específicas.

Descripción: La elección de componentes probados se realiza por el fabricante, considerando la seguridad, en función de la fiabilidad de los elementos (por ejemplo la utilización de unidades físicas ensayadas en operación para satisfacer los requisitos de seguridad alta o el almacenamiento de programas relacionados con la seguridad únicamente en memorias seguras). La seguridad de las memorias se puede referir a un acceso no autorizado, a las influencias del entorno (compatibilidad electromagnética, radiación, etc.) así como a la respuesta de los elementos en caso de disfunción.

Referencia:

IEC 61163-1:2006 *Cribado de elementos mediante esfuerzos. Parte 1: Elementos reparables fabricados en lotes*.

B.3.4 Modularización

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Reducir la complejidad y evitar las disfunciones relacionadas con la interfaz de los subsistemas.

Descripción: Cada subsistema, en todos los niveles de diseño, está claramente definido y es de tamaño limitado (únicamente algunas funciones). Las interfaces entre los subsistemas se mantienen tan simples como sea posible y las interacciones (es decir, los datos comunes, el intercambio de información) se reducen al mínimo. La complejidad de los subsistemas individuales también está limitada.

Referencias:

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Software Reliability. Principles and Practices. G. J. Myers, Wiley-Interscience, New York, 1976, ISBN-10: 0471627658, ISBN-13: 978-0471627654.

B.3.5 Herramientas de diseño asistido por ordenador

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2 y en las tablas A.4 y C.4 de la Norma IEC 61508-3.

Objetivo: Realizar el procedimiento de diseño más sistemáticamente. Incluir los elementos de construcción automáticos apropiados que ya están disponibles y ensayados.

Descripción: Deberían utilizarse las herramientas de diseño asistido por ordenador (CAD) para el diseño del hardware y del software, cuando existan y esté justificado por la complejidad del sistema. La corrección de estas herramientas se debería demostrar con ensayos específicos, con una larga historia de uso satisfactorio o con una verificación independiente de los resultados para el sistema particular relacionado con la seguridad que se está diseñando.

Las herramientas de soporte deberían seleccionarse por sus grados de integración. En este contexto, las herramientas se integran si funcionan de manera cooperativa de manera que las salidas desde una herramienta tienen un contenido y formato adecuados para la entrada automática de la siguiente herramienta, minimizando así la posibilidad de introducir el error humano en el reacondicionamiento de los resultados intermedios.

Referencias:

Overview of Technology Computer-Aided Design Tools and Applications in Technology Development, Manufacturing and Design. W. Fichtner, Journal of Computational and Theoretical Nanoscience, Volume 5, Number 6, June 2008, pp. 1089-1105(17).

The Electromagnetic Data Exchange: Much more than a Common Data Format. P.E. Frandsen et al. In *Proceeding of the 2nd European Conference on Antennas and Propagation*. The Institution of Engineering and Technology (IET), 2007, ISBN 978-0-86341-842-6.

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

B.3.6 Simulación

NOTA Esta técnica/medida se menciona en las tablas B.2, B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Realizar una inspección completa y sistemática de un circuito eléctrico/electrónico, de las características funcionales y del dimensionamiento correcto de los componentes.

Descripción: La función del circuito del sistema relacionado con la seguridad se simula con el ordenador con un modelo software del comportamiento. Los componentes individuales del circuito tienen cada uno su propio comportamiento simulado y la respuesta del circuito en el que se conectan se estudia observando las especificaciones en los límites de cada componente.

B.3.7 Inspección (revisión y análisis)

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Mostrar las diferencias entre la especificación y la implementación.

Descripción: Las funciones especificadas del sistema relacionado con la seguridad se estudian y evalúan para asegurar que el sistema relacionado con la seguridad está conforme con los requisitos de la especificación. Cualquier duda relativa a la implementación y utilización del producto se documenta con el fin de que pueda resolverse. Contrariamente a un sondeo, el autor es pasivo y el inspector es activo durante el procedimiento de inspección.

Referencias:

IEC 61160:2005 *Revisión de diseño*.

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

ANSI/IEEE 1028:1997, *IEEE Standard for software reviews*.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.3.8 Sondeo

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.6 de la Norma IEC 61508-2.

Objetivo: Mostrar las diferencias entre la especificación y la implementación.

Descripción: Las funciones especificadas del proyecto del sistema relacionado con la seguridad se estudian y evalúan para asegurar que el sistema relacionado con la seguridad satisface los requisitos dados en la especificación. Cualquier duda y punto débil relativo a la realización y utilización del producto se documenta con el fin de que pueda resolverse. Contrariamente a una inspección, el autor es activo y el inspector es pasivo durante el sondeo.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

ANSI/IEEE 1028:1997, *IEEE Standard for software reviews*.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

Methodisches Testen von Programmen. G. J. Myers, Oldenbourg Verlag, München, Wien, 1987.

B.4 Procedimientos de operación y de mantenimiento de los sistemas E/E/PE

Objetivo general: Desarrollar los procedimientos que ayuden a evitar las disfunciones durante la operación y el mantenimiento del sistema relacionado con la seguridad.

B.4.1 Instrucciones de operación y de mantenimiento

NOTA Esta técnica/medida se menciona en la tabla B.4 de la Norma IEC 61508-2.

Objetivo: Evitar errores durante la operación y el mantenimiento del sistema relacionado con la seguridad.

Descripción: Las instrucciones para el usuario contienen informaciones esenciales sobre la forma de utilizar y mantener el sistema relacionado con la seguridad. En unos casos particulares, en las instrucciones habrá también ejemplos sobre la forma de instalar el sistema relacionado con la seguridad en general. Es necesario que todas las instrucciones sean fácilmente comprensibles. Deberían utilizarse figuras y esquemas para describir las dependencias y los procedimientos complejos.

Referencia: *Guidelines for Safe Automation of Chemical Processes*. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.4.2 Facilidad de utilización

NOTA Esta técnica/medida se menciona en la tabla B.4 de la Norma IEC 61508-2.

Objetivo: Reducir la complejidad durante la operación del sistema relacionado con la seguridad.

Descripción: El buen funcionamiento del sistema relacionado con la seguridad depende, en gran medida, de la operación humana. Teniendo en cuenta el diseño del sistema y el diseño del lugar de trabajo, la persona que desarrolla el sistema relacionado con la seguridad debe asegurarse de que:

- la necesidad de intervención humana se reduce a un mínimo estricto;
- la intervención necesaria es tan simple como sea posible;
- el potencial de daño que resulta de los errores del operador es reducido;
- los medios de intervención y las indicaciones se diseñan en función de los requisitos ergonómicos;
- los medios puestos a disposición del operador son simples, bien etiquetados y de utilización intuitiva;
- el operador no está sobrecargado, ni siquiera en condiciones extremas;
- la formación sobre los procedimientos y los medios de intervención se adaptan al nivel de conocimiento y de motivación del usuario que recibe la formación.

B.4.3 Facilidad de mantenimiento

NOTA Esta técnica/medida se menciona en la tabla B.4 de la Norma IEC 61508-2.

Objetivo: Simplificar los procedimientos de mantenimiento del sistema relacionado con la seguridad y diseñar los medios necesarios para un diagnóstico y una reparación eficaces.

Descripción: El mantenimiento preventivo y la reparación se realizan a menudo en condiciones difíciles y bajo presión de plazos. En consecuencia, la persona que desarrolla el sistema relacionado con la seguridad debería asegurar que:

- las medidas de mantenimiento relacionadas con la seguridad se necesitan tan poco como sea posible; lo ideal sería que nunca fueran necesarias;
- se incluyen las herramientas de diagnóstico suficientes, sensibles y fáciles de manipular para estas reparaciones inevitables; las herramientas deberían incluir todas las interfaces necesarias;
- si se deben desarrollar u obtener herramientas de diagnóstico separadas, deberían estar disponibles a tiempo.

B.4.4 Posibilidades de operación limitada

NOTA Esta técnica/medida se menciona en las tablas B.4 y B.6 de la Norma IEC 61508-2.

Objetivo: Reducir las posibilidades de operación para un usuario normal.

Descripción: Esta aproximación reduce las posibilidades de operación:

- limitando la operación en los modos de operación especiales, por ejemplo con conmutadores con llave;
- limitando el número de elementos en operación;
- limitando el número de modos de operación generalmente posibles.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.4.5 Operación únicamente por operadores cualificados

NOTA Esta técnica/medida se menciona en las tablas B.4 y B.6 de la Norma IEC 61508-2.

Objetivo: Evitar las disfunciones de operación debidas a una mala utilización.

Descripción: El operador del sistema relacionado con la seguridad se forma a un nivel correspondiente al nivel de integridad de seguridad y de complejidad del sistema relacionado con la seguridad. La formación incluye el aprendizaje de las bases del proceso de producción y el conocimiento de la relación entre el sistema relacionado con la seguridad y el EUC.

Referencia:

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.4.6 Protección contra los errores humanos

NOTA Esta técnica/medida se menciona en las tablas B.4 y B.6 de la Norma IEC 61508-2.

Objetivo: Proteger el sistema contra todos los tipos de errores del operador.

Descripción: Las entradas erróneas (valores, tiempos, etc.) se detectan con los controles de verosimilitud o supervisando del EUC. Para integrar estos medios en el diseño, hay que establecer en las primeras fases qué entradas son posibles y cuáles son admisibles.

B.4.7 (No utilizado)

B.4.8 Protección contra las modificaciones

NOTA Esta técnica/medida se menciona en las tablas A.17 y A.18 de la Norma IEC 61508-2.

Objetivo: Proteger el sistema relacionado con la seguridad con medios técnicos contra las modificaciones del hardware.

Descripción: Las modificaciones o manipulaciones se detectan automáticamente, por ejemplo con los controles de verosimilitud de las señales de los sensores, una detección con los procesos técnicos y con los ensayos automáticos de arranque. Si se detecta una modificación, se lanza una acción de emergencia.

B.4.9 Acuse de recibo de las entradas

NOTA Esta técnica/medida se menciona en las tablas A.17 y A.18 de la Norma IEC 61508-2.

Objetivo: El propio operador detecta un error durante la operación antes de activar el EUC.

Descripción: Se reenvía al operador una entrada para el EUC enviada por el sistema relacionado con la seguridad antes de ser enviada al EUC, de forma que el operador puede detectar y corregir el error. El diseño del sistema debería tener en cuenta las acciones anormales y no provocadas del personal y los límites de velocidad superior e inferior y el sentido de la respuesta humana. Esto evitaría, por ejemplo, que el operador apriete las teclas muy rápido y que el sistema tome la presión de dos teclas por una sola o que una misma tecla sea apretada dos veces porque el sistema (visualización) fue demasiado lento para reaccionar a la primera tecla. La pulsación de una tecla solo debería ser válida la primera vez para la recogida de datos críticos; se debe evitar que se produzca una situación peligrosa en el sistema cuando se presiona un número ilimitado de veces la tecla “enter” o la tecla “sí”.

Deberían incluirse procedimientos de fin de temporización con preguntas de elección múltiple (sí, no, etc.) para las veces en las que el operador dude y deje el sistema en espera.

La posibilidad de reinicializar un PES relacionado con la seguridad hace vulnerable al sistema, a menos que el hardware/software estén diseñados teniendo en cuenta estas situaciones.

B.5 Integración de los sistemas E/E/PE

Objetivo general: Evitar las disfunciones durante la fase de integración y revelar todas las disfunciones que tienen lugar durante esta fase y durante la fases precedentes.

B.5.1 Ensayo funcional

NOTA Esta técnica/medida se menciona en las tablas B.3 y B.5 de la Norma IEC 61508-2 y en las tablas A.5, A.6, A.7, C.5, C.6 y C.7 de la Norma IEC 61508-3.

Objetivo: Revelar las disfunciones durante las fases de especificación y diseño. Evitar las disfunciones durante la implementación y la integración del software y del hardware.

Descripción: Durante los ensayos funcionales, se realizan revisiones para ver si las características especificadas del sistema se han logrado. El sistema recibe datos de entrada que caracterizan correctamente el funcionamiento normalmente esperado. Se observan las salidas y su respuesta se compara con la que se indica en la especificación. Se documentan las diferencias en relación a la especificación y las indicaciones de una especificación incompleta.

El ensayo funcional de los componentes electrónicos diseñados para una arquitectura de varios canales implica habitualmente que los componentes fabricados se ensayen con componentes similares previamente validados. Además, se recomienda ensayar los componentes fabricados conjuntamente con otros componentes similares del mismo lote con el fin de identificar los fallos de modo común que de otra forma habrían quedado ocultos.

También es necesario que la capacidad de trabajo del sistema sea suficiente, véase el apartado C.5.20.

Referencias:

Software Testing and Quality Assurance. K. Naik, P. Tripathy, Wiley Interscience, 2008, Print ISBN: 9780471789116
Online ISBN: 9780470382844.

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Practical Software Testing: A Process-oriented Approach. I. Burnstein, Springer, 2003, ISBN 0387951318, 9780387951317.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.5.2 Ensayo de “caja negra”

NOTA Esta técnica/medida se menciona en las tablas B.3, B.5 y B.6 de la Norma IEC 61508-2 y en las tablas A.5, A.6, A.7, C.5, C.6 y C.7 de la Norma IEC 61508-3.

Objetivo: Controlar el comportamiento dinámico en las condiciones reales de funcionamiento. Revelar las disfunciones en el cumplimiento de la especificación funcional y evaluar su utilidad y su robustez.

Descripción: Las funciones de un sistema o de un programa se ejecutan en un entorno especificado, con los datos de ensayo especificados deducidos sistemáticamente de la especificación según criterios establecidos. Esto expone el comportamiento del sistema y permite una comparación con la especificación. No se utiliza ningún conocimiento de la estructura interna del sistema para dirigir el ensayo. El objetivo es determinar si la unidad funcional realiza correctamente todas las funciones requeridas por la especificación. La técnica que consiste en formar clases de equivalencia es un ejemplo de los criterios para definir los datos de ensayo de caja negra. El conjunto de datos de entrada se subdivide en rangos de valores de entrada (clases de equivalencia) con la ayuda de la especificación. Los casos de ensayo se forman a partir de:

- los datos de los rangos permitidos;
- los datos fuera de los rangos permitidos;
- los datos en los límites de los rangos;
- los valores extremos;
- las combinaciones de las clases anteriores.

Otros criterios pueden ser eficaces para elegir los casos de ensayo en las diferentes actividades de ensayo (ensayo del módulo, ensayo de integración y ensayo del sistema). Por ejemplo, para los ensayos del sistema en el marco de una validación, se utiliza el criterio “condiciones de operación extremas”.

Referencias:

Software Testing and Quality Assurance. K. Naik, P. Tripathy, Wiley Interscience, 2008, Print ISBN: 9780471789116
Online ISBN: 9780470382844.

Essentials of Software Engineering. Frank F. Tsui, Orlando Karam. Jones & Bartlett, 2006. ISBN 076373537X, 9780763735371.

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Systematic Software Testing. Rick D. Craig, Stefan P. Jaskiel. Artech House, 2002. ISBN 1580535089, 9781580535083.

B.5.3 Ensayo estadístico

NOTA Esta técnica/medida se menciona en las tablas B.3, B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Verificar el comportamiento dinámico del sistema relacionado con la seguridad y evaluar su utilidad y su robustez.

Descripción: Esta aproximación ensaya un sistema o un programa con unos datos de entrada elegidos en función de la distribución estadística esperada de los datos de entrada reales, es decir el perfil operacional.

Referencias:

A discussion of statistical testing on a safety-related application. S Kuball, J H R May, Proc. IMechE Vol. 221 Part O: J. Risk and Reliability, Institution of Mechanical Engineers, 2007.

Practical Reliability Engineering. P. O'Connor, D. Newton, R. Bromley, John Wiley and Sons, 2002, ISBN 0470844639, 9780470844632.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

Dependability of Critical Computer Systems 1. F. J. Redmill, Elsevier Applied Science, 1988, ISBN 1-85166-203-0.

B.5.4 Experiencia en campo

NOTA 1 Esta técnica/medida se menciona en las tablas B.3, B.5 y B.6 de la Norma IEC 61508-2.

NOTA 2 Véase también en el apartado C.2.10 una medida similar y en el anexo D una aproximación estadística, ambas en el contexto del software.

Objetivo: Utilizar el retorno de la experiencia en campo con diferentes aplicaciones como una de las medidas que permiten evitar fallos, tanto durante la integración de los sistemas E/E/PE como durante la validación de la seguridad de los sistemas E/E/PE.

Descripción: Utilización de los componentes o de los subsistemas que han demostrado que no tienen fallos, o son sólo fallos sin importancia, cuando se han usado durante un periodo suficiente y en muchas aplicaciones distintas. La persona que desarrolla el sistema debe prestar atención a las funciones que han sido realmente validadas por el retorno de experiencia, en particular para los componentes complejos con multitud de funciones posibles (por ejemplo, el sistema operativo, los circuitos integrados). Por ejemplo, consideremos los subprogramas de autotests para la detección de fallos: si no hay ninguna avería del hardware durante el periodo de operación, estos subprogramas no se pueden considerar como ensayados, ya que no han realizado nunca su función de detección de fallos.

Para que se pueda aplicar la experiencia en campo, se deben cumplir las siguientes condiciones:

- especificaciones sin cambios;
- 10 sistemas en aplicaciones diferentes;
- 10⁵ h en operación y al menos un año de historia de servicio.

NOTA 3 Las normas sectoriales pueden especificar números diferentes.

La experiencia en campo se demuestra con la documentación del fabricante y/o de la compañía que opera el sistema. Es necesario que esta documentación contenga al menos:

- la designación exacta del sistema y sus componentes, incluyendo el control de la versión del hardware;
- los usuarios y el tiempo de aplicación;
- el número de horas de funcionamiento;
- los procedimientos para la elección de los sistemas y aplicaciones seleccionadas para la prueba;
- los procedimientos para la detección y el registro de los fallos así como para su corrección.

Referencias:

IEC 60300-3-2:2004 *Gestión de la confiabilidad. Parte 3-2: Guía de aplicación. Recogida de datos de confiabilidad en la explotación.*

Guidelines for Safe Automation of Chemical Processes. CCPS, AIChE, New York, 1993, ISBN-10: 0-8169-0554-1, ISBN-13: 978-0-8169-0554-6.

B.6 Validación de la seguridad de los sistemas E/E/PE

Objetivo general: Probar que el sistema E/E/PE relacionado con la seguridad es conforme con la especificación de los requisitos de seguridad del sistema E/E/PE y con la especificación de los requisitos de diseño del sistema E/E/PE.

B.6.1 Ensayos funcionales en las condiciones del entorno

NOTA Esta técnica/medida se menciona en la tabla B.5 de la Norma IEC 61508-2.

Objetivo: Evaluar si el sistema relacionado con la seguridad está protegido contra las influencias habituales del entorno.

Descripción: El sistema se somete a distintas condiciones ambientales (por ejemplo, de acuerdo con la serie de Normas IEC 60068 o IEC 61000) en las cuales se evalúa la fiabilidad de las funciones de seguridad (así como su compatibilidad con las normas mencionadas).

Referencias:

IEC 60068-1:1988, *Ensayos ambientales. Parte 1: Generalidades y guía*.
Modificación 1 (1992)

IEC 61000-4-1:2006 *Compatibilidad electromagnética (CEM). Parte 4-1: Técnicas de ensayo y de medida. Visión de conjunto de la serie IEC 61000-4*

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.6.2 Ensayos de inmunidad a las ondas de choque

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Evaluar la capacidad del sistema relacionado con la seguridad para soportar los picos de las ondas de choque.

Descripción: El sistema se carga con un programa de aplicación típico y todas las líneas periféricas (todas las interfaces digitales, analógicas y serie, así como las conexiones del bus y la alimentación, etc.) se someten a señales de ruido normalizado. Con el fin de obtener una declaración cuantitativa, se recomienda aproximar cuidadosamente el límite de la onda de choque. La categoría de ruido elegida no se cumple si falla la función.

Referencias:

IEC 61000-4-5:2005 *Compatibilidad electromagnética (CEM). Parte 4-5: Técnicas de ensayo y de medida. Ensayos de inmunidad a las ondas de choque*.

C37.90.1-2002, *IEEE Standard for Surge Withstand Capability (SWC) Tests for Relays and Relay Systems Associated with Electric Power Apparatus*.

B.6.3 (No utilizado)

B.6.4 Análisis estático

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2 y en las tablas A.9, B.8, C.9 y C.18 de la Norma IEC 61508-3.

Objetivo: Evitar los fallos sistemáticos que pueden causar las disfunciones del sistema ensayado, o bien al principio o bien después de años de operación.

Descripción: Esta aproximación sistemática y eventualmente asistida por ordenador revisa las características estáticas específicas del sistema prototipo para asegurar la completitud, la coherencia y la ausencia de ambigüedad en relación a los requisitos en cuestión (por ejemplo directrices de construcción, especificaciones del sistema y ficha técnica del equipo). El análisis estático es reproducible. Se aplica a un prototipo que ha llegado a un estado de finalización bien definido. Unos ejemplos de análisis estático para el hardware y el software son:

- el análisis de coherencia del flujo de datos (tal como el ensayo consistente en la verificación de que un objeto de datos se interpreta en todos sitios con el mismo valor);
- el análisis del flujo de control (tal como la determinación de la ruta de acceso o del código inaccesible);

- el análisis de la interfaz (tal como la búsqueda de la transferencia de variables entre diferentes módulos del software);
- el análisis del flujo de datos para detectar las secuencias sospechosas de creación, designación y supresión de variables;
- el ensayo de cumplimiento de las directrices específicas (por ejemplo, distancias en el aire y líneas de fuga, distancias de ensamblado, disposición de la unidad física, unidades físicas mecánicamente sensibles, utilización exclusiva de unidades físicas que se introdujeron).

Referencias:

Static Analysis and Software Assurance. D. Wagner, Lecture Notes in Computer Science, Volume 2126/2001, Springer, 2001, ISBN 978-3-540-42314-0.

An Industrial Perspective on Static Analysis. B A Wichmann, A A Canning, D L Clutterbuck, L A Winsborrow, N J Ward and D W R Marsh. Software Engineering Journal., 69 – 75, March 1995.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.6.5 Análisis y ensayo dinámicos

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2 y en las tablas A.5, A.9, B.2, C.5, C.9 y C.12 de la Norma IEC 61508-3.

Objetivo: Detectar las disfunciones de la especificación revisando el comportamiento dinámico de un prototipo en un estado de finalización avanzado.

Descripción: El análisis dinámico de un sistema relacionado con la seguridad se realiza sometiendo un prototipo casi operacional del sistema relacionado con la seguridad a los datos de entrada típicos del entorno previsto en operación. El análisis es satisfactorio si el comportamiento observado del sistema relacionado con la seguridad es conforme al comportamiento requerido. Es necesario corregir toda disfunción del sistema relacionado con la seguridad y analizar de nuevo la nueva versión operacional.

Referencias:

The Concept of Dynamic Analysis. T. Ball, ESEC/FSE '99, Lecture Notes in Computer Science, Springer, 1999, ISBN 978-3-540-66538-0.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.6.6 Análisis de disfunciones

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

B.6.6.1 Análisis de los modos de disfunción y de sus efectos (AMFE)

Objetivo: Analizar un diseño del sistema estudiando sistemáticamente todas las fuentes posibles de disfunciones de los componentes de este sistema y determinando los efectos de estas disfunciones en el comportamiento y la seguridad del sistema.

Descripción: El análisis se hace generalmente en una reunión de ingenieros. Los componentes de un sistema se analizan uno después de otro para establecer un conjunto de modos de disfunción del componente, sus causas y sus efectos (localmente y a nivel global del sistema), así como los procedimientos y las recomendaciones para la detección. Si se tienen en cuenta las recomendaciones, se documentan como acciones de corrección que se han tomado.

Referencias:

IEC 60812:2006 *Técnicas de análisis de la fiabilidad de sistemas. Procedimiento de análisis de los modos de disfunción y de sus efectos (AMFE)*.

Risk Assessment and Risk Management for the Chemical Process Industry. H.R. Greenberg, J.J. Cramer, John Wiley and Sons, 1991, ISBN 0471288829, 9780471288824.

Reliability Technology. A. E. Green, A. J. Bourne, Wiley-Interscience, 1972, ISBN 0471324809.

B.6.6.2 Diagramas de causa-consecuencia

NOTA Esta técnica/medida se menciona en las tablas B.3, B.4, C.13 y C.14 de la Norma IEC 61508-3.

Objetivo: Analizar y modelizar, de forma gráfica compacta, las secuencias de los eventos que se pueden desarrollar en un sistema como consecuencia de una combinación de eventos básicos.

Descripción: Esta técnica se puede considerar como una combinación de análisis por árbol de disfunciones y por árbol de eventos. Comienza desde un evento crítico (iniciador) y el gráfico de consecuencias se traza hacia adelante utilizando puertas SI/NO que describen el éxito o la disfunción de algunas operaciones. Esto permite construir secuencias de eventos que conducen, o bien a un accidente o bien a una situación controlada. A continuación de construyen gráficos de causas para cada disfunción (es decir, árboles de disfunciones). Después, partiendo de una situación de accidente y yendo hacia atrás se obtiene un árbol de disfunciones global con esta situación de accidente como evento superior. Hacia adelante, se determinan las posibles consecuencias del evento. El gráfico puede incorporar símbolos en los vértices que describen las condiciones de propagación por las distintas ramas partiendo del vértice. También se pueden incluir retardos. Estas condiciones también se pueden describir con los árboles de disfunción. Las líneas de propagación se pueden asociar con símbolos lógicos para hacer el diagrama más compacto. Hay definido un conjunto de símbolos normalizados para utilizarlos en los diagramas de causa-consecuencia. Los diagramas se pueden utilizar para generar los árboles de disfunción y para calcular la probabilidad de ocurrencia de ciertas consecuencias críticas. También pueden utilizarse para producir árboles de eventos.

Referencias:

IEC 62502¹⁾ *Técnicas de análisis de la confiabilidad. Análisis por árbol de eventos (ETA)*.

The Cause Consequence Diagram Method as a Basis for Quantitative Accident Analysis. B. S. Nielsen, Danish Atomic Energy Commission, Riso-M-1374, 1971.

B.6.6.3 Análisis por árbol de eventos (ETA)

NOTA Esta técnica/medida se menciona en las tablas B.4 y C.14 de la Norma IEC 61508-3.

Objetivo: Modelizar, de forma gráfica, la secuencia de los eventos que se pueden desarrollar en un sistema después de un evento inicial e indicar también las consecuencias graves que se pueden producir. Un árbol de eventos es difícil de construir partiendo de cero y la utilización del diagrama de consecuencias es útil.

Descripción: En la parte superior del diagrama aparecen las condiciones de la secuencia que son pertinentes para la progresión de los eventos siguientes al evento inicial. Partiendo del evento inicial, que es el objetivo del análisis, se traza una línea hacia la primera condición de la secuencia. Aquí, el diagrama se divide en dos ramas, “sí” y “no”, describiendo como los eventos futuros dependen de la condición. Para cada una de estas ramas, se continúa hacia la condición siguiente de la misma forma. No todas las condiciones son pertinentes para todas las ramas. Se continúa hasta el final de la secuencia y cada rama del árbol construido representa una consecuencia posible. Siempre que las condiciones de las secuencias sean independientes, el árbol de eventos puede servir para calcular la probabilidad de las diferentes consecuencias, basada en la probabilidad y número de condiciones en la secuencia. Como las condiciones son raramente completamente independientes, tal cálculo debe considerarse y realizarse por analistas cualificados.

1) En estudio.

Referencias:

IEC 62502²⁾ *Técnicas de análisis de la confiabilidad. Análisis por árbol de eventos (ETA)*.

Risk Assessment and Risk Management for the Chemical Process Industry. H.R. Greenberg, J.J. Cramer, John Wiley and Sons, 1991, ISBN 0471288829, 9780471288824.

B.6.6.4 Análisis de los modos de disfunción, de sus efectos y de su criticidad (FMECA)

NOTA Esta técnica/medida se menciona en las tablas A.10, B.4, C.10 y C.14 de la Norma IEC 61508-3.

Objetivo: Establecer un orden de criticidad de los componentes que puedan ser el origen de una herida, un daño o una degradación del sistema a través de disfunciones únicas, con el fin de determinar qué componentes pueden necesitar una atención particular y medidas de control durante el diseño o la operación.

Descripción: Este método es comparable al AMFE, pero hay una o más columnas para indicar la criticidad, que se puede clasificar de varias formas. El método más laborioso se describe por la *Society for Automotive Engineers* (SAE) en el documento ARP 926. En este procedimiento, el grado de criticidad de un componente se indica por el número de disfunciones de un tipo particular esperadas durante el transcurso de cada millón de operaciones que se producen en un modo crítico. El grado de criticidad es una función de nueve parámetros, la mayoría de los cuales se deben medir. Un método muy simple para determinar la criticidad es multiplicar la probabilidad de disfunción del componente por el daño que se podría causar; este método es similar a la evaluación del factor simple de riesgo.

Referencias:

IEC 60812:2006 *Técnicas de análisis de la fiabilidad de sistemas. Procedimiento de análisis de los modos de disfunción y de sus efectos (AMFE)*.

Software criticality analysis of COTS/SOUP. P.Bishop, T.Clement, S.Guerra. In *Reliability Engineering & System Safety*, Volume 81, Issue 3, September 2003, Elsevier Ltd., 2003.

Software FMEA techniques. P.L.Goddard. In *Proc Annual 2000 Reliability and Maintainability Symposium*, IEEE, 2000, ISBN: 0-7803-5848-1.

B.6.6.5 Análisis por árbol de disfunciones (AAF)

NOTA Esta técnica/medida se menciona en las tablas B.4 y C.14 de la Norma IEC 61508-3.

Objetivo: Facilitar el análisis de eventos o de asociaciones de eventos que tendrían una consecuencia grave o peligrosa y realizar el cálculo de probabilidad del evento superior.

Descripción: Partiendo de un evento que fuera la causa inmediata de una consecuencia grave o peligrosa ("evento superior"), se realiza el análisis con el fin de identificar las causas de dicho evento. Esto se realiza en varios pasos usando operadores lógicos (y, o, etc.). Las causas intermedias se analizan de la misma forma y así sucesivamente hasta los eventos básicos donde se para el análisis.

El método es gráfico y se utiliza un conjunto de símbolos normalizados para trazar el árbol de disfunciones. Al final del análisis el árbol de disfunciones representa la función lógica que liga los eventos básicos (generalmente disfunciones de componentes) al evento superior (la disfunción global del sistema). La técnica está destinada principalmente para el análisis de sistemas de hardware, pero también se ha intentado aplicar esta aproximación al análisis de disfunciones del software. Esta técnica puede usarse cualitativamente para el análisis de disfunciones (identificación de escenarios de disfunción: grupos mínimos de corte o implicantes principales), semicuantitativamente (mediante la graduación de escenarios de acuerdo a sus probabilidades) o cuantitativamente para los cálculos de probabilidad del evento superior (véase C.6).

2) En estudio.

Referencias:

IEC 61025:2006 *Análisis por árbol de disfunción (AAF)*.

From safety analysis to software requirements. K.M. Hansen, A.P. Ravn, A.P. V Stavridou. IEEE Trans Software Engineering, Volume 24, Issue 7, Jul 1998.

B.6.6.6 Modelos de Markov

NOTA Véase el apartado B.1 de la Norma IEC 61508-6 para la utilización de esta técnica frente a los diagramas de bloques de fiabilidad, en el contexto del análisis de integridad de seguridad del hardware.

Objetivo: Modelizar el comportamiento del sistema mediante un gráfico de transición de estados y evaluar los parámetros de probabilidad del sistema (por ejemplo, la no-fiabilidad, no-disponibilidad, MTTF, MUT, MDT, etc.).

Descripción: Es un autómata de estados finitos (véase B.2.3.2) representado mediante un gráfico dirigido. Los nodos (círculos) representan los estados y los arcos (flechas) entre nodos representan las transiciones (disfunciones, reparaciones, etc.) que se producen entre los estados. Los arcos están ponderados con las correspondientes tasas de disfunción o tasas de reparación. La propiedad fundamental de los procesos homogéneos de Markov es que el futuro sólo depende del presente: un cambio de estado, N , al siguiente estado, $N+1$, es independiente del estado previo, $N-1$. Esto implica que todas las leyes de probabilidad de los modelos son exponenciales.

Los eventos de disfunción, estados y tasas pueden detallarse de manera que se obtiene una descripción precisa del sistema, por ejemplo disfunciones detectadas y no detectadas, manifestación de una disfunción mayor, etc. Los intervalos de ensayo de prueba pueden modelarse adecuadamente utilizando los llamados procesos multifase de Markov donde las probabilidades de los estados al final de una fase (por ejemplo, justo antes del ensayo de prueba) pueden utilizarse para calcular las condiciones iniciales para la siguiente fase (por ejemplo, las probabilidades de varios estados después de que se haya realizado un ensayo de prueba).

La técnica de Markov es adecuada para modelizar sistemas múltiples en los que el nivel de redundancia varía con el tiempo debido a la disfunción y reparación de componentes. Otros métodos clásicos, por ejemplo AMFE y AAF no pueden adaptarse rápidamente para modelizar los efectos de disfunciones a lo largo del ciclo de vida del sistema puesto que no existe una fórmula combinatoria simple para calcular las probabilidades correspondientes.

En los casos más simples, las fórmulas que describen las probabilidades del sistema están fácilmente disponibles en la literatura o pueden calcularse manualmente y también existen algunos métodos de simplificación (es decir, reducir el número de estados) para manejar los casos más complejos.

En cualquier caso, matemáticamente hablando, un gráfico de Markov homogéneo es un grupo simple y común de ecuaciones diferenciales lineales con coeficientes constantes. Esto ha sido analizado durante mucho tiempo y se han desarrollado poderosos algoritmos que están disponibles para manejarlas. En consecuencia, cuando el tamaño del modelo aumenta, es muy eficaz utilizar estos algoritmos que están implementados en varios paquetes de software.

Hay que advertir que el tamaño del gráfico aumenta exponencialmente con el número de componentes: esta es la llamada explosión combinatoria. En consecuencia, esta técnica es utilizable sin aproximaciones sólo para sistemas pequeños.

Cuando no se tienen que manejar leyes exponenciales –modelos de semi-Markov – debería utilizarse la simulación de Monte Carlo (véase B.6.6.8).

Referencias:

IEC 61165:2006 *Aplicación de las técnicas de Markov*.

The Theory of Stochastic Processes. R. E. Cox and H. D. Miller, Methuen and Co. Ltd., London, UK, 1963.

Finite MARKOV Chains. J. G. Kemeny and J. L. Snell. D. Van Nostrand Company Inc, Princeton, 1959.

The Theory and Practice of Reliable System Design. D. P. Siewiorek and R. S. Swarz, Digital Press, 1982.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.6.6.7 Diagramas de Bloques de Fiabilidad (RBD)

NOTA 1 Esta técnica/medida se utiliza en el anexo B de la Norma IEC 61508-6.

NOTA 2 Véase también el apartado C.6.4 “Diagramas de Bloques de Fiabilidad”

Objetivo: Modelizar, en forma de diagrama, el grupo de eventos que deben tener lugar y las condiciones que deben cumplirse para la operación exitosa de un sistema o tarea.

Descripción: El objetivo del análisis se representa como un camino de éxito formado por bloques, líneas y funciones lógicas. Un camino de éxito comienza en un lado del diagrama y continúa a través de los bloques y uniones hacia el otro lado del diagrama. Un bloque representa una condición o un evento y el camino puede pasar si la condición es verdadera o el evento ha tenido lugar. Si el camino llega a una unión, se continua si la lógica se la unión se cumple. Si se alcanza un vértice, se puede continuar a lo largo de las líneas salientes. Si existe al menos un camino de éxito a través del diagrama, es que el objetivo del análisis está funcionando correctamente.

Un RBD es una representación estructural del sistema modelado. Es una especie de circuito eléctrico: cuando la corriente encuentra un camino desde la entrada hasta la salida, significa que el sistema modelado está funcionando correctamente, cuando el circuito se corta significa que el sistema modelado ha fallado. Esto conduce al concepto de grupos mínimos de corte, que representan combinaciones de disfunciones (es decir, lugares donde el RBD se “corta”) conduciendo a la disfunción del sistema modelado.

Matemáticamente un RBD es similar a un árbol de disfunción. Representa la función lógica que liga los estados de los componentes individuales (que fallan o funcionan bien) al estado del sistema completo (que falla o funciona bien). En consecuencia, los cálculos son similares a los descritos para los árboles de disfunción.

Referencias:

IEC 61078:2006 *Técnicas de análisis de la confiabilidad. Método del diagrama de bloques de la fiabilidad y métodos booleanos*.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.6.6.8 Simulación de Monte Carlo

NOTA Esta técnica/medida se menciona en la tabla B.4 de la Norma IEC 61508-3 y se utiliza en la Norma IEC 61508-6, anexo B.

Objetivo: Simular los fenómenos del mundo real mediante la generación de números aleatorios cuando los métodos analíticos no son practicables.

Descripción: Las simulaciones de Monte Carlo se utilizan para resolver dos clases de problemas:

- probabilístico, cuando se utilizan números aleatorios para generar fenómenos estocásticos; y
- determinístico, cuando son matemáticamente traducidos en un problema equivalente de probabilidad (por ejemplo, cálculos integrales).

El principio de la simulación de Monte Carlo es la utilización de números aleatorios para animar un modelo de comportamiento funcional o disfuncional del sistema en estudio. Tales modelos de comportamiento son suministrados mediante modelos de transición de estados (gráfico de Markov, redes de Petri, lenguajes formales, etc.). La simulación de Monte Carlo se ejecuta para producir una muestra estadística grande a partir de la cual se obtienen los resultados estadísticos.

Cuando se utilizan las simulaciones de Monte Carlo debe tenerse cuidado en asegurar que las desviaciones, tolerancias o ruido tienen valores razonables. Esto debe gestionarse a través del intervalo de confianza que se obtiene fácilmente a partir de las simulaciones. Al contrario que los modelos analíticos, la simulación de Monte Carlo es auto-aproximativa. Los eventos despreciables sencillamente no aparecen sin necesidad de identificarlos para simplificar el modelo.

Un principio general de las simulaciones de Monte Carlo es restablecer y reformular el problema de manera que los resultados obtenidos sean tan precisos como sea posible en vez de enfrentar el problema como se estableció inicialmente.

En el contexto de esta norma, la simulación de Monte Carlo puede utilizarse para los cálculos de SIL y para tener en cuenta las incertidumbres de los datos de fiabilidad. Con los ordenadores actuales, los cálculos de SIL 4 son fácilmente alcanzables.

Referencias:

Monte Carlo Methods. J. M. Hammersley, D. C. Handscomb, Chapman & Hall, 1979.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.6.6.9 Modelos de árbol de disfunciones

NOTA 1 Véase la Norma IEC 61508-6 para la utilización de esta técnica en el contexto del análisis de la integridad de seguridad del hardware.

NOTA 2 Los árboles de disfunciones ya han sido descritos anteriormente como medios de validación de la seguridad (véase B.6.6.5). Son también ampliamente utilizados para el análisis de disfunciones y los cálculos de probabilidad.

Objetivo: Construir, mediante una aproximación gráfica sistemática descendente, la función lógica que liga los eventos básicos (modos de disfunción) al evento más significativo (evento no deseado).

Descripción: Esto es tanto un método de análisis que ayuda al analista a desarrollar el modelo paso a paso como un modelo matemático para el cálculo de probabilidades. Permite realizar:

- análisis cualitativo mediante la identificación y clasificación de escenarios de disfunción (grupos mínimos de corte o implicantes principales);
- análisis semicuantitativo mediante la graduación de escenarios de acuerdo con sus probabilidades de ocurrencia;
- análisis cuantitativo mediante el cálculo de la probabilidad del evento más significativo.

Como los RBD (Diagramas de Bloques de Fiabilidad), un árbol de disfunción representa la función lógica (de Boole) que liga los estados de los componentes individuales (funcionando o en disfunción) al estado del sistema completo (funcionando o en disfunción). En consecuencia, cuando los componentes son independientes, los cálculos de probabilidad pueden realizarse simplemente aplicando las propiedades básicas de las probabilidades aplicadas a la función lógica. Esto no es fácil puesto que este es un modelo estático que básicamente funciona sólo con probabilidades genuinas (es decir, constantes). Las probabilidades dependientes del tiempo deben manejarse cuidadosamente. Por ejemplo la PFD_{avg} de los sistemas de seguridad que comprenden componentes periódicamente ensayados no pueden calcularse directamente y esto es incluso más difícil para la PFH de los sistemas que funcionan en modo continuo. En consecuencia, sólo los ingenieros de fiabilidad con un profundo entendimiento de las matemáticas subyacentes deberían realizar los cálculos de no-disponibilidad/PFD y de no-fiabilidad/PFH con este método.

Los cálculos pueden realizarse a mano para árboles de disfunción muy sencillos, pero a lo largo de los últimos 50 años se han desarrollado e implementado muchos algoritmos para manejar las ecuaciones lógicas complejas. El estado del arte en la actualidad es utilizar BDD (Binary Decision Diagrams), que es una técnica de codificación compacta de la ecuación lógica en la memoria de un ordenador. En la actualidad, es el único método que puede realizar los cálculos de probabilidad sin aproximaciones en sistemas de tamaño industrial. Es también lo suficientemente rápido como para permitir manejar las incertidumbres mediante la simulación de Monte Carlo.

Referencia:

IEC 61025:2006 *Análisis por árbol de disfunción (AAF)*.

B.6.6.10 Modelos estocásticos generalizados de red de Petri (GSPN)

NOTA 1 Véase la Norma IEC 61508-6 para la utilización de esta técnica en el contexto del análisis de la integridad de seguridad del hardware.

NOTA 2 Las redes de Petri ya han sido mencionadas como método semiformal (véase B.2.3.3). También pueden ser utilizadas eficazmente en el contexto de la integridad de seguridad del hardware.

Objetivo: Construir gráficamente un modelo funcional y disfuncional que se comporte tan parecido como sea posible al sistema real modelado con el fin de suministrar un soporte eficaz para la simulación de Monte Carlo.

Descripción: Este es un autómata asíncrono de estados finitos como el descrito en el apartado B.2.3.3, excepto que la propiedad buena trazada cuando se realiza la validación formal no existe cuando se modela el comportamiento disfuncional de un sistema relacionado con la seguridad. Los llamados nodos (representados mediante círculos) representan los estados potenciales y las llamadas transiciones (representadas mediante rectángulos) representan los eventos que probablemente ocurran. Además del marcado de nodos (véase B.2.3.3) pueden utilizarse los mensajes o predicados para validar (habilitar) las transiciones y el retardo que transcurre desde la validación de una transición hasta su lanzamiento puede ser determinista o estocástico. Eso es por lo que estas Redes de Petri se llaman Redes de Petri “estocásticas generalizadas”.

Las redes de Petri constituyen modelos flexibles de comportamiento que se han probado como muy eficaces como apoyo a la simulación de Monte Carlo (véase B.6.6.8). Excepto la precisión de la propia simulación de Monte Carlo que, en cualquier caso, es siempre conocida, todas las limitaciones de los otros métodos (dependencia, explosión combinatoria, leyes no exponenciales, etc.) son superadas. Con los ordenadores actuales esto ha dejado de ser un problema incluso para las evaluaciones de SIL4.

Referencias:

IEC 62551³⁾ *Técnicas de análisis de la confiabilidad. Modelizado con red de Petri (CDI)*.

Sécurisation des architectures informatiques. Jean-Louis Boulanger, Hermès – Lavoisier, 2009, ISBN: 978-2-7462-1991-5.

B.6.7 Análisis del caso más desfavorable

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Evitar las disfunciones sistemáticas que provienen de las asociaciones desfavorables de las condiciones del entorno y de las tolerancias de los componentes.

Descripción: La capacidad operativa de un sistema y el dimensionamiento de los componentes se estudian o se calculan sobre una base teórica. Las condiciones del entorno se modifican hasta sus valores marginales más altos admisibles. Las respuestas esenciales del sistema se revisan y se comparan con la especificación.

B.6.8 Ensayo funcional extendido

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Revelar las disfunciones en el transcurso de las fases de especificación, de diseño y de desarrollo. Controlar el comportamiento del sistema relacionado con la seguridad en caso de entradas raras o no especificadas.

3) En estudio.

Descripción: El ensayo funcional extendido estudia el comportamiento funcional del sistema relacionado con la seguridad en respuesta a las condiciones de entrada que solo se supone que se producen raramente (por ejemplo, una disfunción grave) o que están fuera de la especificación del sistema relacionado con la seguridad (por ejemplo, una mala utilización). Para las condiciones raras, el comportamiento observado del sistema relacionado con la seguridad se compara con la especificación. Cuando la respuesta del sistema relacionado con la seguridad no se especifica, hay que controlar que la seguridad de la planta se preserve con la respuesta observada.

Referencias:

Software Testing and Quality Assurance. K. Naik, P. Tripathy, Wiley Interscience, 2008, Print ISBN: 9780471789116 Online ISBN: 9780470382844.

The Art of Software Testing, Second Edition. G. Myers et al., Wiley & Sons, New York, 2004, ISBN 0471469122, 9780471469124.

Dependability of Critical Computer Systems 3. P. G. Bishop et al., Elsevier Applied Science, 1990, ISBN 1-85166-544-7.

B.6.9 Ensayo del caso más desfavorable

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Ensayar los casos especificados durante el análisis de los casos más desfavorables.

Descripción: La capacidad operativa del sistema y el dimensionamiento de los componentes se ensayan en las condiciones de los casos más desfavorables. Las condiciones del entorno se modifican hasta sus valores marginales más altos admisibles. Las respuestas esenciales del sistema se revisan y se comparan con la especificación.

B.6.10 Ensayo de inserción de fallos

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.6 de la Norma IEC 61508-2.

Objetivo: Introducir o simular fallos en el hardware del sistema y documentar su respuesta.

Descripción: Se trata de un método cualitativo de evaluación de la confiabilidad. Es preferible utilizar los esquemas de cableado, los esquemas eléctricos y los esquemas de bloques funcionales detallados para describir la ubicación y tipo de fallo y la forma en la que se introduce; por ejemplo: la alimentación de diferentes módulos se puede cortar; la alimentación, el bus o las líneas de direcciones se pueden poner en cortocircuito o en circuito abierto; algunos componentes o sus puertos se pueden abrir o cortocircuitar; los relés pueden no cerrarse o no abrirse o hacerlo en un mal momento, etc. Las disfunciones del sistema resultantes se clasifican como en las tablas 1 y 2 de la Norma IEC 60812, por ejemplo. En principio, se introducen los fallos únicos, en régimen estable. Sin embargo, si un fallo no se identifica por los ensayos de diagnóstico integrados o no es evidente, se puede dejar en el sistema y se puede estudiar el efecto de un segundo fallo. El número de fallos fácilmente puede llegar a centenares.

El trabajo se realiza con un equipo multidisciplinar y el fabricante del sistema debería estar presente y ser consultado. El tiempo medio de funcionamiento entre disfunciones para los fallos que tienen graves consecuencias se debería calcular o estimar. Si el tiempo calculado es corto, se deberían realizar modificaciones.

Referencias:

IEC 60812:2006 *Técnicas de análisis de la fiabilidad de sistemas. Procedimiento de análisis de los modos de disfunción y de sus efectos (AMFE)*.

IEC 61069-5:1994 *Medida y control en los procesos industriales. Apreciación de las propiedades de un sistema con el fin de su evaluación. Parte 5: Evaluación de la seguridad de funcionamiento de un sistema*.

ANEXO C (Informativo)**PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA LA OBTENCIÓN
DE LA INTEGRIDAD DE SEGURIDAD DEL SOFTWARE
(VÉASE LA NORMA IEC 61508-3)****C.1 Generalidades**

La presentación de las técnicas contenidas en este anexo no debería considerarse como completa o exhaustiva.

C.2 Requisitos y diseño detallado

NOTA Las técnicas y medidas se describen en el capítulo B.2.

C.2.1 Métodos estructurados de diagrama

NOTA Esta técnica/medida se menciona en las tablas A.2 y A.4 de la Norma IEC 61508-3.

C.2.1.1 Generalidades

Objetivo: El objetivo principal de los métodos estructurados es promover la calidad de desarrollo del software centrando la atención sobre las primeras partes del ciclo de vida. Para hacer esto, los métodos se apoyan en procedimientos y notaciones precisos e intuitivos (asistidos por ordenador), para determinar y documentar los requisitos y las características de implementación en un orden lógico y de forma estructurada.

Descripción: Existe toda una gama de métodos estructurados. Algunos están diseñados para las funciones tradicionales de procesamiento y de transacciones de datos mientras otros están más orientados al control de procesos y a aplicaciones en tiempo real (que tienden a ser más críticos para la seguridad). UML (véase C.3.12) contiene muchos ejemplos de notaciones estructuradas.

Los métodos estructurados son principalmente “herramientas de razonamiento” que permiten la percepción y la partición sistemática de un problema o de un sistema. Sus principales características son:

- orden lógico de razonamiento, partición de un problema importante en partes más fácilmente manejables;
- análisis y documentación del sistema total, incluyendo el entorno así como el sistema requerido;
- descomposición de los datos y de la función del sistema requerido;
- listas de control, es decir, listas de elementos que necesitan un análisis;
- baja sobrecarga intelectual - métodos simples, intuitivos y pragmáticos;
- frecuentemente con gran énfasis en el desarrollo de un modelo de diagrama del sistema previsto y en el soporte de la herramienta CASE para el método global.

Las notaciones de apoyo para el análisis y la documentación de problemas y de entidades del sistema (por ejemplo, los flujos de datos y de procesos) tienden a ser precisas, pero las notaciones que expresan las funciones de procesamiento realizadas por estas entidades tienden a ser más informales. Sin embargo, algunos métodos utilizan en parte notaciones matemáticas formales (por ejemplo, expresiones regulares o las máquinas de estados finitos). Una alta precisión no solamente reduce la importancia de los errores de comprensión sino que permite el procesamiento automático.

Otra ventaja de las notaciones estructuradas es su visibilidad que permite la verificación intuitiva de una especificación o de un diseño por el usuario en función de su profundo pero no formalizado conocimiento.

Esta visión general describe varios métodos estructurados de forma más precisa.

Referencias:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Software Design. D. Budgen, Pearson Education, 2003, ISBN 0201722194, 9780201722192.

C.2.1.2 Expresión controlada de requisitos (CORE, Controlled Requirements Expression)

Objetivo: Asegurarse de que todos los requisitos se han determinado y expresado.

Descripción: Este método tiene por objetivo salvar las distancias entre el cliente/usuario final y el analista. No es matemáticamente riguroso pero facilita el proceso de comunicación (el método CORE permite la expresión de los requisitos mejor que la especificación). La aproximación se estructura y la expresión pasa por diferentes niveles de afinamiento. El método CORE permite una vista más amplia del problema, teniendo en cuenta el conocimiento del entorno en el que el sistema será utilizado así como puntos de vista de los distintos tipos de usuarios. Este método incluye las directrices y la táctica a utilizar para reconocer las diferencias en relación al “diseño general”. Las diferencias se pueden corregir o identificar explícitamente y documentar. Las especificaciones pueden no ser completas pero los problemas no resueltos y las zonas de alto riesgo son detectados y se deben tener en cuenta en el diseño posterior.

Referencias:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Requirements Engineering. E. Hull, K. Jackson, J. Dick. Springer, 2005, ISBN 1852338792, 9781852338794.

C.2.1.3 Desarrollo de Jackson del sistema (JSD, Jackson System Development)

Objetivo: Método de desarrollo que cubre el desarrollo de los sistemas software desde los requisitos hasta el código, prestando una atención particular a los sistemas en tiempo real.

Descripción: El método JSD es un procedimiento de desarrollo en varias fases en las que el desarrollador modeliza el comportamiento del mundo real que sirve de base a las funciones del sistema, determina las funciones requeridas y las introduce en el modelo, después transforma la especificación resultante en una especificación realizable en el entorno objetivo. Por tanto, cubre las fases tradicionales de especificación, diseño y desarrollo pero es diferente de los métodos tradicionales por el hecho de que no es de diseño descendente.

Además, este método presta una gran atención a la etapa inicial de descubrimiento de entidades del mundo real que son de interés para el sistema que se está construyendo y a su modelización y a qué puede ocurrir con ellas. Una vez realizado el análisis del “mundo real” y creado el modelo, las funciones requeridas del sistema se analizan con el fin de determinar como se pueden integrar en ese mundo real. Las descripciones estructuradas de todos los procesos del modelo se adjuntan al modelo del sistema resultante y todo se transforma después en programas susceptibles de funcionar en el entorno del software y del hardware previsto.

Referencias:

Systems Analysis and Design. D. Yeates, A. Wakefield. Pearson Education, 2003, ISBN 0273655361, 9780273655367.

An Overview of JSD. J. R. Cameron. IEEE Transactions on Software Engineering, SE-12, No. 2, February 1986.

C.2.1.4 Yourdon en tiempo real

Objetivo: Especificación y diseño de los sistemas en tiempo real.

Descripción: El esquema de desarrollo que sirve de base a esta técnica implica una evolución en tres etapas del sistema en desarrollo. La primera etapa comprende la construcción de un “modelo esencial” que describe el comportamiento requerido por el sistema. La segunda comprende la construcción de un modelo de implementación que describe las estructuras y los mecanismos que, cuando son implementados, contienen el comportamiento requerido. La tercera etapa comprende la realización propiamente dicha del sistema en el hardware y en el software. Las tres etapas corresponden aproximadamente a las fases tradicionales de especificación, diseño y desarrollo pero prestan una gran atención al hecho de que en cada etapa el desarrollador tiene que realizar una actividad de modelización.

El modelo básico está constituido por dos partes:

- el modelo del entorno, que contiene una descripción de la frontera entre el sistema y su entorno así como una descripción de los eventos externos a los que es necesario que el sistema responda; y
- el modelo de comportamiento, que contiene los esquemas que describen la transformación realizada por el sistema en respuesta a los eventos así como una descripción de los datos que es necesario que el sistema tenga para responder.

El modelo de implementación se divide también en submodelos, que cubren la asignación de los procesos individuales a los procesadores y la descomposición de los procesos en módulos del software.

La técnica de adquisición de estos modelos incluye un cierto número de otras técnicas bien conocidas: diagramas de flujos de datos, gráficos de transformación, inglés estructurado, diagramas de transición de estados y redes de Petri. Además, este método incluye las técnicas para simular el diseño propuesto del sistema bien sobre papel o bien mecánicamente a partir de los modelos realizados.

Referencias:

Real-time Systems Development. R. Williams. Butterworth-Heinemann, 2006, ISBN 0750664711, 9780750664714.

Structured Development for Real-Time Systems (3 Volumes). P. T. Ward and S. J. Mellor. Yourdon Press, 1985.

C.2.2 Diagramas de flujos de datos

NOTA Esta técnica/medida se menciona en las tablas B.5 y B.7 de la Norma IEC 61508-3.

Objetivo: Describir el flujo de datos a través de un programa en forma de diagrama.

Descripción: Los diagramas de flujo de datos muestran como los datos de entrada se transforman en datos de salida, cada etapa del diagrama muestra una transformación diferente.

Los diagramas de flujo de datos tienen tres aspectos:

- flechas anotadas: representan los flujos de datos entrantes y salientes asociados a cada centro de transformación, con anotaciones documentando que es el dato;
- burbujas anotadas: representan los centros de transformación, con anotaciones documentando la transformación;
- operadores (y, o exclusivo): estos operadores sirven para unir las flechas anotadas.

Cada burbuja de un diagrama de flujo de datos se puede considerar como una caja negra autónoma que transforma sus entradas en salidas en cuanto las primeras estén disponibles. Una de las principales ventajas de las burbujas reside en el hecho de que muestran las transformaciones sin hacer ninguna suposición en lo referente a la forma en que estas transformaciones se implantan. Un diagrama de flujo de datos puro no incluye información de control o secuencial, estos elementos se tratan con extensiones de tiempo real en la notación, como en el caso del método Yourdon en tiempo real (véase C.2.1.4).

El mejor método para la preparación de los diagramas de flujo de datos consiste en considerar las entradas del sistema y tratarlas hasta las salidas correspondientes. Cada burbuja debe representar una transformación distinta - su salida debería ser, bajo cierto punto de vista, diferente de su entrada. No existen reglas para determinar la estructura general del diagrama y la construcción de un diagrama de flujo de datos es uno de los aspectos creativos del diseño de un sistema. Como todos los diseños, se trata de un procedimiento iterativo en el que los intentos se refinan hasta obtener el diagrama final.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

ISO 5807:1985 *Procesamiento de la información. Símbolos de documentación y convenciones aplicables a los datos, a los diagramas de flujo de programas y de sistemas, a los esquemas de redes de programas y de recursos de sistemas.*

ISO/IEC 8631:1989 *Tecnología de la información. Estructuras de programas y normas para su presentación.*

C.2.3 Diagramas de estructuras

NOTA Esta técnica/medida se menciona en la tabla B.5 de la Norma IEC 61508-3.

Objetivo: Mostrar la estructura de un programa en forma de diagrama.

Descripción: Los diagramas de estructuras constituyen una notación complementaria a los diagramas de flujo de datos. Describen el sistema de programación y la jerarquía de las diferentes partes en forma de gráfico, como un árbol. Muestran cómo los elementos de un diagrama de flujo de datos se pueden implementar como una jerarquía de unidades del programa.

Un diagrama de estructura muestra las relaciones entre los módulos del programa sin incluir las informaciones relativas al orden de activación de las unidades. Estos diagramas se realizan con la ayuda de los cuatro símbolos siguientes:

- rectángulo anotado con el nombre del módulo;
- línea que une los rectángulos creando una estructura;
- flecha con círculo (círculo vacío), anotada con el nombre de los datos intercambiados entre los elementos del diagrama de estructura (normalmente, la flecha con círculo es paralela a la línea que une los rectángulos del diagrama);
- flecha con círculo (círculo relleno), anotada con el nombre de la señal de control que pasa de un módulo al otro en el diagrama de estructura, la flecha es también paralela a la línea que une los dos módulos.

Es posible obtener un cierto número de diagramas de estructura diferentes a partir de un diagrama de flujo de datos no trivial.

Los diagramas de flujo de datos representan la relación entre las informaciones y las funciones del sistema. Los diagramas de estructura representan la forma en la que los elementos del sistema se implementan. Las dos técnicas, aunque diferentes, proporcionan una representación válida del sistema.

Referencias:

Software Design & Development. G. Lancaster. Pascal Press, 2001, ISBN 1741251753, 9781741251753.

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

C.2.4 Métodos formales

NOTA Esta técnica/medida se menciona en las tablas A.1, A.2, A.4 y B.5 de la Norma IEC 61508-3.

C.2.4.1 Generalidades

Objetivo: Desarrollo del software basándose en las matemáticas. Estos métodos constan de técnicas de diseño y de codificación formales.

Descripción: Los métodos formales permiten desarrollar la descripción de un sistema en alguna fase de su especificación, diseño o implementación. La descripción resultante está en una notación estricta que se puede someter a un análisis matemático para detectar las distintas clases de incoherencias o inexactitudes. Además, la descripción puede, en ciertos casos, analizarse con una máquina con un rigor similar a la verificación de la sintaxis de un programa fuente por un compilador, o animarse con el fin de visualizar los distintos aspectos del comportamiento del sistema descrito. La animación puede proporcionar una confianza adicional en lo referente a que el sistema cumple los requisitos reales así como los requisitos formales especificados, porque mejora la comprensión humana del comportamiento especificado.

Un método formal ofrece generalmente una notación (normalmente utilizando una forma de representación matemática discreta), una técnica que permite obtener una descripción con esa notación y diferentes formas de análisis que permiten verificar la descripción en cuanto a distintas propiedades de corrección.

En los siguientes apartados de esta norma se describen varios métodos formales: CCS, CSP, HOL, LOTOS, OBJ, lógica temporal, VMD y Z. Otras técnicas como las máquinas de estados finitos y las redes de Petri (véase el anexo B) pueden considerarse como métodos formales según el grado de conformidad de estas técnicas, según se utilicen, con una base matemática rigurosa.

Referencias:

Formal Specification: Techniques and Applications. N. Nissanke, Springer-Verlag Telos, 1999, ISBN 1852330023.

The Practice of Formal Methods in Safety-Critical Systems. S. Liu, V. Stavridou, B. Dutertre, J. Systems Software 28, 77-87, Elsevier, 1995.

Formal Methods: Use and Relevance for the Development of Safety-Critical Systems. L. M. Barroca, J. A. McDermid, The Computer Journal 35 (6), 579-599, 1992.

How to Produce Correct Software. An Introduction to Formal Specification and Program Development by Transformations. E. A. Boiten et al, The Computer Journal 35 (6), 547-554, 1992.

C.2.4.2 CCS – Calculus of Communicating Systems

Objetivo: CCS es un medio de describir y razonar el comportamiento de los sistemas correspondientes a los procesos concurrentes que se comunican.

Descripción: El método CCS es una técnica de cálculo matemático aplicable al comportamiento de los sistemas. El diseño de los sistemas se modeliza como una red de procesos independientes funcionando secuencialmente o en paralelo. Los procesos se comunican por medio de puertos (similares a los canales del método CSP), la comunicación solo se establece cuando ambos procesos están listos. El no determinismo se puede modelizar. A partir de una descripción abstracta de alto nivel del sistema completo (conocida con el nombre de traza), es posible afinar el sistema por pasos con el fin de obtener una composición de varios procesos que se comunican en donde el comportamiento total corresponde al requerido para el sistema entero. También es posible utilizar un diseño ascendente combinando los procesos y deduciendo las propiedades del sistema resultante utilizando las reglas de interferencias relacionadas con las reglas de composición.

Referencia:

Communication and Concurrency. R. Milner. Pearson Education, 1989, ISBN 9780131150072.

C.2.4.3 CSP – *Communicating Sequential Processes*

Objetivo: CSP es una técnica de especificación de sistemas de software concurrentes, es decir, de sistemas de procesos que se comunican funcionando concurrentemente.

Descripción: El método CSP proporciona un lenguaje para la especificación de sistemas de procesos y el ensayo para verificar que la implementación de los procesos satisface sus especificaciones (descritas como una traza, es decir, constituidas por una secuencia autorizada de eventos).

Un sistema se modeliza como una red de procesos independientes combinados secuencialmente o en paralelo. Cada proceso se describe en función de todos los comportamientos posibles. Los procesos pueden comunicarse (datos de sincronización o de intercambio) por medio de canales, la comunicación solo se establece cuando los dos procesos están listos. El tiempo relativo de los eventos se puede modelizar.

La teoría relativa al método CSP se ha incorporado en la arquitectura del "transputer" INMOS y el lenguaje OCCAM permite que un sistema especificado según el método CSP pueda implementarse directamente en la red de "transputer".

Referencia:

Communicating Sequential Processes: The First 25 Years. A. Abdallah, C. Jones, J. Sanders. (Eds.). Springer, 2004, ISBN 3540258132, 9783540258131.

C.2.4.4 HOL – *Higher Order Logic*

Objetivo: Este lenguaje formal está destinado para servir de base a la especificación y a la verificación del hardware.

Descripción: El método HOL utiliza una notación lógica particular y un sistema de máquina de soporte, ambos desarrollados por el laboratorio informático de la Universidad de Cambridge. La notación lógica se deriva en gran parte de la teoría simple de los tipos de Church y el sistema de máquina de soporte se basa en el sistema LCF (logic of computable functions) (lógica de las funciones calculables).

Referencia:

Higher-Order Computational Logic. J. Lloyd. In *Computational Logic: Logic Programming and Beyond*, Lecture Notes in Computer Science, Springer Berlin / Heidelberg, 2002, ISBN 978-3-540-43959-2.

C.2.4.5 LOTOS

Objetivo: LOTOS es un medio para describir y razonar el comportamiento de los sistemas correspondientes a los procesos concurrentes que se comunican.

Descripción: El método LOTOS (language for temporal ordering specification) se basa en el método CCS con ciertos elementos suplementarios de las álgebras relacionadas CSP y CIRCAL (circuit calculus). Suple las debilidades del método CCS en lo relativo a las estructuras de datos y a las expresiones de valor combinándolo con un segundo componente basado en el lenguaje de tipos de datos abstractos ACT ONE. El aspecto de la definición del proceso del método LOTOS se podría utilizar con otros formalismos para la descripción de los tipos de datos abstractos.

Referencias:

Model Checking for Software Architectures. R. Mateescu. In Software Architecture, Lecture Notes in Computer Science, Springer Berlin / Heidelberg, 2004, ISBN 978-3-540-22000-8.

ISO 8807:1989 *Sistemas de procesamiento de la información. Interconexión de sistemas abiertos. LOTOS. Técnica de descripción formal basada en la organización temporal del comportamiento observacional*.

C.2.4.6 OBJ

Objetivo: Proporcionar una especificación del sistema precisa con retorno del usuario y validación del sistema antes de la implementación.

Descripción: OBJ es un lenguaje de especificación algebraica. Los usuarios especifican los requisitos en términos de ecuaciones algebraicas. Los aspectos relacionados con el comportamiento o la realización del sistema se especifican en términos de operaciones actuando sobre los tipos de datos abstractos (ADT). Un tipo de datos abstractos es similar a un paquete ADA en donde el comportamiento operacional es visible mientras que los detalles de implementación están “ocultos”

Una especificación OBJ, así como la implementación posterior por pasos, se puede someter a las mismas técnicas de demostración formales que los otros métodos formales. Además, dado que los aspectos de la especificación OBJ relacionados con la construcción son ejecutables por una máquina, se puede realizar la validación del sistema directamente a partir de la especificación. La ejecución consiste principalmente en evaluar una función por sustitución de ecuaciones (reescritura) hasta que se obtenga un valor específico de salida. Esta ejecución permite al usuario final del sistema considerado tener una “vista” del mismo en el momento de la especificación del sistema, sin estar necesariamente al corriente de las técnicas de especificación formales asociadas.

Como otras técnicas ADT, OBJ es únicamente aplicable a los sistemas secuenciales o a los aspectos secuenciales de los sistemas concurrentes. OBJ se ha utilizado para la especificación de aplicaciones industriales de pequeña y gran escala.

Referencia:

Software Engineering with OBJ: Algebraic Specification in Action. J. Goguen, G. Malcolm. Springer, 2000, ISBN 0792377575, 9780792377573.

C.2.4.7 Lógica temporal

Objetivo: Expresión directa de los requisitos relativos a la seguridad y a la operación y demostración formal del mantenimiento de estas características durante los estados de desarrollo posteriores.

Descripción: La lógica de los predicados de primer orden estándar no consta de ningún concepto de tiempo. La lógica temporal aumenta las posibilidades de la lógica de primer orden añadiendo los operadores modales (por ejemplo “sucesivos” o “eventualmente”). Estos operadores se pueden utilizar para calificar aseveraciones relativas al sistema. Por ejemplo, se puede requerir que las propiedades de seguridad mantengan el operador “sucesivos”, mientras que otros estados del sistema deseados se puede requerir que se alcancen “eventualmente” a partir de otros estados iniciales. Las fórmulas temporales se interpretan durante la secuencia de estados (comportamientos). El concepto de estado depende del nivel de descripción elegido. Esto se puede aplicar al sistema entero, a un elemento del sistema o a un programa informático.

Las restricciones y los intervalos de tiempos cuantificados no se tratan de forma explícita en la lógica temporal. Los tiempos absolutos se deben tratar creando estados temporales suplementarios como parte de la descripción de estados.

Referencia:

Mathematical Logic for Computer Science. M. Ben-Ari. Springer, 2001, ISBN 1852333197, 9781852333195.

C.2.4.8 VDM, VDM++ – Vienna Development Method

Objetivo: Especificación e implementación sistemática de programas secuenciales (VDM) y de programas en tiempo real (VDM++) concurrentes.

Descripción: VDM es una técnica de especificación basada en las matemáticas y una técnica de afinamiento de las implementaciones de forma que permitan un control de conformidad en relación a la especificación.

La técnica de especificación se basa en modelos, en los que el estado del sistema se modeliza en términos de estructuras siguiendo la teoría de los conjuntos, que sirven para describir las invariantes (predicados), y las operaciones asociadas a este estado se modelizan especificando sus condiciones iniciales y sus condiciones finales en términos de estados del sistema. Las operaciones se pueden controlar con el fin de preservar las invariantes del sistema.

La implementación de la especificación se realiza por la materialización del estado del sistema en términos de estructuras de datos del lenguaje objetivo y por el perfeccionamiento de las operaciones en términos de programas en el lenguaje objetivo. Las fases de materialización y de perfeccionamiento proporcionan las obligaciones de control para establecer que son correctas. Es el diseñador el que elige si es obligatorio o no la ejecución de los controles.

El método VDM se utiliza principalmente durante la fase de especificación pero también se puede utilizar durante la fase de diseño y de implementación generando un código fuente. Es solamente aplicable a los programas secuenciales o a los procesos secuenciales de sistemas concurrentes.

La extensión de tiempo real concurrente y orientada a objetos de VDM, VDM++, es un lenguaje de especificación formal basado en el lenguaje ISO VDM-SL, y en el lenguaje orientado a objetos Smalltalk.

VDM++ suministra un rango amplio de constructores de forma que un usuario puede especificar formalmente y de forma orientada a objetos los sistemas concurrentes de tiempo real. En VDM++, una especificación formal completa consiste en una colección de especificaciones de clases, y opcionalmente un espacio de trabajo.

Las posibilidades de tiempo real de VDM++ son:

- las expresiones temporales, destinadas a denotar el momento actual y el momento de invocación de un método dentro del cuerpo del método;
- las postexpresiones datadas, unidas a un método para especificar los límites superiores e inferiores del instante de ejecución para una realización correcta del programa;
- se han introducido las variables temporales continuas. Con sentencias de suposiciones y efectos se pueden especificar relaciones (por ejemplo, las ecuaciones diferenciales) entre estas funciones de tiempo. Esta característica ha demostrado una gran utilidad en la especificación de requisitos relativos a los sistemas que funcionan en un entorno temporal continuo. Las etapas de perfeccionamiento conducen a soluciones de software discretas para este tipo de sistemas.

Referencias:

ISO/IEC 13817-1:1996 *Tecnología de la información. Lenguajes de programación, sus entornos e interfaces del software del sistema. Método de desarrollo de Viena. Lenguaje de especificación. Parte 1: Lenguaje de base*.

Systematic Software Development using VDM. C. B. Jones. Prentice-Hall. 2nd Edition, 1990.

Conformity Clause for VDM-SL, G. I. Parkin and B. A. Wichmann, *Lecture Notes in Computer Science 670*, FME'93 Industrial-Strength Formal Methods, First International Symposium of Formal Methods in Europe. Editors: J. C. P. Woodcock and P. G. Larsen. Springer Verlag, 501-520.

C.2.4.9 Z

Objetivo: Z es una notación de lenguaje de especificación para sistemas secuenciales y una técnica de diseño que permite al desarrollador pasar de una especificación Z a los algoritmos ejecutables de una forma que permite probar su conformidad en relación a la especificación.

Z se utiliza principalmente durante la etapa de especificación, pero se ha ideado un método para pasar de la especificación al diseño y a la implementación. Este método está más adaptado al desarrollo de sistemas secuenciales orientados dados.

Descripción: Como VDM, la técnica de especificación se basa en modelos en los que el estado del sistema se modeliza en términos de estructuras en la teoría de los conjuntos que sirven para definir las invariantes (predicados), y las operaciones asociadas a este estado se modelizan especificando sus precondiciones y sus postcondiciones en términos de estados del sistema. Se puede probar que las operaciones conservan las invariantes, demostrando así su coherencia. La parte formal de una especificación se divide en esquemas permitiendo la estructuración de las especificaciones por refinamiento.

Normalmente, una especificación Z es una mezcla del método Z formal y de texto explicativo informal en lenguaje natural (un texto formal es demasiado conciso para permitir una lectura fácil y a menudo necesita una explicación, mientras que el lenguaje natural informal puede ser fácilmente vago e impreciso).

Contrariamente al VDM, Z es una notación más que un método completo. Sin embargo, se ha desarrollado un método asociado (llamado método B) y se puede utilizar conjuntamente con el método Z. El método B se basa en el principio de refinamiento por pasos.

Referencias:

Formal Specification using Z, 2nd Edition. D. Lightfoot. Palgrave Macmillan, 2000, ISBN 9780333763278.

The B-Method. S. Schneider. Palgrave Macmillan, 2001, ISBN 9780333792841.

C.2.5 Programación defensiva

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-3.

Objetivo: Producir programas capaces de detectar los flujos de control o de datos anormales o los valores de datos anormales durante la ejecución y de reaccionar a estos fallos de forma predeterminada y aceptable.

Descripción: Se pueden utilizar un gran número de técnicas durante la programación para verificar la ausencia de fallos relacionados con los controles o con los datos. Estas técnicas se pueden aplicar sistemáticamente durante toda la programación de un sistema para reducir las probabilidades de procesamiento de datos erróneos.

Existen dos dominios de técnicas defensivas que se superponen. Un software intrínsecamente libre de errores se diseña para tener en cuenta sus propios defectos de diseño. Estos defectos se pueden deber a errores de diseño o de codificación o a requisitos erróneos. Las técnicas defensivas incluyen por ejemplo:

- un control del rango de las variables;
- un control de verosimilitud de los valores, en la medida de lo posible;
- un control del tipo, del dimensionamiento y del rango de los parámetros al principio del procedimiento.

Estas tres primeras recomendaciones permiten asegurar que los números manipulados por el programa son razonables, en términos de función del programa y de significado físico de variables.

Los parámetros solo de lectura y de lectura-escritura deberían separarse y deberían comprobarse sus accesos. Las funciones deberían tratar todos los parámetros como parámetros solo de lectura. Las constantes literales no deberían ser accesibles para escritura. Esto permite detectar cualquier sobreescritura accidental o mala utilización de las variables.

Un software tolerante a los fallos se diseña para “prever” las disfunciones en su propio entorno o la utilización de condiciones desviadas de las condiciones nominales o previstas, y reaccionar de una forma predeterminada. Las técnicas incluyen lo siguiente:

- control de verosimilitud de las variables de entrada y de las variables intermedias con significado físico;
- control del efecto de las variables de salida, preferiblemente por observación directa de los cambios de estado del sistema asociado;
- control de la configuración del software, incluyendo la existencia y accesibilidad del hardware previsto y de la completitud del software. Este es particularmente importante para mantener la integridad después de los procedimientos de mantenimiento.

Ciertas técnicas de programación defensiva, tales como la comprobación de secuencias de los flujos de control, también tienen en cuenta las disfunciones externas.

Referencias:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Dependability of Critical Computer Systems: Guidelines Produced by the European Workshop on Industrial Computer Systems, Technical Committee 7 (EWICS TC7, Systems Reliability, Safety, and Security). Elsevier Applied Science, 1989, ISBN 1851663819, 9781851663811.

C.2.6 Reglas de diseño y de codificación

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-3.

C.2.6.1 Generalidades

Objetivo: Facilitar la verificabilidad, fomentar un estudio objetivo en equipo y aplicar un método de diseño estándar.

Descripción: Las reglas a seguir se acuerdan entre los participantes al principio del proyecto. Estas reglas constan de los métodos de diseño y desarrollo a seguir (por ejemplo, JSP, las redes de Petri, etc.) y las reglas de codificación asociadas (véase C.2.6.2).

Estas reglas se establecen para facilitar el desarrollo, la verificación, la evaluación y el mantenimiento. En consecuencia, se deberían tener en cuenta las herramientas disponibles, en particular analizadores y herramientas de ingeniería inversa.

Referencias:

IEC 60880:2006 *Centrales nucleares. Instrumentación y sistemas de control importantes para la seguridad. Aspectos lógicos (software) de los sistemas informáticos que realizan funciones de categoría A*.

Verein Deutscher Ingenieure. Software-Zuverlässigkeit. Grundlagen, Konstruktive Massnahmen, Nachweisverfahren. VDI-Verlag, 1993, ISBN 3-18-401185-2.

C.2.6.2 Reglas de codificación

NOTA Esta técnica/medida se menciona en la tabla B.1 de la Norma IEC 61508-3.

Objetivo: Reducir la probabilidad de errores en el código relacionado con la seguridad y facilitar su verificación.

Descripción: Los siguientes principios indican cómo las reglas de codificación relacionadas con la seguridad (para cualquier lenguaje de programación) pueden ayudar a cumplir los requisitos normativos de la Norma IEC 61508-3 y a lograr las “propiedades deseables” informativas (véase el anexo F). Deberían tenerse en cuenta las herramientas de soporte disponibles.

Requisitos y recomendaciones de la Norma IEC 61508-3	Sugerencias sobre reglas de codificación
Aproximación modular (tabla A.2-7, tabla A.4-4)	Límite de tamaño del módulo de software (tabla B.9-1) y control de la complejidad del software (tabla B.9-2). Ejemplos: • especificación del tamaño local y métrica y límites de complejidad (para módulos); • especificación de la métrica y límites de complejidad “global” (para organización global de módulos); • límite de número de parámetros/número fijo de parámetros de subprograma (tabla B.9-4). Ocultación/encapsulado de la información (tabla B.9-3): por ejemplo, incentivos para utilizar características particulares de lenguajes. Interfaz completamente definida (tabla B.9-6). Ejemplos: • especificación explícita de firmas de función; • programación de aserción de disfunción (tabla A.2-3a) y verificación de datos (7.9.2.7), con especificación explícita de pre-condiciones y post-condiciones para funciones, de aserciones, de invariantes de tipos de datos.
Inteligibilidad del código • Promover la inteligibilidad del código (7.4.4.13) • Legible, inteligible y ensayable (7.4.6)	Convenciones de nombrado que promuevan los nombres con significado y no ambiguos. Ejemplo: evitar nombres que podrían confundirse (por ejemplo, IO y 10). Nombres simbólicos para valores numéricos. Procedimientos y directrices para la documentación del código fuente (7.4.4.13). Por ejemplo: • Explicar los motivos y los significados (y no sólo qué). • Advertencias. • Efectos colaterales. Cuando sea practicable, la siguiente información debe contenerse en el código fuente (7.4.4.13): • Entidad legal (por ejemplo: empresa, autor(es), etc.). • Descripción. • Entradas y salidas. • Historia de la gestión de la configuración. (Véase también la aproximación modular).

Requisitos y recomendaciones de la Norma IEC 61508-3	Sugerencias sobre reglas de codificación
Verificabilidad y capacidad de ensayo • Facilitar la verificación y el ensayo (7.4.4.13) • Facilitar la detección errores de diseño o programación (7.4.4.10) • Verificación formal (tabla A.5-9) • Prueba formal (tabla A.9-1)	• Empaquetadores de funciones “críticas” de biblioteca, chequear pre-condiciones y post-condiciones. • Incentivos para la utilización de características del lenguaje que puedan expresar las restricciones en el uso de elementos de datos particulares o funciones (por ejemplo, const). • Para la verificación soportada por una herramienta: reglas para cumplir con las limitaciones de las herramientas seleccionadas (siempre que esto no afecte a más objetivos esenciales). • Utilización limitada de recursividad (tabla B.1-6) y de otras formas de dependencias circulares. (Véase también la aproximación modular).
Verificación estática de la conformidad con el diseño especificado (7.9.2.12)	Directrices de codificación para la implementación de los conceptos o limitaciones de diseño específicos. Por ejemplo: • Directrices de codificación del comportamiento cíclico, con tiempo máximo de ciclo garantizado (tabla A.2-13a). • Directrices de codificación para arquitectura activada por tiempo (tabla A.2-13b). • Directrices de codificación para arquitectura dirigida por evento, con tiempo máximo de ciclo garantizado (tabla A.2-13c). • Bucles con número máximo de iteraciones estáticamente determinado (excepto para el bucle infinito del diseño cíclico). • Directrices de codificación para la asignación de recursos estáticos (tabla A.2-14) y evitar objetos dinámicos (tabla B.1-2). • Directrices de codificación para sincronización estática de acceso a recursos compartidos (tabla A.2-15). • Directrices de codificación para cumplir con el uso limitado de interrupciones (tabla B.1-4). • Directrices de codificación para evitar variables dinámicas (tabla B.1-3a). • Chequeo en línea de la instalación de variables dinámicas (tabla B.1-3b). • Directrices de codificación para asegurar la compatibilidad con otros lenguajes de programación utilizados (7.4.4.10). Directrices para facilitar la trazabilidad con el diseño.

Requisitos y recomendaciones de la Norma IEC 61508-3	Sugerencias sobre reglas de codificación
Subgrupo de lenguaje (tabla A.3-3) - Prohibir características no seguras de lenguaje (7.4.4.13) - Utilizar sólo características de lenguaje definidas (7.4.4.10) - Programación estructurada (tabla A.4-6) - Lenguaje de programación fuertemente tipificado (tabla A.3-2) - Conversión de tipos no automática (tabla B.1-8)	Exclusión de características de lenguaje que conduzcan a diseños no estructurados. Por ejemplo, - Utilización limitada de punteros (tabla B.1-5). - Utilización limitada de recursividad (tabla B.1-6). - Utilización limitada de uniones del tipo C. - Utilización limitada de excepciones como las de Ada o C++. - Flujo de control no estructurado en programas en lenguajes de más alto nivel (tabla B.1-7). - Un punto de entrada/un punto de salida en subrutinas y funciones (tabla B.9-5). - Conversión de tipos no automática. - Utilización limitada de efectos colaterales no evidentes de firmas de funciones (por ejemplo, de variables estáticas). Evaluación de condiciones sin efectos colaterales y en todas las formas de aserciones. Utilización limitada o documentada de características específicas de compilador. Utilización limitada de construcciones de lenguaje que potencialmente puedan llevar a equívoco. Reglas a aplicar cuando, sin embargo, se utilicen estas características de lenguaje.
Buenas prácticas de programación (7.4.4.13)	Cuando sea aplicable: - Directrices de codificación para asegurar que, cuando sea necesario, las expresiones de punto flotante sean evaluadas en el orden correcto (por ejemplo, “a-b+c” no siempre es igual a “a+c-b”). - En comparaciones de punto flotante: utilizar sólo desigualdades (menor que, menor o igual que, mayor que, mayor o igual que) en vez de igualdades estrictas. - Directrices para la compilación condicional y el “pre-procesado”. - Chequeo sistemático de las condiciones de retorno (éxito/disfunción). Documentación, y, cuando sea posible, automatización de la producción de código ejecutable (makefiles). Evitar los efectos colaterales no evidentes de las firmas de funciones. Cuando tales efectos existan, directrices para documentarlos. Utilización de paréntesis cuando el orden de los operadores no sea obvio. Captura de supuestas situaciones imposibles (por ejemplo, un caso “por defecto” en la sentencia <i>switch</i> de C). Utilización de “empaquetadores” para módulos críticos, en particular para chequear las pre y post-condiciones y las condiciones de retorno. Directrices de codificación para tratar los errores conocidos del compilador y grupo de límites mediante evaluación del compilador.

C.2.6.3 Ni variables dinámicas ni objetos dinámicos

NOTA Esta técnica/medida se menciona en las tablas A.2 y B.1 de la Norma IEC 61508-3.

Objetivo: Excluir:

- Cualquier solape de memoria no deseado o no detectado.
- Cualquier cuello de botella de recursos durante el tiempo de ejecución (relacionado con la seguridad).

Descripción: En el caso de esta medida, las variables dinámicas y los objetos dinámicos son aquellas variables y objetos que tienen su memoria asignada y su dirección absoluta determinada en el momento de la ejecución. El valor de la asignación de memoria y de la dirección correspondiente depende del estado del sistema en el momento de la asignación, lo que significa la imposibilidad de un control por el compilador o cualquier otra herramienta fuera de línea.

Debido a que el número de variables y de objetos dinámicos, así como el espacio de memoria disponible para la asignación de nuevos objetos o variables dinámicas, depende del estado del sistema en el momento de la asignación, es posible que las disfunciones se produzcan en el momento de la asignación o de la utilización de variables o de objetos. Por ejemplo, si el espacio de memoria disponible en el lugar asignado por el sistema es insuficiente, se puede producir la sobreescritura accidental del contenido de otra variable. Estos incidentes se evitan cuando no se utiliza ningún objeto o variable dinámica.

Son necesarias las restricciones en la utilización de objetos dinámicos cuando el comportamiento dinámico no puede predecirse de manera precisa mediante algún análisis estático (es decir, antes de la ejecución del programa), y en consecuencia no pueda garantizarse la ejecución predecible del programa.

C.2.6.4 Control en línea durante la creación de variables dinámicas u objetos dinámicos

NOTA 1 Esta técnica/medida se menciona en la tabla B.1 de la Norma IEC 61508-3.

Objetivo: Controlar que la memoria que debe asignarse a las variables y a los objetos dinámicos está libre antes de la asignación, asegurando que la asignación de variables y objetos dinámicos durante la ejecución no tiene ninguna incidencia sobre las variables, los datos o el código existente.

Descripción: En el caso de esta medida, las variables dinámicas son aquellas que tienen su memoria asignada y su dirección absoluta determinada en el momento de la ejecución (en este sentido, las variables son también los atributos de las instancias de los objetos).

Mediante medios hardware o software, la memoria se controla para asegurar que está libre antes de la asignación de una variable o un objeto dinámico (por ejemplo, para evitar cualquier desbordamiento de pila). Si la asignación no está autorizada (como, por ejemplo, cuando la memoria en la dirección determinada es insuficiente), es necesario tomar las acciones apropiadas. Después de la utilización de una variable o de un objeto dinámico (como, por ejemplo, después de la salida de un subprograma), es necesario liberar toda la memoria asignada a esta variable.

NOTA 2 Una solución alternativa es demostrar estáticamente que la memoria es adecuada en todos los casos.

C.2.6.5 Utilización limitada de las interrupciones

NOTA Esta técnica/medida se menciona en la tabla B.1 de la Norma IEC 61508-3.

Objetivo: Asegurar que el software es verificable y comprobable.

Descripción: Debería limitarse la utilización de las interrupciones. Las interrupciones se pueden utilizar si simplifican el sistema. El procesamiento software de las interrupciones debería inhibirse durante las operaciones críticas (por ejemplo, las operaciones con duración crítica o los cambios críticos de los datos) de las funciones ejecutadas. Cuando se utilizan las interrupciones, las operaciones no interrumpibles deberían tener un tiempo de procesamiento máximo definido, de forma que se pueda calcular el tiempo máximo durante el cual una interrupción es inhibida. La utilización y el enmascaramiento de las interrupciones deberían documentarse exhaustivamente.

C.2.6.6 Utilización limitada de los punteros

NOTA Esta técnica/medida se menciona en la tabla B.1 de la Norma IEC 61508-3.

Objetivo: Evitar los problemas que resultan del acceso a los datos sin control previo del rango o del tipo del puntero. Permitir el ensayo y la verificación de los módulos del software. Limitar las consecuencias de las disfunciones.

Descripción: La aritmética de punteros en el software de la aplicación se puede utilizar al nivel del código fuente únicamente si el tipo de datos y el rango de valores del puntero (con el fin de asegurar que la referencia del puntero se sitúa en el espacio de dirección correcto) se controlan antes del acceso. La comunicación entre las diferentes tareas del software de la aplicación no se debería realizar por referencia directa a las tareas. Los intercambios de datos se deberían realizar por medio del sistema operativo.

C.2.6.7 Utilización limitada de la recursividad

NOTA Esta técnica/medida se menciona en la tabla B.1 de la Norma IEC 61508-3.

Objetivo: Evitar la utilización no verificable y que no se pueda probar de las llamadas de los subprogramas.

Descripción: En caso de utilización de la recursividad, es necesario que haya un criterio claro que permita predecir la profundidad de la recursividad.

C.2.7 Programación estructurada

NOTA Esta técnica/medida se menciona en la tabla A.4 de la Norma IEC 61508-3.

Objetivo: Diseñar e implementar el programa de forma que sea práctico de analizar sin ejecución. El programa puede contener únicamente un mínimo absoluto de comportamiento que no se pueda probar estadísticamente.

Descripción: Los principios siguientes se deberían aplicar para minimizar la complejidad estructural:

- dividir el programa en módulos de software de tamaño más pequeño y apropiado asegurando que estos módulos están tan desacoplados como es posible y que todas las interacciones sean explícitas;
- componer el flujo de control de los módulos del software con la ayuda de construcciones estructuradas: secuencias, iteraciones y selecciones;
- mantener el número posible de rutas a través de un módulo del software tan bajo como sea posible y que la relación entre los parámetros de entrada y de salida sea tan simple como sea posible;
- evitar las ramificaciones complicadas y, en particular, los saltos incondicionales ("goto") en los lenguajes de alto nivel;
- cuando sea posible, relacionar las condiciones de los bucles y de las ramificaciones con los parámetros de entrada;
- evitar las decisiones de ramificación y de bucles basadas en la utilización de cálculos complejos.

Se deberían utilizar las características del lenguaje de programación que fomentan esta aproximación con preferencia frente a otras características que son (supuestamente) más eficaces, excepto cuando la eficacia sea una prioridad absoluta (por ejemplo, en el caso de algunos sistemas críticos relacionados con la seguridad).

Referencias:

Concepts in Programming Languages. J. C. Mitchell. Cambridge University Press, 2003, ISBN 0521780985, 9780521780988.

A Discipline of Programming. E. W. Dijkstra. Englewood Cliffs NJ, Prentice-Hall, 1976.

C.2.8 Ocultación/encapsulado de la información

NOTA Esta técnica/medida se menciona en la tabla B.9 de la Norma IEC 61508-3.

Objetivo: Prevenir cualquier acceso involuntario a los datos o procedimientos y así mantener una buena estructura del programa.

Descripción: Los datos que son globalmente accesibles a todos los elementos del software se pueden modificar accidentalmente o de forma errónea por uno de estos elementos. Cualquier modificación de estas estructuras de datos puede necesitar el examen detallado del código y unas modificaciones importantes.

La ocultación de las informaciones es una aproximación general para minimizar estas dificultades. Las estructuras de datos esenciales son “ocultadas” y no pueden ser manipulables más que con la ayuda de un conjunto determinado de procedimientos de acceso. Esto permite modificar las estructuras internas y añadir procedimientos suplementarios sin afectar al comportamiento del resto del software. Por ejemplo, un directorio de nombres puede incluir los procedimientos de acceso “inserción”, “anulación” y “búsqueda”. Los procedimientos de acceso y las estructuras de datos internos se pueden reescribir (por ejemplo, para utilizar un método de consulta diferente o para almacenar los nombres en un disco duro) sin afectar al comportamiento lógico del resto del software.

Se debería utilizar el concepto de tipo de dato abstracto. Si no se proporciona una ayuda directa, puede ser necesario verificar que la abstracción no se ha roto inadvertidamente.

Referencias:

Concepts in Programming Languages. J. C. Mitchell. Cambridge University Press, 2003, ISBN 0521780985, 9780521780988.

On the Design and Development of Program Families. D. L. Parnas. IEEE Trans SE-2, March 1976.

C.2.9 Aproximación modular

NOTA Esta técnica/medida se menciona en las tablas A.4 y B.9 de la Norma IEC 61508-3.

Objetivo: Descomposición de un sistema de software en pequeñas partes comprensibles de forma que se limite la complejidad del sistema.

Descripción: Una aproximación modular o modularización consta de varias reglas relativas a las fases de diseño, de codificación y de mantenimiento de un proyecto de software. Estas reglas varían de acuerdo con el método de diseño utilizado. La mayor parte de los métodos incluyen las reglas siguientes:

- un módulo del software (o equivalentemente, un subprograma) debería tener que cumplir una sola tarea o función bien definida;
- las conexiones entre los módulos del software deberían limitarse y definirse estrictamente; la coherencia debe ser fuerte al nivel de cada módulo;
- deberían generarse colecciones de subprogramas que proporcionen varios niveles de módulos del software;
- el tamaño de los subprogramas debería limitarse a un valor específico como, por ejemplo, de dos a cuatro veces el tamaño de la pantalla;
- los subprogramas deberían tener una sola entrada y una sola salida;
- los módulos del software se deberían comunicar con otros módulos a través de sus interfaces (cuando se utilicen variables globales o comunes, deberían estar bien estructuradas, su acceso controlado y justificar su utilización en cada caso);
- todas las interfaces de los módulos del software deberían estar perfectamente documentadas;

- la interfaz de los módulos del software debería contener solo los parámetros necesarios para su función. Sin embargo, esta recomendación es complicada por la posibilidad de que un lenguaje de programación pueda permitir parámetros por defecto o que se utilice una aproximación orientada a objetos.

Referencias:

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

Concepts in Programming Languages. J. C. Mitchell. Cambridge University Press, 2003, ISBN 0521780985, 9780521780988.

C.2.10 Utilización de los elementos del software probados/verificados

NOTA Esta técnica/medida se menciona en las tablas A.2, C.2, A.4 y C.4 de la Norma IEC 61508-3.

Objetivo: Evitar la necesidad de que los diseños y elementos de software tengan que ser ampliamente revalidados o rediseñados para cada nueva aplicación. Aprovechar los diseños que no han sido verificados formalmente o rigurosamente pero que se benefician de una historia operativa considerable. Aprovechar un elemento de software preexistente que haya sido verificado para una aplicación diferente y para el que existe un cuerpo de evidencia de verificación.

Descripción: Esta medida verifica que los elementos de software están suficientemente libres de errores sistemáticos de diseño y/o de disfunciones operacionales.

Generalmente es impracticable construir sistemas complejos a partir de partes rudimentarias. Generalmente es necesario hacer uso de subconjuntos mayores (“elementos”, véase la Norma IEC 61508-4, apartados 3.4.5 y 3.2.8) que hayan sido previamente desarrollados para suministrar alguna función útil y que puedan ser reutilizados para implementar alguna parte del nuevo sistema.

Los PES bien diseñados y estructurados están hechos de un número de elementos de software que son claramente distintos y que interactúan unos con otros de maneras claramente especificadas. La construcción de una biblioteca de tales elementos de software generalmente aplicables que pueden reutilizarse en varias aplicaciones permite que muchos de los recursos necesarios para validar los diseños sean compartidos por más de una aplicación.

Sin embargo, para aplicaciones relacionadas con la seguridad es esencial tener suficiente confianza en que el nuevo sistema que incorpora estos elementos preexistentes tiene la integridad de seguridad exigida, y que la seguridad no está comprometida por algún comportamiento incorrecto del elemento preexistente.

Con el fin de ganar confianza en que el comportamiento de un elemento preexistente puede ser conocido de manera precisa, son posibles dos puntos de vista:

- analizar la historia detallada de operación del elemento para demostrar que el elemento ha sido “probado en uso”;
- evaluar un cuerpo de evidencia de verificación que haya sido compilado sobre el comportamiento del elemento, para determinar si el elemento cumple los requisitos de esta norma.

C.2.10.1 Probado en uso

Solamente en casos raros, el argumento de “probado en uso” (véase la Norma IEC 61508-4, 3.8.18) será suficiente razón para que un elemento de software de confianza logre la integridad de seguridad necesaria. Para los componentes complejos que incluyen un gran número de funciones posibles (como por ejemplo, un sistema operativo), es esencial determinar aquellas funciones que han sido realmente probadas en utilización. Por ejemplo, cuando un programa de ensayo automático se utiliza para detectar las disfunciones del hardware, si no se producen disfunciones durante el periodo de funcionamiento, no se puede considerar que el programa de ensayo para la detección de disfunciones se ha probado en uso.

Un elemento de software se puede considerar como probado en uso si cumple los siguientes criterios:

- misma especificación;
- sistemas utilizados en aplicaciones diferentes;
- al menos un año de historia en servicio;
- tiempo de funcionamiento acorde al nivel de integridad de seguridad o número adecuado de demandas; demostración de una tasa de disfunción no relacionada con la seguridad inferior a:
 - 10^{-2} por demanda (año) con un nivel de confianza del 95% necesita 300 ejecuciones operacionales (años);
 - 10^{-5} por demanda (año) con un nivel de confianza del 99,9% necesita 690 000 ejecuciones operacionales (años);

NOTA 1 Véase el anexo D para algunos aspectos matemáticos que apoyan las estimaciones numéricas anteriores. Véase también el apartado B.5.4 para una medida similar y la aproximación estadística.

- es necesario que cualquier experiencia en operación esté relacionada con un perfil de demanda conocido en lo relativo a las funciones del elemento del software, con el fin de asegurar que la experiencia en operación creciente conduce realmente a un aumento del conocimiento del comportamiento del elemento de software en relación al perfil de demanda;
- ninguna disfunción relacionada con la seguridad.

NOTA 2 Una disfunción puede ser crítica para la seguridad en un contexto y no serlo en otro, y viceversa.

Los puntos siguientes se deben documentar para verificar que un elemento del software cumple los criterios anteriores:

- identificación exacta de cada sistema y de sus elementos, incluyendo los números de versión (tanto para el software como para el hardware);
- identificación de los usuarios y tiempo de utilización;
- tiempo de ejecución;
- procedimiento de selección del sistema aplicado al usuario y del caso de aplicación;
- procedimientos de detección y registro de las disfunciones y de eliminación de fallos.

C.2.10.2 Evaluación de un cuerpo de evidencia de verificación

Un elemento de software preexistente (véase la Norma IEC 61508-4, 3.2.8) es un elemento que ya existe y que no ha sido desarrollado específicamente para el proyecto o sistema relacionado con la seguridad actual. El software preexistente podría ser un producto comercialmente disponible, o podría haber sido desarrollado por alguna organización para un producto o sistema previo. El software preexistente puede o no haber sido desarrollado de acuerdo a los requisitos de esta norma.

Con el fin de evaluar la integridad de seguridad del nuevo sistema que incorpora el software preexistente, se necesita un cuerpo de evidencia de verificación para determinar el comportamiento del elemento preexistente. Este puede derivarse (1) de la propia documentación del suministrador del elemento y de los registros del proceso de desarrollo del elemento, o (2) puede crearse o suplementarse mediante actividades de cualificación adicional realizadas por el desarrollador del nuevo sistema relacionado con la seguridad, o por terceras partes. Esto es el “manual de seguridad para elementos conformes” que define las capacidades y limitaciones del elemento de software potencialmente reutilizable.

En cualquier caso, debe existir (o debe crearse) un manual de seguridad para elementos conformes que sea adecuado para hacer posible una evaluación de la integridad de la función de seguridad específica que dependa total o parcialmente del elemento reutilizado. En otro caso, debe llegarse a la conclusión conservadora de que el elemento no se ha justificado para una reutilización relacionada con la seguridad. (Esto no quiere decir que el elemento no pueda justificarse en ningún caso, sino que simplemente se ha encontrado que la evidencia es insuficiente para el caso particular).

Esta norma tiene requisitos específicos para los contenidos del manual de seguridad para elementos conformes, véase la Norma IEC 61508-2, anexo D y la Norma IEC 61508-3, anexo D y los apartados 7.4.2.12 y 7.4.2.13 de la Norma IEC 61508-3.

Como breve indicación del contenido, el manual de seguridad expondrá lo siguiente:

- que el diseño del elemento es conocido y está documentado;
- que el elemento ha estado sujeto a verificación y validación utilizando una aproximación sistemática con ensayo documentado y revisión de todas las partes del diseño y código del elemento;
- que las funciones no utilizadas y no necesarias del elemento no evitarán que el nuevo sistema cumpla sus requisitos de seguridad;
- que todos los mecanismos de disfunción creíbles del elemento en el nuevo sistema han sido identificados y que se ha implementado la mitigación apropiada.

Una evaluación de la seguridad funcional del nuevo sistema debe establecer que el elemento reutilizado se aplica estrictamente dentro de los límites de capacidad que han sido justificados mediante la evidencia y las hipótesis del manual de seguridad para elementos conformes del elemento.

Referencias:

Component-Based Software Development: Case Studies. Kung-Kiu Lau. World Scientific, 2004, ISBN 9812388281, 9789812388285.

Software Reuse and Reverse Engineering in Practice. P. A. V. Hall (ed.), Chapman & Hall, 1992, ISBN 0-412-39980-6.

Software criticality analysis of COTS/SOUP. P. Bishop, T. Clement, S. Guerra. In Reliability Engineering & System Safety, Volume 81, Issue 3, September 2003, Elsevier Ltd., 2003.

C.2.11 Trazabilidad

Objetivo: Mantener la consistencia entre las fases del ciclo de vida.

Descripción: Con el fin de asegurar que el software que resulta de las actividades del ciclo de vida cumple los requisitos para el correcto funcionamiento del sistema relacionado con la seguridad, es esencial asegurar la consistencia entre las fases del ciclo de vida. Un concepto clave aquí es la “trazabilidad” entre actividades. Esto, esencialmente, es un análisis de impacto para chequear (1) que las decisiones tomadas en una fase temprana están adecuadamente implementadas en fases posteriores (trazabilidad hacia adelante) y (2) que las decisiones tomadas en una fase tardía son realmente exigidas y mandadas por las decisiones tempranas.

La trazabilidad hacia adelante se ocupa principalmente de revisar que un requisito está adecuadamente tratado en fases posteriores del ciclo de vida. La trazabilidad hacia adelante es valiosa en varios puntos del ciclo de vida de seguridad:

- desde los requisitos de seguridad del sistema hasta los requisitos de seguridad del software;
- desde la especificación de requisitos de seguridad del software hasta la arquitectura del software;

- desde la especificación de requisitos de seguridad del software hasta el diseño del software;
- desde la Especificación de Diseño del Software hasta las especificaciones de ensayo del módulo y de integración;
- desde los requisitos de diseño del sistema y del software para la integración hardware/software hasta las especificaciones del ensayo de integración hardware/software;
- desde la especificación de requisitos de seguridad del software hasta el plan de validación de la seguridad del software;
- desde la especificación de requisitos de seguridad del software hasta el plan de modificación del software (incluyendo la reverificación y la revalidación);
- desde la Especificación de Diseño del Software hasta el plan de verificación del software (incluyendo la verificación de datos);
- desde los requisitos del capítulo 8 de la Norma IEC 61508-3 hasta el plan para la evaluación de la seguridad funcional del software.

La trazabilidad hacia atrás se ocupa principalmente de revisar que cada decisión de implementación (interpretada en un contexto amplio y no confinada a la implementación del código) está claramente justificada por algún requisito. Si esta justificación falta, entonces es que la implementación contiene algo innecesario que añadirá complejidad pero no necesariamente tratará ningún requisito genuino del sistema relacionado con la seguridad. La trazabilidad hacia atrás es valiosa en varios puntos del ciclo de vida de la seguridad:

- desde los requisitos de seguridad hasta las necesidades de seguridad percibidas;
- desde la arquitectura del software hasta la especificación de requisitos de seguridad del software;
- desde el diseño detallado del software hasta la arquitectura del software;
- desde el código del software hasta el diseño detallado del software;
- desde el plan de validación de la seguridad del software hasta la especificación de requisitos de seguridad del software;
- desde el plan de modificación del software hasta la especificación de requisitos de seguridad del software;
- desde el plan de verificación del software (incluyendo la verificación de los datos) hasta la especificación de diseño del software.

Referencia:

Requirements Engineering. E. Hull, K. Jackson, J. Dick. Springer, 2005, ISBN 1852338792, 9781852338794.

C.2.12 Diseño de software sin estados (o diseño de estados limitados)

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Limitar la complejidad del comportamiento del software.

Descripción: Considérese un programa software que procesa una secuencia de transacciones: recibe una secuencia de entradas y produce una salida en respuesta a cada entrada. El programa puede también memorizar alguna o toda su historia en un "estado" y puede tener este estado en cuenta cuando calcula cómo responder a futuras entradas.

Cuando la salida del programa en respuesta a una entrada específica viene completamente determinada por dicha entrada, se dice que el programa no tiene memoria o que no tiene estado. Cada transacción entrada/salida es completa en el sentido de que ninguna transacción está influenciada por una transacción anterior y una entrada específica da como resultado la misma salida asociada.

En contraste, cuando el programa tiene en cuenta tanto su entrada específica como su estado memorizado al calcular su salida, es capaz de tener un comportamiento más complejo puesto que puede entregar diferentes salidas en respuesta a la misma entrada en diferentes ocasiones. La respuesta a una entrada específica depende del contexto (es decir, la entradas y las salidas previas) en el que se recibe la entrada. Otra consideración que es pertinente para algunas aplicaciones (típicamente en comunicaciones) es que el comportamiento del programa puede ser particularmente sensible a los cambios del estado almacenado, si se introduce inadvertida o maliciosamente.

El diseño sin estados (o de estados limitados) es una aproximación general que tiene como objetivo minimizar la complejidad potencial del comportamiento software para evitar o minimizar la utilización de la información de estados en el diseño del software.

Referencias:

Introduction to Automata Theory, Languages, and Computation (3rd Edition). J. Hopcroft, R. Motwani, J. Ullman, Addison-Wesley Longman Publishing Co, 2006, ISBN:0321462254.

Stateless connections. T. Aura, P. Nikander. In Proc International Conference on Information and Communications Security (ICICS'97), ed Yongfei Han. Springer, 1997, ISBN 354063696X, 9783540636960.

C.2.13 Análisis numérico fuera de línea

NOTA Esta técnica/medida se menciona en la tabla A.9 de la Norma IEC 61508-3.

Objetivo: Asegurar la precisión de los cálculos numéricos.

Descripción: La imprecisión numérica puede surgir en el cálculo de una función matemática como consecuencia de la utilización de representaciones finitas de funciones ideales y números. El error de truncado se introduce cuando una función se aproxima mediante un número finito de términos de una serie infinita tal como una serie de Fourier. El error de redondeo se introduce mediante la representación de precisión finita de números reales en un ordenador físico. Cuando se realiza incluso el cálculo más simple en punto flotante, la validez de los cálculos debe revisarse para asegurar que se consigue la precisión exigida por la aplicación.

Referencia:

Guide to Scientific Computing. P.R. Turner. CRC Press, 2001, ISBN 0849312426, 9780849312427.

C.2.14 Mapas de secuencia de mensaje

NOTA Esta técnica/medida se menciona en las tablas B.7 y C.17 de la Norma IEC 61508-3.

Objetivo: Ayudar en la captura de los requisitos del sistema en las fases tempranas del diseño del desarrollo software incluyendo los requisitos y el diseño de la arquitectura del software. En UML, el nombre “Diagrama de Secuencia del Sistema” se utiliza para esta notación.

Descripción: El “Mapa de Secuencia de Mensaje” es un mecanismo de diagrama para describir el comportamiento de un sistema en términos de la comunicación que tiene lugar entre los actores del sistema (un actor puede ser un ser humano, un sistema de ordenador o un elemento o un objeto de software, dependiendo de la fase del diseño). Para cada actor, se dibuja una “línea de vida” vertical sobre el diagrama y las flechas entre la líneas de vida se utilizan para representar mensajes. Las acciones en la recepción de los mensajes pueden mostrarse opcionalmente como cajas sobre los diagramas. Se construye una colección de escenarios (que describen tanto el comportamiento deseable como el no deseable) como una especificación del comportamiento exigido del sistema. Estos escenarios tienen varios usos. Pueden animarse para demostrar el comportamiento del sistema a los usuarios finales. Pueden transformarse en una implementación ejecutable del sistema. Pueden formar la base de los datos de ensayo.

UML contiene extensiones del concepto original de Mapa de Secuencia de Mensaje en forma de construcciones de selección e iteración que permiten a los escenarios ramificarse y cerrarse en bucles, suministrando una notación más compacta. También pueden definirse subdiagramas, los cuales pueden ser referenciados por varios diagramas de secuencia de mayor nivel. También pueden representarse eventos de temporizado y externos.

Referencias:

“*Message Sequence charts*”, D. Harel, P. Thiagarajan. In *UML for Real: Design of Embedded Real-Time Systems*. ed. L. Lavagno. Springer, 2003, ISBN 1402075014, 9781402075018.

ISO/IEC 19501:2005 *Tecnología de la información. Procesamiento abierto distribuido. Lenguaje unificado de modelado (UML) Versión 1.4.2.*

C.3 Diseño de la arquitectura

C.3.1 Detección y diagnóstico de fallos

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Detectar los fallos de un sistema susceptibles de ocasionar una disfunción para establecer las bases de las contramedidas a utilizar para minimizar las consecuencias de las disfunciones.

Descripción: La detección de fallos es la actividad que consiste en verificar la presencia de estados erróneos en un sistema (estados causados por un fallo del sistema o subsistema a controlar). El objetivo principal de la detección de fallos es neutralizar los efectos de los resultados erróneos. Un sistema que actúa conjuntamente con los elementos paralelos, y que renuncia al control cuando detecta que sus propios resultados son incorrectos, se llama “dispositivo autocontrolado”.

La detección de fallos se basa sobre los principios de redundancia (principalmente para la detección de los fallos del hardware - véase la Norma IEC 61508-2, anexo A) y de diversidad (fallos del software). Es necesario un sistema de voto para decidir sobre la corrección de los resultados. Métodos especiales aplicables son: la programación por aserciones, la programación N-versiones y la técnica de supervisión diversa; y para el hardware: la introducción de sensores adicionales, bucles de control, códigos detectores de error, etc.

La detección de fallos se puede obtener por los controles efectuados sobre los valores o los tiempos a diferentes niveles, en particular al nivel físico (temperatura, tensión, etc.), lógico (códigos detectores de error), funcional (aserción) o externo (controles de verosimilitud). Los resultados de estos controles se pueden memorizar y asociar a los datos correspondientes para facilitar la búsqueda de las disfunciones.

Los sistemas complejos están constituidos por subsistemas. La eficacia de la detección de fallos, del diagnóstico y de la compensación de los fallos depende de la complejidad de las interacciones entre los subsistemas, que tiene un efecto sobre la propagación de los fallos.

El diagnóstico de fallos debería utilizarse al nivel más pequeño de subsistema, ya que los pequeños subsistemas permiten un diagnóstico de fallos más detallado (detección de los estados erróneos).

Los sistemas de información integrados al nivel de la empresa pueden comunicar el estado de los sistemas relacionados con la seguridad de forma periódica a los otros sistemas de supervisión, incluyendo las informaciones de ensayo de diagnóstico. Cualquier detección de fallo se puede destacar y usarse para lanzar una acción correctiva antes de que la situación se vuelva peligrosa. Finalmente, en caso de incidente, la documentación de estos fallos puede facilitar la investigación posterior.

Referencia:

Dependability of Critical Computer Systems 1. F. J. Redmill, Elsevier Applied Science, 1988, ISBN 1-85166-203-0.

C.3.2 Códigos de detección y corrección de errores

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Detectar y corregir los errores en las informaciones sensibles.

Descripción: Para una información de n bits, se genera un bloque codificado de k bits con el fin de permitir la detección y la corrección de r errores. Dos ejemplos tipo son los códigos de Hamming y los códigos polinomiales.

En los sistemas relacionados con la seguridad, normalmente será necesario descartar los datos erróneos en vez de corregirlos, puesto que sólo una parte predeterminada de errores se puede corregir de forma satisfactoria.

Referencia:

Fundamentals of Error-correcting Codes, W. Huffman, V. Pless. Cambridge University Press, 2003, ISBN 0521782805, 9780521782807.

C.3.3 Programación por aserción de las disfunciones

NOTA Esta técnica/medida se menciona en la tabla A.17 de la Norma IEC 61508-2 y en las tablas A.2 y C.2 de la Norma IEC 61508-3.

Objetivo: Detectar los fallos residuales de diseño del software durante la ejecución de un programa para evitar las disfunciones críticas para la seguridad del sistema y continuar la ejecución con el fin de obtener un alto nivel de fiabilidad.

Descripción: La idea principal del método de programación por aserciones es controlar una precondition (la validez de las condiciones iniciales se verifica antes de la ejecución de una secuencia de instrucciones) y una postcondition (los resultados se verifican después de la ejecución de una secuencia de instrucciones). Si la precondition o postcondition no se cumple, el proceso informa del error.

Por ejemplo,

```
aserción <precondición>;  
  acción 1;  
  :  
  :  
  acción x;  
aserción <postcondición>
```

Referencias:

Exploiting Traces in Program Analysis. A. Groce, R. Joshi. Lecture Notes in Computer Science vol 3920, Springer Berlin / Heidelberg, 2006, ISBN 978-3-540-33056-1.

Software Development – A Rigorous Approach. C. B. Jones, Prentice-Hall, 1980.

C.3.4 Supervisor diversificado

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Proteger contra los fallos residuales de especificación y de implementación del software susceptibles de afectar a la seguridad.

Descripción: Pueden distinguirse dos aproximaciones (1) la función supervisora y la función supervisada en el mismo ordenador, con alguna garantía de independencia entre ellos; y (2) la función supervisora y la función supervisada en ordenadores separados.

Un supervisor diversificado es un supervisor externo que se implementa en un ordenador independiente según una especificación diferente. Este supervisor diversificado tiene por objetivo verificar que el ordenador principal ejecuta las operaciones de forma segura, pero no necesariamente correctas. El supervisor diversificado supervisa el ordenador principal de forma permanente. El supervisor diversificado impide cualquier estado inseguro del sistema. Además, en caso de detección de un estado potencialmente peligroso del ordenador principal, es necesario que el sistema vuelva al estado seguro o bien por el supervisor diversificado o bien por el ordenador principal.

El hardware y el software del supervisor diversificado se deberían clasificar y calificar según el SIL apropiado.

Referencia:

Requirements based Monitors for Real-Time Systems, D. Peters, D. Parnas. IEEE Transactions on Software Engineering, vol. 28, no. 2, 2002.

C.3.5 Diversidad del software (programación diversificada)

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Detectar y enmascarar los fallos residuales de diseño e implementación del software durante la ejecución de un programa para evitar las disfunciones del sistema críticas para la seguridad y continuar la ejecución con el fin de obtener un alto nivel de fiabilidad.

Descripción: En programación diversificada, una especificación de programa dada se diseña y se implementa N veces de diferentes formas. Los mismos valores de entrada se proporcionan a las N versiones y los resultados producidos por las N versiones se comparan. Si el resultado se considera válido, se transmite a las salidas del ordenador.

Un requisito esencial es que las N versiones sean independientes una de otra en algún sentido, de manera que no fallen simultáneamente debido a la misma causa. En la práctica puede ser muy difícil lograr y demostrar la independencia de las versiones que es la base de la aproximación de N -versiones.

Las N versiones se pueden ejecutar en paralelo en ordenadores separados o alternativamente se pueden ejecutar todas las versiones en el mismo ordenador y someter los resultados a un voto interno. Se pueden utilizar diferentes estrategias de voto para las N versiones según los requisitos de la aplicación, como se indica a continuación:

- si el sistema tiene un estado seguro, es posible pedir el acuerdo completo (todas las N versiones están de acuerdo); en caso contrario, se utiliza un valor de salida que provoque que el sistema alcance el estado seguro. Para los sistemas con disparo simple, el voto se puede orientar sistemáticamente en el sentido de la seguridad. En este caso, la respuesta segura sería disparar cuando el disparo sea demandado por una versión cualquiera. Esta aproximación normalmente solo necesita dos versiones ($N = 2$);
- para los sistemas sin estado seguro, se pueden utilizar las estrategias de voto mayoritario. En caso de no unanimidad, las aproximaciones probabilísticas se pueden utilizar para maximizar la posibilidad de seleccionar el valor correcto como, por ejemplo, tomando la media, paralizando temporalmente las salidas hasta la vuelta a la unanimidad, etc.

Esta técnica no elimina los fallos residuales de diseño del software, ni los errores en la interpretación de la especificación, pero realiza una medida de detección y de enmascaramiento antes de que la seguridad sea vea afectada.

Referencias:

Modelling software design diversity – a review, B. Littlewood, P. Popov, L. Strigini. ACM Computing Surveys, vol 33, no 2, 2001.

The N-Version Approach to Fault-Tolerant Software, A. Avizienis, IEEE Transactions on Software Engineering, vol. SE-11, no. 12 pp.1491-1501, 1985.

An experimental evaluation of the assumption of independence in multi-version programming, J.C. Knight, N.G. Leveson. IEEE Transactions on Software Engineering, vol. SE-12, no 1, 1986.

In Search of Effective Diversity: a Six Language Study of Fault-Tolerant Flight Control Software. A. Avizienis, M. R. Lyu and W. Schutz. 18th Symposium on Fault-Tolerant Computing, Tokyo, Japan, 27-30 June 1988, IEEE Computer Society Press, 1988, ISBN 0-8186-0867-6.

C.3.6 Recuperación hacia atrás

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Permite un funcionamiento correcto en presencia de una o varios fallos.

Descripción: Cuando se detecta un fallo, el sistema se lleva a un estado interno precedente en el que la coherencia se ha demostrado. Este método necesita guardar frecuentemente el estado interno en los puntos de control bien definidos. Esto se puede hacer globalmente (para la base de datos completa) o por incrementos (cambiando solo entre los puntos de control). El sistema debe compensar los cambios que se han producido durante ese tiempo utilizando el diario de acciones (rastreo de las acciones), la compensación (cualquier efecto del cambio se anula) o la interacción externa (manual).

Referencias:

Looking into Compensable Transactions. Jing Li, Huibiao Zhu, Geguang Pu, Jifeng He. In Software Engineering Workshop, 2007. SEW 2007. IEEE, 2007, ISBN 978-0-7695-2862-5.

Software Fault Tolerance (Trends in Software, No. 3), M. R. Lyu (ed.), John Wiley & Sons, April 1995, ISBN 0471950688.

C.3.7 Mecanismos de recuperación de fallos por relanzamiento

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Intentar la recuperación funcional a partir de una condición de fallo detectada, con la ayuda de mecanismos de relanzamiento.

Descripción: A continuación de una detección de fallo o de error, se efectúan intentos para restablecer la situación ejecutando de nuevo el mismo código. La recuperación por relanzamiento puede ser tan completa como un reinicio y un procedimiento de re arranque o una pequeña replanificación y re arranque de alguna tarea, tras el fin de un temporizador software o una acción de supervisión de una tarea. Las técnicas de relanzamiento se utilizan comúnmente para la recuperación de los fallos o de los errores de comunicación y las condiciones de relanzamiento se pueden señalar a partir de un error del protocolo de comunicación (checksum, etc.) o a partir de un exceso de tiempo de respuesta acusado en la recepción de comunicación.

Referencia:

Reliable Computer Systems: Design and Evaluation, D.P. Siewiorek, R.S. Schwartz. A.K. Peters Ltd., 1998, ISBN 156881092X, 9781568810928.

C.3.8 Degradación “escalonada”

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Mantener disponibles las funciones más críticas del sistema, a pesar de las disfunciones, por abandono de las funciones menos críticas.

Descripción: Esta técnica da prioridades a las diferentes funciones que se deben ejecutar por el sistema. El diseño garantiza que si no hay recursos suficientes para ejecutar todas las funciones del sistema, se ejecutan las funciones de prioridad más alta. Por ejemplo, las funciones de consignación de los errores y de los eventos pueden ser menos prioritarias que las funciones de control del sistema, lo que permite continuar controlando el sistema en caso de disfunción del hardware encargado de la consignación de errores. Además, en caso de disfunción del hardware encargado del control del sistema, sin disfunción del encargado de la consignación de errores, este último hardware asume la función de control.

Esta técnica se aplica principalmente al hardware pero también al sistema entero incluyendo el software. Es necesario tener en cuenta esta técnica al principio de la fase de diseño.

Referencias:

Towards the Integration of Fault, Resource, and Power Management, T. Siridakis. In Computer Safety, Reliability, and Security: 23rd International Conference, SAFECOMP 2004. Eds. Maritta Heisel et. al. Springer, 2004, ISBN 3540231765, 9783540231769.

Achieving Critical System Survivability Through Software Architectures, J.C. Knight, E.A. Strunk. Springer Berlin / Heidelberg, 2004, 978-ISBN 3-540-23168-4.

The Evolution of Fault-Tolerant Computing. Vol. 1 of Dependable Computing and Fault-Tolerant Systems, Edited by A. Avizienis, H. Kopetz and J. C. Laprie, Springer Verlag, 1987, ISBN 3-211-81941-X.

Fault Tolerance, Principle and Practices. T. Anderson and P. A. Lee, Vol. 3 of Dependable Computing and Fault-Tolerant Systems, Springer Verlag, 1987, ISBN 3-211-82077-9.

C.3.9 Corrección de fallos utilizando las técnicas de inteligencia artificial

NOTA 1 Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Ser capaz de reaccionar a los peligros eventuales de forma muy flexible utilizando una combinación de métodos y de modelos de procesos y algún tipo análisis de seguridad y de fiabilidad en línea.

Descripción: La previsión de los fallos (tendencias obtenidas por cálculo), las corrección de los fallos y las acciones de mantenimiento y de supervisión se pueden realizar de forma muy eficaz en diversos canales del sistema utilizando los sistemas basados en la inteligencia artificial (IA) ya que las reglas se pueden deducir directamente de las especificaciones y verificar por comparación con estas especificaciones. Algunos fallos comunes que se introducen en las especificaciones, pensando que ya están implícitas en ciertas reglas de diseño e implementación, se pueden evitar de forma eficaz gracias a esta aproximación, en particular cuando se aplica una combinación de modelos y de métodos de una forma funcional o descriptiva.

Los métodos se seleccionan de tal forma que los fallos se puedan corregir y se puedan minimizar los efectos de las disfunciones para obtener la integridad de seguridad deseada.

NOTA 2 Véase en el apartado C.3.2 el aviso sobre la corrección de datos erróneos, y la Norma IEC 61508-3, tabla A.2, punto 5, para las recomendaciones negativas relativas a esta técnica.

Referencia:

Fault Diagnosis: Models, Artificial Intelligence, Applications. J. Korbicz, J. Koscielny, Z. Kowalczyk, W. Cholewa. Springer, 2004, ISBN 3540407677, 9783540407676.

C.3.10 Reconfiguración dinámica

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Conservar la funcionalidad del sistema a pesar de un fallo interno.

Descripción: La arquitectura lógica del sistema debe ser tal que pueda asignarse a un conjunto de recursos disponibles del sistema. Esta arquitectura debe ser capaz de detectar la disfunción de un recurso físico y de asignar de nuevo la arquitectura lógica a los recursos limitados todavía en funcionamiento. Aunque tradicionalmente el concepto esté más limitado a la recuperación de unidades del hardware averiadas, se aplica también a las unidades del software que fallan cuando la “redundancia de ejecución” es suficiente para permitir un relanzamiento del software, o cuando existen suficientes datos redundantes como para que una disfunción individual y aislada se considere como poco importante.

Es necesario tener en cuenta esta técnica al principio de la fase de diseño del sistema.

Referencia:

Dynamic Reconfiguration of Software Architectures Through Aspects. C. Costa et al. Lecture Notes in Computer Science, Volume 4758/2007, Springer Berlin / Heidelberg, 2007, ISBN 978-3-540-75131-1.

C.3.11 Seguridad y funcionamiento en tiempo real: Arquitectura Activada por Tiempo

NOTA 1 Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Composición e implementación transparente de la tolerancia al fallo en sistemas críticos de seguridad en tiempo real con comportamiento predecible.

Descripción: En un sistema de Arquitectura Activada por Tiempo (TTA, Time-Triggered Architecture), todas las actividades del sistema se inician y basan en la progresión de una base de tiempos global sincronizada. Cada aplicación se asigna a una franja de tiempo fija sobre un bus activado por tiempo, que contiene los mensajes intercambiados entre las tareas de cada aplicación que pueden, en consecuencia, intercambiarse sólo de acuerdo a un calendario definido. En los sistemas dirigidos por eventos, las actividades del sistema son activadas por eventos arbitrarios en puntos impredecibles del tiempo. Las ventajas clave de la TTA son (véase la referencia Scheidler, Heiner et al.):

- capacidad de composición, que disminuye en gran medida el esfuerzo exigido de ensayo y certificación del sistema;
- implementación transparente de la tolerancia al fallo, que hace la arquitectura altamente recomendable para aplicaciones críticas de seguridad;
- provisión de una base de tiempos globalmente sincronizada, que facilita el diseño de los sistemas distribuidos en tiempo real.

La comunicación entre nodos se realiza utilizando el *Protocolo Activado por Tiempo TTP/C* (véase la referencia Kopetz, Hexel et al.) de acuerdo a un calendario estático, que decide cuando transmitir un mensaje y si el mensaje recibido es pertinente o no para el módulo electrónico particular. El acceso al bus se controla mediante un esquema de *acceso múltiple por división en tiempo (TDMA)* derivado de la noción global de tiempo.

El protocolo TTP/C garantiza (véase la referencia Rushby) cuatro servicios básicos (servicios de núcleo) en una red de nodos de TTA (véase la referencia Kopetz, Bauer):

- Transporte determinista y puntual de mensajes: Transporte de mensajes desde el puerto de salida del elemento que envía a los puertos de entrada de los elementos que reciben dentro de un límite de tiempo conocido con anticipación. Se ofrece un servicio de transporte tolerante al fallo mediante un servicio de comunicación activado por tiempo que está disponible a través de la interfaz temporal del firewall que elimina la propagación del error de control por diseño y minimiza el acoplamiento entre los elementos. El transporte puntual de mensajes con latencia y jitter mínimos es crucial para lograr la estabilidad del control en aplicaciones en tiempo real.
- Sincronización de Reloj Tolerante al fallo: El controlador de comunicación genera una base de tiempos globalmente sincronizada tolerante al fallo (con una precisión de unos pocos pulsos de reloj) que se suministra al subsistema anfitrión.

- Diagnósis Consistente de Nodos en Disfunción (Servicio de Miembro): El controlador de comunicación informa a cada SRU (“unidad reemplazable más pequeña”) sobre el estado de cada una de las otras SRU de un grupo con una latencia menor que una ronda de TDMA.
- Fuerte Aislamiento de Fallo: Un subsistema anfitrión en fallo peligroso (incluyendo su software) puede producir salidas de datos erróneos, pero no puede nunca interferir de otra manera en la correcta operación del resto de un grupo de TTP/C. Se garantiza la disfunción por silencio en el dominio temporal mediante el comportamiento activado por tiempo del controlador de comunicación.

NOTA 2 Otros protocolos activados por tiempo son FlexRay y TT-Ethernet (Ethernet activada por tiempo).

Referencias:

Time-Triggered Architecture (TTA). C. Scheidler, G. Heiner, R. Sasse, E. Fuchs, H. Kopetz, C. Temple. In *Advances in Information Technologies: The Business Challenge*, ed. JY. Roger. IOS Press, 1998, ISBN 9051993854, 9789051993851.

A Synchronisation Strategy for a TTP/C Controller. H. Kopetz, R. Hexel, A. Krueger, D. Millinger, A. Schedl. SAE paper 960120, Application of Multiplexing Technology SP 1137, Detroit, SAE Press, Warrendale, 1996.

The Time-Triggered Architecture. H. Kopetz, G. Bauer. Proceedings of the IEEE Special Issue on Modeling and Design of Embedded Software, October 2002.

An Overview of Formal Verification for the Time-Triggered Architecture. J. Rushby: Invited paper, Oldenburg, Germany, September 9-12, 2002. Proceedings FTRTFT 2002, Springer LNCS 2469, 2002, ISBN 978-3-540-44165-6.

C.3.12 UML

NOTA Esta técnica/medida se menciona en la tabla B.7 de la Norma IEC 61508-3.

Objetivo: Proporcionar un conjunto completo de notaciones para modelar el comportamiento deseado de los sistemas complejos.

Descripción: El UML es, como su nombre indica, una colección de requisitos y notaciones de diseño prevista para suministrar un soporte completo para el desarrollo de software. Algunas partes del UML se basan en notaciones introducidas primero por otros métodos (tales como los diagramas de secuencia de sistema y los diagramas de transición de estados) y otras notaciones son únicas para UML. El UML está fuertemente orientado hacia los conceptos orientados a objetos aunque algunas notaciones pueden utilizarse sin ninguna necesidad de proceder a una programación orientada a objetos. El UML está soportado por numerosas herramientas CASE comercialmente disponibles, muchas de las cuales son capaces de generar código automáticamente a partir de modelos UML.

Las notaciones UML que son más aplicables generalmente a la especificación y el diseño de sistemas relacionados con la seguridad son las siguientes:

- diagramas de clase;
- casos de utilización;
- diagramas de actividad;
- diagramas de transición de estados (mapas de estado);
- diagramas de secuencia de sistema.

Otras notaciones UML son pertinentes para la expresión del diseño de la arquitectura del software (estructura del software) pero no se han enumerado específicamente aquí.

Los diagramas de transición de estados se describen en el apartado B.2.3.2 y los diagramas de secuencia de sistema en el apartado C.2.14. Las otras notaciones se describen en los tres siguientes apartados.

C.3.12.1 Diagramas de clase

Los diagramas de clase definen las clases de objeto con las que tiene que tratar el software. Se basan en los diagramas tempranos de entidad-relación-atributo pero están adaptados para el diseño orientado a objetos. Cada clase (de las que habrá una o más instancias conocidas como objetos en el tiempo de ejecución) se representa como un rectángulo y las variadas relaciones entre las clases se muestran como líneas o flechas. Las operaciones o métodos ofrecidos por cada clase y los atributos de datos de cada clase, pueden añadirse al diagrama. Las relaciones que pueden representarse consisten tanto en relaciones de referencia con su cardinalidad (una instancia de clase A puede referirse a una o más instancias de clase B) como en relaciones de especialización (la clase X es un refinamiento de la clase Y) con posiblemente métodos y atributos adicionales. La herencia múltiple puede dibujarse.

C.3.12.2 Casos de utilización

Los casos de utilización suministran una descripción textual del comportamiento deseado del sistema en respuesta a un escenario particular, normalmente desde el punto de vista de los actores externos, incluyendo a los usuarios humanos del sistema y sistemas externos. Los sub-escenarios alternativos de un caso de utilización dado pueden utilizarse para representar un comportamiento opcional, especialmente en casos de respuesta de error. Se desarrolla una colección de casos de utilización para suministrar una especificación completa suficientemente de los requisitos del sistema. Las sentencias de utilización pueden ser el punto de partida para el desarrollo de modelos más rigurosos tales como los diagramas de secuencia del sistema y diagramas de actividad.

Los diagramas de casos de utilización suministran una representación pictórica del sistema y de los actores que están involucrados en los casos de utilización, pero no son rigurosos y para la especificación sólo es importante el texto del caso de utilización.

C.3.12.3 Diagramas de actividad

Un diagrama de actividad muestra la secuencia prevista de acciones a realizar por el elemento de software (frecuentemente un objeto en un diseño orientado a objetos) incluyendo el comportamiento secuencial e iterativo (algunos aspectos se parecen mucho a una mapa de flujo). Los diagramas de actividad, sin embargo, permiten describir en paralelo las acciones de numerosos elementos, con las interacciones entre los elementos indicadas mediante flechas en el diagrama. Los puntos de sincronización donde una actividad tiene que esperar a una o más entradas desde las otras actividades antes de poder proceder, se muestran mediante un símbolo similar a un nodo de la red de Petri.

Referencia:

ISO/IEC 19501:2005 *Tecnología de la información. Procesamiento abierto distribuido. Lenguaje unificado de modelado (UML) Versión 1.4.2.*

C.4 Herramientas de desarrollo y lenguajes de programación

C.4.1 Lenguajes de programación fuertemente tipificados

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-3.

Objetivo: Reducir la probabilidad de fallos utilizando un lenguaje que permite un control de alto nivel por el compilador.

Descripción: Cuando un lenguaje de programación fuertemente tipificado se compila, varios controles se efectúan en lo relativo al modo de utilización de los tipos de variables como, por ejemplo, en el procedimiento de llamada y de acceso a los datos externos. La compilación fallará y se emitirá un mensaje de error por cada utilización no conforme a las reglas predefinidas.

Estos lenguajes suelen permitir los tipos de datos definidos por el usuario a partir de los tipos de datos básicos del lenguaje (como los números enteros o reales). Estos tipos se pueden utilizar después exactamente de la misma forma que los tipos básicos. Se imponen controles estrictos para asegurar que se utiliza el tipo correcto. Estos controles se imponen al programa entero, también si este se ha realizado a partir de unidades compiladas por separado. Los controles también permiten verificar que coinciden el número y el tipo de argumentos de los procedimientos, incluso cuando se referencian a partir de módulos del software compilados por separado.

Los lenguajes fuertemente tipificados fomentan en general otros aspectos de buenas prácticas de ingeniería del software tales como las estructuras de control fácilmente analizables (por ejemplo, si .. entonces .. si no .. hacer .. mientras, etc.) que conducen a programas bien estructurados.

Algunos ejemplos de tipos de lenguajes fuertemente tipificados son los lenguajes Pascal, Ada y Modula 2.

Referencia:

Concepts in Programming Languages. J. C. Mitchell. Cambridge University Press, 2003, ISBN 0521780985, 9780521780988.

C.4.2 Subconjuntos de lenguajes

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-3.

Objetivo: Reducir la probabilidad de introducir fallos de programación y aumentar la probabilidad de detección de los fallos restantes.

Descripción: El lenguaje se examina con el fin de determinar las estructuras de programación que o bien son propensas a los errores o bien son difíciles de analizar, por ejemplo utilizando métodos de análisis estático. Se define después un subconjunto del lenguaje para excluir estas estructuras.

Referencias:

Practical Experiences of Safety- and Security-Critical Technologies, P. Amey, A.J. Hilton. Ada User Journal, June, 2004.

Safer C: Developing Software for High-integrity and Safety-critical Systems. L. Hatton, McGraw-Hill, 1994, ISBN 0077076400, 9780077076405.

Requirements for programming languages in safety and security software standard. B. A. Wichmann. Computer Standards and Interfaces. Vol. 14, pp 433-441, 1992.

C.4.3 Herramientas certificadas y traductores certificados

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-3.

Objetivo: Las herramientas son necesarias para ayudar a los desarrolladores durante las diferentes fases de desarrollo del software. En la medida de lo posible, las herramientas deberían estar certificadas de forma que aseguren un cierto nivel de confianza en lo relativo a la corrección de las salidas.

Descripción: La certificación de una herramienta generalmente se realizará por un organismo independiente, a menudo nacional, con la ayuda de criterios determinados independientemente, típicamente en las normas nacionales e internacionales. Lo ideal sería que las herramientas utilizadas durante las fases de desarrollo (especificación, diseño, codificación, ensayo y validación) y las utilizadas para la gestión de configuración, se sometieran a la certificación.

Actualmente, solo los compiladores (traductores) se someten regularmente a los procedimientos de certificación; estos procedimientos se establecen por organismos de certificación nacionales y permiten ensayar los compiladores (traductores) en relación a las normas internacionales como las relativas a los lenguajes Ada y Pascal.

Es importante resaltar que las herramientas certificadas y los traductores certificados se certifican en general únicamente en relación a sus propias normas de lenguaje o de procesos. En general, no se certifican de ninguna forma en lo relativo a la seguridad.

Referencias:

The certification of software tools with respect to software standards, P. Bunyakiati et al. In IEEE International Conference on Information Reuse and Integration, IRI 2007, IEEE, 2007, ISBN 1-4244-1500-4.

Certified Testing of C Compilers for Embedded Systems. O. Morgan. In: 3rd Institution of Engineering and Technology Conference on Automotive Electronics. IEEE, 2007, ISBN 978-0-86341-815-0.

The Ada Conformity Assessment Test Suite (ACATS), version 2.5, Ada Conformity Assessment Authority, <http://www.ic.org/compilerstesting.html>, Apr. 2002.

C.4.4 Herramientas y traductores: aumento de confianza como resultado de su utilización

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-3.

Objetivo: Evitar las dificultades debidas a las disfunciones del traductor susceptibles de producirse durante el desarrollo, la verificación y el mantenimiento de un paquete de software.

Descripción: Utilización de un traductor para el que no existe evidencia de funcionamiento incorrecto durante muchos proyectos anteriores. Se debería evitar la utilización de traductores sin experiencia de operación o con fallos graves conocidos a menos que haya alguna garantía que asegure un funcionamiento correcto (por ejemplo, véase C.4.4.1).

Si el traductor mostrara pequeñas deficiencias, las estructuras de lenguaje correspondientes se anotarán con el fin de que se eviten durante un proyecto relacionado con la seguridad.

Otro procedimiento consiste en limitar la utilización del lenguaje a los elementos comúnmente utilizados.

Esta recomendación se basa en la experiencia adquirida durante varios proyectos. Se ha demostrado que los traductores inmaduros constituyen una seria dificultad para el desarrollo del software. Esto hace que no sea factible el desarrollo de software relacionado con la seguridad.

También es necesario saber que actualmente no existe ningún método que permita demostrar la corrección de todas las partes de una herramienta o un traductor.

C.4.4.1 Comparación del programa fuente y del código ejecutable

Objetivo: Verificar que las herramientas utilizadas para producir una imagen PROM no han introducido ningún error en la imagen PROM.

Descripción: La imagen PROM se somete a un proceso de ingeniería inversa para obtener los módulos “objeto”. Estos módulos “objeto” se someten a un proceso de ingeniería inversa para obtener los ficheros del lenguaje. Utilizando las técnicas apropiadas los ficheros del lenguaje así regenerados se comparan con el código fuente real utilizado originalmente para definir la PROM.

La mayor ventaja de esta técnica es que las herramientas (compilador, ensamblador, etc.) utilizadas para producir la imagen PROM no necesitan ser validadas para todos los programas. Esta técnica permite verificar que el fichero fuente utilizado para el sistema relacionado con la seguridad considerado está correctamente transformado.

Referencias:

Demonstrating Equivalence of Source Code and PROM Contents. D. J. Pavey and L. A. Winsborrow. The Computer Journal Vol. 36, No. 7, 1993.

Formal demonstration of equivalence of source code and PROM contents: an industrial example. D. J. Pavey and L. A. Winsborrow. Mathematics of Dependable Systems, Ed. C. Mitchell and V. Stavridou, Clarendon Press, 1995, ISBN 0-198534-91-4.

Assuring Correctness in a Safety Critical Software Application. L. A. Winsborrow and D. J. Pavey. High Integrity Systems, Vol. 1, No. 5, pp 453-459, 1996.

C.4.5 Lenguajes de programación adecuados

NOTA Esta técnica/medida se menciona en la tabla A.3 de la Norma IEC 61508-3.

Objetivo: Tener en cuenta los requisitos de esta norma internacional tanto como sea posible, en particular en lo relativo a la programación defensiva, fuerte tipificación, programación estructurada y eventualmente las aserciones. El lenguaje de programación elegido debería conducir a un código fácilmente verificable con un mínimo de esfuerzo y facilitar el desarrollo, la verificación y el mantenimiento del programa.

Descripción: El lenguaje debería estar definido completamente y sin ambigüedad. El lenguaje debería estar orientado hacia el usuario o hacia el problema mejor que hacia el procesador/plataforma de la máquina. Los lenguajes comúnmente utilizados y sus subconjuntos se prefieren frente a los lenguajes especializados.

Además de las características ya mencionadas, el lenguaje debería permitir:

- una estructura por bloques;
- el control del tiempo de traducción; y
- el control del tipo y dimensiones de las matrices en tiempo de ejecución.

El lenguaje debería favorecer:

- la utilización de módulos del software pequeños y manejables;
- la restricción de acceso a los datos en módulos del software específicos;
- la definición de rangos de variables; y
- cualquier otro tipo de construcción para limitar los errores.

Si el funcionamiento seguro del sistema depende de limitaciones de tiempo real, el lenguaje debería permitir también el procesamiento de las excepciones /interrupciones.

Es deseable que el lenguaje se soporte por un traductor apropiado, unas bibliotecas apropiadas de módulos del software preexistentes, un programa de depuración y unas herramientas para el control y el desarrollo de las versiones.

Actualmente, en el momento del desarrollo de esta norma, no se sabe todavía si es preferible utilizar los lenguajes orientados a objetos u otros lenguajes convencionales.

Las siguientes características hacen difícil la verificación y deberían evitarse:

- los saltos incondicionales, a excepción de las llamadas a subrutinas;
- la recursividad;
- los punteros, las pilas o cualquier tipo de objetos o de variables dinámicas;
- la gestión de las interrupciones al nivel del código fuente;
- las entradas o salidas múltiples de bucles, bloques o subprogramas;

- la declaración o inicialización implícita de las variables;
- los registros de variantes y las equivalencias; y
- los parámetros de procedimiento.

Los lenguajes de bajo nivel, en particular los lenguajes ensambladores, presentan problemas relacionados con el hecho de que están, por naturaleza, orientados hacia el procesador/plataforma de la máquina.

Una de las propiedades deseadas de un lenguaje es que su diseño y su utilización den unos programas cuya ejecución sea previsible. Para un lenguaje de programación definido de forma apropiada, existe un subconjunto de este lenguaje que asegura que la ejecución de un programa es previsible. Este subconjunto no se puede, en general, determinar estáticamente, aunque numerosas limitaciones estáticas pueden ayudar a asegurar una ejecución previsible. Esto necesitaría, típicamente, la demostración de que los índices de las matrices están dentro de los límites y que no se pueden producir desbordamientos numéricos, etc.

La tabla C.1 proporciona las recomendaciones aplicables a los lenguajes de programación específicos.

Referencias:

Concepts in Programming Languages. J. C. Mitchell. Cambridge University Press, 2003, ISBN 0521780985, 9780521780988.

IEC 60880:2006 *Centrales nucleares. Instrumentación y sistemas de control importantes para la seguridad. Aspectos lógicos (software) de los sistemas informáticos que realizan funciones de categoría A*.

IEC 61131-3:2003 *Autómatas programables. Parte 3: Lenguajes de programación*.

ISO/IEC 1539-1:2004 *Tecnología de la información. Lenguajes de programación. Fortran. Parte 1: Lenguaje básico*.

ISO/IEC 7185:1990 *Tecnología de la información. Lenguajes de programación. Pascal*.

ISO/IEC 8652:1995 *Tecnología de la información. Lenguajes de programación. Ada*.

ISO/IEC 9899:1999 *Lenguajes de programación. C*.

ISO/IEC 10206:1991 *Tecnología de la información. Lenguajes de programación. Pascal ampliado*.

ISO/IEC 10514-1:1996 *Tecnología de la información. Lenguajes de programación. Parte 1: Modulo 2, lenguaje básico*.

ISO/IEC 10514-3:1998 *Tecnología de la información. Lenguajes de programación. Parte 3: Modulo 2 orientado a objetos*.

ISO/IEC 14882:2003 *Lenguajes de programación. C++*.

ISO/IEC/TR 15942:2000 *Tecnología de la información. Lenguajes de programación. Guía para la utilización del lenguaje de programación Ada en sistemas de alta integridad*.

Tabla C.1 – Recomendaciones aplicables a los lenguajes de programación específicos

	Lenguaje de programación	SIL1	SIL2	SIL3	SIL4
1	ADA	HR	HR	R	R
2	Subconjunto de ADA	HR	HR	HR	HR
3	Java	NR	NR	NR	NR

	Lenguaje de programación	SIL1	SIL2	SIL3	SIL4
4	Subconjunto de Java (que no incluya ninguna recolección de basura ni ninguna recolección de basura que provoque que el código de aplicación se detenga durante un periodo significativo de tiempo). Véase el anexo G para guía sobre la utilización de las funciones orientadas a objetos	R	R	NR	NR
5	PASCAL (véase la NOTA 1)	HR	HR	R	R
6	Subconjunto de PASCAL	HR	HR	HR	HR
7	FORTRAN 77	R	R	R	R
8	Subconjunto de FORTRAN 77	HR	HR	HR	HR
9	C	R	–	NR	NR
10	Subconjunto de C y reglas de codificación y utilización de herramientas de análisis estático	HR	HR	HR	HR
11	C++ (véase el anexo G para guía sobre la utilización de las funciones orientadas a objetos)	R	–	NR	NR
12	Subconjunto de C++ y reglas de codificación y utilización de herramientas de análisis estático (véase el anexo G para guía sobre la utilización de las funciones orientadas a objetos).	HR	HR	HR	HR
13	Ensamblador	R	R	–	–
14	Subconjunto de ensamblador con reglas de codificación	R	R	R	R
15	Diagramas en escalera	R	R	R	R
16	Diagramas en escalera con subconjunto definido de lenguaje	HR	HR	HR	HR
17	Diagramas de bloques funcionales	R	R	R	R
18	Diagramas de bloques funcionales con subconjunto definido de lenguaje	HR	HR	HR	HR
19	Texto estructurado	R	R	R	R
20	Texto estructurado con subconjunto definido de lenguaje	HR	HR	HR	HR
21	Diagrama funcional secuencial	R	R	R	R
22	Diagrama funcional secuencial con subconjunto definido de lenguaje	HR	HR	HR	HR
23	Lista de instrucciones	R	–	NR	NR
24	Lista de instrucciones con subconjunto definido de lenguaje	HR	R	R	R
NOTA 1	Las recomendaciones R, HR, - y NR se explican en el anexo A de la Norma 61508-3.				
NOTA 2	El software del sistema consta del sistema operativo, drivers y los módulos del software que forman parte del sistema. El software normalmente es proporcionado por el vendedor del sistema relacionado con la seguridad. El subconjunto de lenguaje se debería elegir cuidadosamente con el fin de evitar estructuras complejas que puedan conducir a fallos de implementación. Se deberían realizar controles para verificar la correcta utilización del subconjunto de lenguaje.				
NOTA 3	El software de aplicación es el software que ha sido desarrollado para una aplicación específica relacionada con la seguridad. En muchos casos, este software se desarrolla por el usuario final o por el suministrador de la aplicación. Cuando se aplican las mismas recomendaciones a varios lenguajes de programación, el desarrollador debería seleccionar el que más se utiliza en la industria o en la unidad de producción correspondiente. El subconjunto de lenguaje se debería elegir cuidadosamente para evitar estructuras complejas que puedan conducir a fallos de implementación. Se deberían realizar controles para verificar la correcta utilización del subconjunto de lenguaje.				
NOTA 4	Cualquier lenguaje específico que no figura en esta tabla no tiene que ser considerado como un lenguaje a excluir. Debería cumplir esta norma internacional.				
NOTA 5	Hay numerosas extensiones del lenguaje Pascal que incluyen Pascal Libre. Las referencias a Pascal incluyen estas extensiones.				
NOTA 6	Java fue diseñado para tener un colector de basura en tiempo de ejecución. Puede definirse un subconjunto de Java que no exija la recolección de basura. Algunas implementaciones de Java suministran recolección progresiva de basura que recupera la memoria libre a medida que se ejecuta el programa y evita que se pare durante el periodo en que la memoria está agotada. Las aplicaciones en tiempo real no deberían utilizar ninguna forma de recolección de basura.				
NOTA 7	Si la implementación de Java exige un intérprete de código Java intermedio en tiempo de ejecución, dicho intérprete debe tratarse como parte del software relacionado con la seguridad de acuerdo a los requisitos de la Norma IEC 61508-3.				
NOTA 8	Para las entradas de la 15 a la 24 véase la Norma IEC 61131-3.				

C.4.6 Generación automática de software

NOTA Esta técnica/medida se menciona en la tabla A.2 de la Norma IEC 61508-3.

Objetivo: Automatizar las tareas con propensión al error de la implementación del software.

Descripción: El diseño del sistema se describe mediante un modelo (una especificación ejecutable) de mayor nivel de abstracción que el código tradicional ejecutable. El modelo se transforma automáticamente mediante un generador de código en la forma ejecutable. El objetivo es mejorar la calidad del software eliminando las tareas manuales propensas al error de codificación. Una ventaja adicional es que se pueden realizar diseños más complejos en un nivel de abstracción mayor.

Referencias:

Embedded Software Generation from System Level Design Languages, H Yu, R. Domer, D. Gajski. In “ASP-DAC 2004: Proceedings of the ASP-Dac 2004 Asia and South Pacific Design Automation Conference, 2004”, IEEE Circuits and Systems Society. IEEE, 2004, ISBN 0780381750, 9780780381759.

Transforming Process Algebra Models into UML State Machines: Bridging a Semantic Gap?. M.F. van Amstel et. al. In Theory and Practice of Model Transformations: First International Conference, ICMT”. ed. A. Vallecillo. Springer, 2008, ISBN 3540699260, 9783540699262.

C.4.7 Gestión del ensayo y herramientas de automatización

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-3.

Objetivo: Fomentar la aproximación sistemática y concienzuda al ensayo del software y del sistema.

Descripción: La utilización de las herramientas de soporte apropiadas mecaniza las tareas más laboriosas y con propensión al error en el desarrollo del sistema y da la capacidad de hacer una aproximación sistemática en la gestión del ensayo. La disponibilidad de soporte fomenta una aproximación más concienzuda tanto en el ensayo normal como en el de regresión.

Referencia:

Managing the Testing Process: Practical Tools and Techniques for Managing Hardware and Software Testing. R.Black, John Wiley and Sons, 2002, ISBN 0471223980, 9780471223986.

C.5 Verificación y modificación

C.5.1 Ensayo probabilístico

NOTA Esta técnica/medida se menciona en las tablas A.5, C.15, A.7 y C.17 de la Norma IEC 61508-3.

Objetivo: Obtener una cifra cuantitativa relativa a las propiedades de confianza del software examinado.

Descripción: El método produce una estimación estadística de la fiabilidad del software. Esta cifra cuantitativa puede tener en cuenta los niveles asociados de confianza y de significación y puede proporcionar:

- una probabilidad de disfunción por demanda;
- una probabilidad de disfunción durante un cierto periodo de tiempo; y
- una probabilidad de confinamiento de los errores.

A partir de estas cifras, se pueden obtener otros parámetros, como:

- la probabilidad de ejecución sin disfunción;
- la probabilidad de supervivencia;
- la disponibilidad;
- el MTBF o la tasa de disfunción; y
- la probabilidad de ejecución segura.

Las consideraciones probabilísticas se basan o bien en el ensayo probabilístico o bien en la experiencia de operación. Normalmente, el número de casos de ensayo o de casos observados en operación es muy importante. Típicamente, el ensayo del modo de demanda de operación necesita un tiempo mucho más corto que el modo continuo de funcionamiento.

Las herramientas de ensayo automatizadas se utilizan normalmente para proporcionar los datos de ensayo y supervisar las salidas de ensayo. Los ensayos importantes se realizan en grandes ordenadores centrales con equipos periféricos de simulación de procesos apropiados. Los datos de ensayo se seleccionan desde un punto de vista sistemático y aleatorio. Por ejemplo, el ensayo global utiliza un perfil de datos de ensayo mientras que la selección aleatoria de datos se puede utilizar para la realización de ensayos individuales detallados.

La utilización de bancos de ensayos del software, la ejecución y la supervisión de los ensayos individuales se determinan en función de los objetivos detallados de los ensayos, como se ha descrito anteriormente. Otras condiciones importantes resultan de las condiciones matemáticas previas que es necesario cumplir para la evaluación de los resultados del ensayo.

Las cifras probabilísticas relativas al comportamiento de cualquier objeto ensayado también se pueden obtener después de la experiencia en operación. Si se cumplen las mismas condiciones, se pueden aplicar los mismos métodos matemáticos utilizados para la evaluación de los resultados del ensayo.

En la práctica, es muy difícil demostrar los niveles de fiabilidad extremadamente elevados con la ayuda de estas técnicas.

Referencias:

A discussion of statistical testing on a safety-related application. S Kuball, J H R May, Proc IMechE Vol. 221 Part O: J. Risk and Reliability, Institution of Mechanical Engineers, 2007.

Estimating the Probability of Failure when Testing Reveals No Failures, W.K. Miller, L.J. Morell, et al.. IEEE Transactions on Software Engineering, Vol. 18, NO.1, pp33-43, January 1992.

Reliability estimation from appropriate testing of plant protection software, J. May, G. Hughes, A.D. Lunn. IEE Software Engineering Journal, v10 n6 pp 206-218, Nov 1995 (ISSN: 0268-6961).

Validation of ultra high dependability for software based systems, B. Littlewood and L. Strigini. Comm. ACM 36 (11), 69-80, 1993.

C.5.2 Registro y análisis de datos

NOTA Esta técnica/medida se menciona en las tablas A.5 y A.8 de la Norma IEC 61508-3.

Objetivo: Documentar todos los datos, decisiones y justificaciones asociadas al proyecto del software para facilitar la verificación, la validación, la evaluación y el mantenimiento.

Descripción: Se mantiene una documentación detallada durante el proyecto, pudiendo incluir:

- los ensayos realizados sobre cada módulo del software;
- las decisiones y sus justificaciones;
- los problemas y sus soluciones.

Esta documentación se puede analizar durante y al final del proyecto para obtener una gran variedad de información. En particular, el registro de los datos es muy importante para el mantenimiento de los sistemas informáticos ya que algunas decisiones tomadas durante el proyecto de desarrollo no siempre las conocen los ingenieros de mantenimiento.

Referencia:

Dependability of Critical Computer Systems 2. F. J. Redmill, Elsevier Applied Science, 1989, SBN ISBN 1851663819, 9781851663811.

C.5.3 Ensayo de interfaz

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-3.

Objetivo: Detectar los errores en las interfaces de los subprogramas.

Descripción: Son posibles varios niveles de detalle o de completitud de los ensayos. Los niveles más importantes son los ensayos para:

- todas las variables de interfaz en sus valores extremos;
- todas las variables de interfaz consideradas individualmente en sus valores extremos con las otras variables de interfaz en su valor normal;
- todos los valores del dominio de cada variable de interfaz con las otras variables de interfaz en su valor normal;
- todos los valores de todas las variables combinadas (esto solo se puede realizar para pequeñas interfaces);
- las condiciones de ensayo especificadas relativas a cada llamada de cada subprograma.

Estos ensayos son particularmente importantes si las interfaces no contienen aserciones que detecten valores incorrectos de parámetros. También son importantes tras generar nuevas configuraciones de subprogramas preexistentes.

C.5.4 Análisis de los valores límites

NOTA Esta técnica/medida se menciona en las tablas B.2, B.3 y B.8 de la Norma IEC 61508-3.

Objetivo: Detectar los errores de software en las fronteras o límites de los parámetros.

Descripción: El dominio de entrada del programa se divide en un cierto número de clases de entrada según relación de equivalencia (véase C.5.7). Estos ensayos deberían cubrir los límites y extremos de las clases. Los ensayos comprueban que los límites del dominio de entrada de la especificación coinciden con los del programa. La utilización del valor cero en el caso de una traducción directa e indirecta a menudo es propensa a errores y necesita una atención especial:

- división por cero;
- caracteres ASCII blancos;

- pila o elemento de lista vacías;
- matriz llena;
- entrada cero de tabla.

Normalmente, los límites para las entradas se corresponden directamente con los del rango de salida. Los casos de ensayo se deberían describir de forma que fueren la salida para que alcance sus valores límites. También es necesario considerar si es posible especificar un caso de ensayo que fuerce que la salida supere los valores límite de la especificación.

Si la salida es una secuencia de datos como, por ejemplo, una tabla impresa, se debería prestar atención particular a los primeros y últimos elementos así como a las listas que contengan cero, uno o dos elementos.

Referencia:

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

C.5.5 Suposición de los errores

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.8 de la Norma IEC 61508-3.

Objetivo: Eliminar los errores de programación comunes.

Descripción: La experiencia adquirida durante los ensayos y la intuición unida al conocimiento y a la curiosidad sobre el sistema ensayado puede añadir algunos casos de ensayos no categorizados al conjunto de casos de ensayos previstos.

Los valores especiales o las combinaciones de los valores son propensos a los errores. Algunos casos de ensayo interesantes se pueden extraer de las listas de control. La robustez del sistema también se puede cuestionar. Ejemplo: ¿Es posible pulsar muy rápido o muy a menudo los botones del panel delantero? ¿Qué pasa cuando pulso simultáneamente dos botones?

Referencia:

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

C.5.6 Implementación de errores

NOTA Esta técnica/medida se menciona en la tabla B.2 de la Norma IEC 61508-3.

Objetivo: Asegurar que un conjunto de casos de ensayo es adecuado.

Descripción: Algunos tipos de errores conocidos se implantan en el programa, posteriormente el programa se ejecuta con los casos de ensayo en condición de ensayo. Si solamente se descubren algunos de los errores implantados, el conjunto de casos de ensayo no es adecuado. La relación entre los errores implantados descubiertos y el número total de errores implantados corresponde a una estimación de la relación entre los errores reales descubiertos y el número total de errores. Esto permite estimar el número de errores restantes y, en consecuencia, el esfuerzo de ensayo restante.

$$\frac{\text{Errores implantados descubiertos}}{\text{Número total de errores implantados}} = \frac{\text{Errores reales descubiertos}}{\text{Número total de errores reales}}$$

La detección de todos los errores implantados puede significar que el conjunto de casos de ensayo es adecuado o que los errores implantados son demasiado fáciles de descubrir. Los límites del método son que para obtener cualquier resultado utilizable, los tipos de errores así como la posición de las implantaciones deben reflejar la distribución estadística de los errores reales.

Referencias:

Software Fault Injection: Inoculating Programs Against Errors. J. Voas, G. McGraw. Wiley Computer Pub., 1998, ISBN 0471183814, 9780471183815.

Faults, Injection Methods, and Fault Attacks. Chong Hee Kim, Jean-Jacques Quisquater, IEEE Design and Test of Computers, vol. 24, no. 6, pp. 544-545, Nov., 2007.

Fault seeding for software reliability model validation. A. Pasquini, E. De Agostino. Control Engineering Practice, Volume 3, Issue 7, July 1995. Elsevier Science Ltd.

C.5.7 Clases de equivalencia y ensayo de las particiones de entrada

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.3 de la Norma IEC 61508-3.

Objetivo: Ensayo del software de forma adecuada con la ayuda de un mínimo de datos de ensayo. Los datos de ensayo se obtienen seleccionando las particiones del dominio de entrada requerido para ejecutar el software.

Descripción: La estrategia de ensayo se basa en la relación de equivalencia de las entradas, que permite determinar una partición del dominio de entrada.

Los casos de ensayo se seleccionan con el objetivo de cubrir todas las particiones previamente especificadas. Para cada clase de equivalencia se tiene en cuenta al menos un caso de ensayo.

Las dos posibilidades básicas para la partición de las entradas son las siguientes:

- clases de equivalencia deducidas de la especificación; la interpretación de la especificación se puede orientar hacia las entradas (por ejemplo, los valores seleccionados se tratan de la misma manera) o hacia la salida (por ejemplo, el conjunto de valores que conducen a un mismo resultado funcional);
- las clases de equivalencia se pueden deducir de la estructura interna del programa. En este caso, los resultados de las clases de equivalencia se determinan a partir del análisis estático del programa (por ejemplo, el conjunto de valores que conducen a un mismo camino durante la ejecución).

Referencias:

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

Static Analysis and Software Assurance. D. Wagner, Lecture Notes in Computer Science, Volume 2126/2001, Springer, 2001, ISBN 978-3-540-42314-0.

C.5.8 Ensayos basados en la estructura

NOTA Esta técnica/medida se menciona en la tabla B.2 de la Norma IEC 61508-3.

Objetivo: Aplicar los ensayos que sirven para ensayar ciertos subconjuntos de la estructura del programa.

Descripción: Basado en el análisis del programa, se elige un conjunto de datos de entrada de forma que ensayen un gran porcentaje (a menudo predefinido) del código del programa. Las medidas de cobertura del código varían en función del nivel de rigor requerido, como se expone a continuación. En todos los casos, el objetivo debería ser el 100% de la métrica de cobertura seleccionada. Si no es posible obtener el 100% de la cobertura, se debería documentar en el informe de ensayo las razones por las que no se logra el 100% (por ejemplo, el código defensivo que sólo puede introducirse si surge un problema de hardware). Las primeras cuatro técnicas de la siguiente lista son las específicamente mencionadas en las recomendaciones de la tabla B.3 de la Norma IEC 61508-3 y son ampliamente soportadas por herramientas de ensayo; las técnicas restantes también podrían ser consideradas.

- **Cobertura del punto de entrada (gráfico de llamada):** asegurar que cada subprograma (subrutina o función) ha sido llamada al menos una vez (esta es la medida menos rigurosa de cobertura estructural).

NOTA n lenguajes orientados a objetos, puede haber varios subprogramas con el mismo nombre que se apliquen a diferentes variantes de un tipo polimórfico (sobrecarga de subprogramas) que pueden invocarse mediante reparto dinámico. En estos casos se debería ensayar cada subprograma sobrecargado.

- **Declaraciones:** asegurar que todas las declaraciones en el código han sido ejecutadas al menos una vez.
- **Ramificaciones:** se deberían verificar todos los lados de cada rama, lo que puede ser imposible para ciertos tipos de códigos defensivos.
- **Condiciones compuestas:** se ensaya cada condición de una ramificación condicional compuesta (es decir, con unión Y/O). Véase MCDC (Modified Condition Decision Coverage, ref. DO178B).
- **LCSAJ:** una secuencia de código lineal con salto es una secuencia lineal de instrucciones de codificación, incluyendo las instrucciones condicionales, terminada por un salto. Muchas de las subrutinas potenciales no serán factibles a causa de las limitaciones impuestas a los datos de entrada por la ejecución del código precedente.
- **Flujo de datos:** las rutas de ejecución son seleccionadas sobre la base de utilización de los datos, como, por ejemplo, una ruta en la que la misma variable se escribe y se lee.
- **Ruta de base:** una de las rutas de un conjunto mínimo de rutas que van desde el principio hasta el final, de tal modo que se incluyan todos los arcos. (Las combinaciones de las rutas que se superponen en este conjunto de base pueden formar cualquier ruta a través de esta parte del programa.) Los ensayos de todas las rutas de base han demostrado su eficacia en la detección de errores.

Referencias:

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

RTCA, Inc. document DO-178B and EUROCAE document ED-12B, *Software Considerations in Airborne Systems and Equipment Certification*, dated December 1, 1992.

C.5.9 Análisis de flujo de control

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Detectar las estructuras de baja calidad y potencialmente incorrectas del programa.

Descripción: El análisis de flujo de control es una técnica de ensayo estático para identificar las zonas susceptibles del código que no siguen las prácticas de una buena programación. El programa se analiza y se genera un gráfico orientado que permite verificar de forma más profunda la presencia de los siguientes elementos:

- código inaccesible que resulta, por ejemplo, de las ramificaciones incondicionales que parten de los bloques de código inaccesibles;
- código anudado. Un código bien estructurado tiene un gráfico de control que se puede reducir a un simple nodo por reducciones sucesivas. Por el contrario, un código poco estructurado solo se puede reducir a un nudo compuesto de varios nodos.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

C.5.10 Análisis de flujo de datos

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Detectar las estructuras de baja calidad y potencialmente incorrectas del programa.

Descripción: El análisis de flujo de datos es una técnica de ensayo estático que permite combinar las informaciones obtenidas a partir del análisis de flujo de control con aquellas relativas a la lectura y escritura de variables en diferentes partes del código. El análisis permite verificar la presencia de los siguientes elementos:

- variables que se pueden leer antes de que sean asignadas a un valor - esto se puede evitar asignando un valor cuando se declare una nueva variable;
- variables que se escriben varias veces antes de ser leídas - lo que puede indicar una omisión de código;
- variables que se escriben pero no se leen nunca - lo que puede indicar un código redundante.

Un fallo de flujo de datos no siempre corresponde directamente a un fallo de programa, pero si se evitan los fallos es menos probable que el código contenga errores.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

C.5.11 Ejecución simbólica

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Mostrar la concordancia entre el código fuente y la especificación.

Descripción: Las variables del programa se evalúan después de haber reemplazado el lado izquierdo por el derecho en todas las asignaciones. Las ramificaciones condicionales y los bucles se traducen en expresiones booleanas. El resultado final consiste en una expresión simbólica para cada variable de programa. Esta expresión es una fórmula para el valor que el programa calcularía para datos reales dados. Esta se puede verificar en relación a la expresión prevista.

Un uso relacionado de la ejecución simbólica es como una forma sistemática de generar datos de ensayo para los caminos a través de la lógica del programa. La capacidad de ejecución simbólica puede incorporarse en un grupo de herramientas integradas para proporcionar una capacidad para ensayar el elemento de software y el análisis del código.

Referencias:

Using symbolic execution for verifying safety-critical systems. A. Coen-Porisini, G. Denaro, C. Ghezzi, M. Pezzé. Proceedings of the 8th European software engineering conference, and 9th ACM SIGSOFT international symposium on Foundations of software engineering. ACM, 2001, ISBN:1-58113-390-1.

Using symbolic execution to guide test generation. G. Lee, J. Morris, K. Parker, G. Bundell, P. Lam. In Software Testing, Verification and Reliability, vol 15, no 1, 2005. John Wiley & Sons, Ltd.

C.5.12 Prueba formal (verificación)

NOTA Esta técnica/medida se menciona en las tablas A.5 y A.9 de la Norma IEC 61508-3.

Objetivo: Probar la corrección de un programa en relación a algún modelo abstracto del programa, con la ayuda de las reglas y de los modelos teóricos y matemáticos.

Descripción: El ensayo es una forma común de examinar la conformidad de un programa. Sin embargo, el ensayo exhaustivo es generalmente inalcanzable dada la complejidad de los programas de valor práctico y en consecuencia, sólo puede examinarse una fracción del posible comportamiento del programa de esta forma. En contraste, la verificación formal aplica operaciones matemáticas a una representación matemática de un programa con el fin de establecer que el programa se comporta como definen todas las posibles entradas.

La verificación formal de un sistema exige un modelo abstracto del programa y de su comportamiento exigido (es decir, una especificación) en un lenguaje con un significado matemático preciso. La especificación puede ser completa o puede estar restringida a propiedades específicas del programa:

- propiedades de corrección funcional, es decir, el programador debería exhibir una funcionalidad particular;
- propiedades de seguridad (es decir, nunca se producirá algún mal comportamiento) y de actividad (es decir, se producirá algún buen comportamiento eventualmente);
- propiedades de temporizado, es decir, se producirá algún comportamiento en un momento particular.

El resultado de la verificación formal es un argumento riguroso de que el modelo abstracto del programa es correcto en relación a la especificación para todas las posibles entradas, es decir, el modelo satisface las propiedades especificadas.

Sin embargo, la corrección del modelo no prueba directamente la corrección del programa real y es necesaria otra etapa para mostrar que el modelo es una abstracción precisa del programa real para las propiedades de interés. Algunas propiedades de interés práctico del programa no pueden formalizarse matemáticamente (por ejemplo, la mayoría de la temporización y la planificación, o las propiedades subjetivas tales como una interfaz de usuario “clara y sencilla”, o por supuesto, cualquier propiedad u objetivo de diseño que no pueda ser expresada en un lenguaje formal). La verificación formal, en consecuencia, no reemplaza completamente a la simulación y al ensayo, sino que complementa aquellas técnicas mediante el suministro de otra evidencia que soporta la correcta operación del programa para todas sus entradas. Mientras que la verificación formal puede asegurar la corrección de un modelo abstracto de un programa, el ensayo asegura que el programa real se comporta como se esperaba.

La utilización de la verificación formal en la fase de diseño puede reducir significativamente el tiempo de desarrollo mediante el descubrimiento de errores significativos y descuidos en una fase temprana del diseño, y así reducir el tiempo exigido iterando entre el diseño y el ensayo.

En el apartado C.2.4 se describen varios métodos formales utilizados en la práctica, por ejemplo, CCS, CSP, HOL, LOTOS, OBJ, lógica temporal, VDM y Z.

C.5.12.1 Chequeo del modelo

El chequeo del modelo es un método para la verificación formal de los sistemas reactivos y concurrentes. Dada una estructura de estados finitos que describa los comportamientos del sistema, se chequea que una propiedad escrita como una fórmula lógica temporal, se mantiene o no contra la estructura. Se emplean algoritmos eficaces (por ejemplo, SPIN, SMV y UPPAAL) para atravesar todos los estados de la estructura automática y exhaustivamente. Cuando la propiedad no se mantiene, se genera un contraejemplo. Este muestra cómo se ha violado la propiedad en la estructura, y contiene información muy útil para investigar el sistema. El chequeo del modelo puede detectar “errores software profundos” que podrían escapársele a la inspección y ensayo tradicionales.

Nótese que el chequeo del modelo es de ayuda en el análisis de la complejidad sutil. Esto puede ser útil en algunas aplicaciones de SIL bajo pero es necesario tener precaución si la complejidad sutil existe en aplicaciones de SIL alto.

Referencias:

Is Proof More Cost-Effective Than Testing?. S. King, R. Chapman, J. Hammond, A. Pryor. IEEE Transactions on Software Engineering, vol. 26 no. 8, August 2000.

Model Checking. E. M. Clarke, O. Grumberg, and D. A. Peled. MIT Press, 1999, ISBN 0262032708, 9780262032704.

Systems and Software Verification: Model-Checking Techniques and Tools. B. Berard, M. Bidoit, A. Finkel, F. Laroussinie, A. Petit, L. Petrucci, Ph. Schnoebelen, and P. Mckenzie, Springer, 2001, ISBN 3-540-41523-8.

Logic in Computer Science: Modelling and Reasoning about Systems. M. Huth and M. Ryan. Cambridge University Press, 2000, ISBN 0521652006, 0521656028.

The Spin Model Checker: Primer and Reference Manual. G. J. Holzmann. Addison-Wesley, 2003, ISBN 0321228626, 9780321228628.

C.5.12.2 (Disponible)

C.5.13 Métricas de complejidad

NOTA Esta técnica/medida se menciona en las tablas B.9 y C.19 de la Norma IEC 61508-3.

Objetivo: Predecir los atributos de los programas a partir de las características del software propiamente dicho o de su desarrollo o desde el histórico de ensayo.

Descripción: Estos modelos evalúan ciertas características estructurales del software y las relacionan con el atributo deseado como la fiabilidad y la complejidad. Son necesarias herramientas de software en la evaluación de la mayor parte de las medidas. Una parte de las métricas aplicables se indica a continuación:

- complejidad teórica del gráfico: esta medida se puede aplicar al principio del ciclo de vida para evaluar los compromisos técnicos y se basa en la complejidad del gráfico de control del programa representado por su número ciclomático;
- número de formas de activar un cierto módulo del software (accesibilidad) - cuanto más accesible es un módulo del software, es más probable tener que depurarlo;
- ciencia de las métricas de tipo Halstead: esta medida calcula la longitud del programa contando el número de operadores y de operandos. Proporciona una medida de la complejidad y del volumen que sirve como base de comparación para la evaluación de los recursos de desarrollo futuros.
- número de entradas y de salidas para cada módulo del software: la minimización del número de puntos de entrada/salida es un factor clave en la utilización de las técnicas de diseño y de programación estructurada.

Referencia:

Metrics and Models in Software Quality Engineering. S.H. Kan. Addison-Wesley, 2003, ISBN 0201729156, 9780201729153.

C.5.14 Inspecciones formales

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Revelar los defectos en un elemento de software

Descripción: La inspección formal es un proceso estructurado para inspeccionar el material software que se realiza por pares de la persona que produce el material, para encontrar defectos y para permitir al productor mejorar el material. El productor no debería tomar parte en el proceso de inspección, salvo para dar explicaciones a los inspectores durante la etapa de familiarización. Las inspecciones formales pueden realizarse sobre elementos específicos del software producidos en cualquier fase del ciclo de vida del desarrollo software.

Antes de que la inspección tenga lugar, los inspectores deberían familiarizarse con los materiales a ser inspeccionados. Los roles de los inspectores en el proceso de inspección deberían estar claramente definidos. Se debería preparar una agenda de inspección. Los criterios de entrada y de salida deberían estar definidos basados en las propiedades exigidas para el elemento de software. Los criterios de entrada son los criterios o requisitos que deben cumplirse antes de que la inspección tenga lugar. Los criterios de salida son los criterios o requisitos que deben cumplirse para completar un proceso específico.

Durante la inspección, los hallazgos de la misma deberían ser formalmente registrados por el moderador, cuyo rol es facilitar la inspección. Debería alcanzarse un consenso sobre los hallazgos por todos los inspectores. Los defectos deberían clasificarse como o bien a) los que exigen rectificación antes de la aceptación, o bien, b) los que exigen rectificación en un tiempo dado/fecha límite. Los defectos identificados deberían remitirse al productor para la subsiguiente rectificación después de haber completado la inspección. Dependiendo del número y alcance de los defectos identificados, el moderador puede determinar si es necesaria otra inspección adicional del material software.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

Fagan, M. *Design and Code Inspections to Reduce Errors in Program Development*. IBM Systems Journal 15, 3 (1976): 182-211.

C.5.15 Sondeo (software)

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Revelar discrepancias entre la especificación y la implementación.

Descripción: El sondeo es una técnica informal, realizada por el productor de un elemento de software en presencia de sus pares con el objetivo de encontrar defectos en el elemento de software. Puede realizarse sobre elementos de software específicos producidos en cualquier fase del ciclo de vida del desarrollo software.

Las funciones especificadas se examinan y evalúan para asegurar que el sistema relacionado con la seguridad es conforme a los requisitos de la especificación. Todos los puntos de duda relativos a la implementación y utilización del producto se documentan de manera que puedan resolverse. En contraste a la inspección formal, el autor es activo durante el procedimiento de sondeo.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

C.5.16 Revisión del diseño

NOTA Esta técnica/medida se menciona en la tabla B.8 de la Norma IEC 61508-3.

Objetivo: Revelar defectos en el diseño del software.

Descripción: Una revisión del diseño es un examen formal, documentado, exhaustivo y sistemático del diseño del software para evaluar los requisitos de diseño y la capacidad del diseño para cumplir estos requisitos e identificar los problemas y proponer soluciones.

Las Revisiones del Diseño suministran los medios para evaluar el estado del diseño contra los requisitos de entrada, y los medios para identificar las oportunidades de otras mejoras. A medida que las actividades del ciclo de vida del desarrollo progresan y se van cumpliendo hitos de diseño detallados de mayor importancia, se deberían mantener las Revisiones del Diseño para revisar todos los aspectos de interfaz, asegurar que el diseño puede verificarse para asegurar que el diseño cumple sus requisitos y asegurar que el diseño más apropiado es consistente con los requisitos de seguridad. Tal revisión está primordialmente prevista para verificar el trabajo de los diseñadores y debería tratarse como una actividad de confirmación y refinamiento.

Puede utilizarse una técnica rigurosa de inspección tal como el “análisis sigiloso del circuito” para detectar el comportamiento software incorrecto tal como un camino o flujo lógico no esperados, salidas no previstas, temporizado incorrecto, acciones no deseadas.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

The Art of Software Testing, second edition. G. J. Myers, T. Badgett, T. M. Codd, C. Sandler, John Wiley and Sons, 2004, ISBN 0471469122, 9780471469124.

IEC 61160:2005 *Revisión de diseño*.

Space Product Assurance, Sneak analysis. Part 2: Clue list. ECSS-Q-40-04A Part 2. ESA Publications Division, Noordwijk, 1997, ISSN 1028-396X.
http://www.everspec.com/ESA/ECSS-Q-40-04A_Part-2_14981/

C.5.17 Prototipo/animación

NOTA Esta técnica/medida se menciona en las tablas B.3 y B.5 de la Norma IEC 61508-3.

Objetivo: Verificar la viabilidad de la implementación del sistema en relación a las limitaciones impuestas. Comunicar al cliente la interpretación del especificador del sistema con el fin de detectar los errores de comprensión.

Descripción: Se selecciona un subconjunto de funciones, de limitaciones y de requisitos de características de funcionamiento del sistema. Se construye un prototipo con la ayuda de herramientas de alto nivel. En este estado, no es necesario tener en cuenta las limitaciones como el ordenador objetivo, el lenguaje de implementación, el tamaño del programa, el mantenimiento, la fiabilidad y la disponibilidad. El prototipo se evalúa en relación a los criterios del cliente y los requisitos del sistema se pueden modificar en función de esta evaluación.

Referencia:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

C.5.18 Simulación del proceso

NOTA Esta técnica/medida se menciona en las tablas A.7, C.7, B.3 y C.13 de la Norma IEC 61508-3.

Objetivo: Ensayar la función de un sistema de software y su interfaz con el mundo exterior, sin que se le permita modificar, de ninguna manera, el mundo real.

Descripción: Creación de un sistema que permite únicamente la realización de ensayos y que simula el comportamiento del equipo sometido a control (EUC).

La simulación puede ser sólo software o una combinación del software y del hardware. Es necesario:

- que proporcione unas entradas equivalentes a las utilizadas después de la instalación real del equipo sometido a control (EUC);
- que responda a las salidas del software ensayado de una forma que represente fielmente la planta controlada;
- que proporcione unas entradas de operador para cualquier perturbación que tenga que afrontar el sistema.

Cuando se está ensayando el software, la simulación puede ser una simulación del hardware objetivo con sus entradas y sus salidas.

Referencias:

EmStar: An Environment for Developing Wireless Embedded Systems Software. J Elson et al.
http://cens.ucla.edu/TechReports/9_emstar.pdf

A hardware-software co-simulator for embedded system design and debugging. A. Ghosh et al. In Proceedings of the IFIP International Conference on Computer Hardware Description Languages and Their Applications, IFIP International Conference on Very Large Scale Integration, 1995. IEEE, 1995, ISBN 4930813670, 9784930813671.

C.5.19 Requisitos de funcionamiento

NOTA Esta técnica/medida se menciona en la tabla B.6 de la Norma IEC 61508-3.

Objetivo: Establecer los requisitos de funcionamiento de un sistema del software que sean demostrables.

Descripción: Se realiza un análisis de las especificaciones relativas al sistema y de los requisitos aplicables al software con el fin de especificar todos los requisitos de funcionamiento generales, específicos, explícitos e implícitos.

Cada requisito de funcionamiento se examina para determinar:

- el criterio de éxito a obtener;

- si se puede obtener una medida en relación al criterio de éxito;
- la precisión potencial de estas medidas;
- las fases del proyecto en las que se pueden estimar las medidas; y
- las fases del proyecto en las que se pueden realizar las medidas.

El carácter práctico de cada requisito de funcionamiento se analiza para obtener una lista de requisitos de funcionamiento, de los criterios de éxito y de las medidas potenciales. Los objetivos principales son los siguientes:

- asociar cada requisito de funcionamiento con al menos una medida;
- en la medida de lo posible, seleccionar las medidas precisas y eficaces que se puedan utilizar tan pronto como sea posible durante el desarrollo;
- especificar los requisitos de funcionamiento esenciales y opcionales y los criterios de éxito; y
- en la medida de lo posible, se debería aprovechar la posibilidad de utilizar una sola medida para varios requisitos de funcionamiento.

Referencia:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

C.5.20 Modelización del funcionamiento

NOTA Esta técnica/medida se menciona en las tablas B.2 y B.5 de la Norma IEC 61508-3.

Objetivo: Asegurar que la capacidad de trabajo del sistema es suficiente para satisfacer los requisitos especificados.

Descripción: La especificación de los requisitos incluye los requisitos de capacidad de procesamiento y de respuesta aplicables a las funciones específicas, que se pueden combinar con las limitaciones de utilización de los recursos totales del sistema. El diseño propuesto del sistema se compara con los requisitos establecidos:

- produciendo un modelo de los procesos del sistema y sus interacciones;
- determinando la utilización de los recursos por cada proceso como, por ejemplo, el tiempo de procesador, el ancho de banda de las comunicaciones, los dispositivos de almacenamiento, etc.;
- determinando la distribución de las demandas enviadas al sistema en unas condiciones medias y en las condiciones más desfavorables;
- calculando la cantidad de procesamiento y el tiempo de respuesta en unas condiciones medias y en las más desfavorables para cada función individual del sistema.

Para los sistemas simples, puede ser suficiente una solución analítica, mientras que para los sistemas complejos, una cierta forma de simulación puede ser más adecuada para obtener unos resultados precisos.

Antes de una modelización detallada, se puede realizar un control más simple apoyándose en el “presupuesto de recursos” sumando los recursos requeridos para todos los procesos. Si los requisitos sobrepasan la capacidad del sistema diseñado, el diseño es imposible. Si el diseño satisface este control, la modelización del funcionamiento puede mostrar que aparecen retrasos y tiempos de respuesta muy largos a causa de la falta de recursos. Para evitar esta situación, los ingenieros a menudo diseñan los sistemas utilizando solo una parte de los recursos totales (por ejemplo, el 50%) con el fin de reducir los riesgos relacionados con la falta de recursos.

Referencia:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

C.5.21 Ensayos de avalancha/estrés

NOTA Esta técnica/medida se menciona en la tabla B.6 de la Norma IEC 61508-3.

Objetivo: Aplicar al objeto ensayado una carga de trabajo excepcionalmente elevada para demostrar que el objeto ensayado soportará fácilmente las cargas de trabajo normales.

Descripción: Se pueden aplicar un gran número de condiciones de ensayo para los ensayos de avalancha/estrés. Algunas de estas condiciones son las siguientes:

- en caso de funcionamiento en modo escrutinio, se generan en la entrada del objeto ensayado un número mucho más importante de cambios por unidad de tiempo que en condiciones normales;
- en caso de funcionamiento en demanda, el número de demandas enviadas al objeto ensayado por unidad de tiempo sobrepasa el previsto en condiciones normales;
- si el tamaño de una base de datos juega un papel importante, este tamaño se incrementa sobre el previsto en condiciones normales;
- los dispositivos influyentes se regulan de forma que funcionen a su velocidad máxima o mínima;
- para los casos extremos, todos los factores influyentes son, en la medida de lo posible, simultáneamente colocados en las condiciones límites.

En estas condiciones de ensayo, el comportamiento en el tiempo del objeto ensayado se puede evaluar. Se puede observar el efecto de los cambios de carga. Se puede verificar la dimensión correcta de los almacenamientos internos, variables dinámicas, pilas, etc.

Referencia:

Software Engineering for Real-time Systems. J. E. Cooling, Pearson Education, 2003, ISBN 0201596202, 9780201596205.

C.5.22 Tiempo de respuesta y limitaciones de memoria

NOTA Esta técnica/medida se menciona en la tabla B.6 de la Norma IEC 61508-3.

Objetivo: Asegurar que el sistema satisfará los requisitos temporales y de memoria.

Descripción: La especificación de los requisitos aplicables al sistema y al software consta de los requisitos de memoria y de respuesta aplicables a las funciones específicas, que pueden combinarse con las limitaciones de utilización de los recursos totales del sistema.

Se efectúa un análisis para determinar las demandas de repartición en las condiciones medias y en el caso más desfavorable. Este análisis necesita una estimación de utilización de los recursos y el tiempo transcurrido para cada función del sistema. Estas estimaciones se pueden obtener de varias formas como, por ejemplo, por comparación con un sistema existente o por prototipo y evaluación de los rendimientos de los sistemas con duración crítica.

C.5.23 Análisis de impacto

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-3.

Objetivo: Determinar los efectos de una modificación o de una mejora aportada a un sistema del software sobre los otros módulos del software de este sistema así como sobre otros sistemas.

Descripción: Antes de realizar la modificación o la mejora al software, se debería realizar un análisis para determinar el impacto de la modificación o de la mejora sobre el software con el fin de determinar qué sistemas del software y módulos del software se ven afectados.

Después del análisis, se debe tomar una decisión en lo relativo a la reverificación del sistema del software. Esta decisión depende del número de módulos del software afectados, de la criticidad de los módulos del software afectados y de la naturaleza de la modificación. Algunas decisiones posibles son:

- reverificación solamente del módulo del software modificado;
- reverificación de todos los módulos del software afectados; o
- reverificación del sistema completo.

Referencia:

Requirements Engineering. E. Hull, K. Jackson, J. Dick. Springer, 2005, ISBN 1852338792, 9781852338794.

C.5.24 Gestión de la configuración del software

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-3.

Objetivo: El objetivo de la gestión de la configuración del software es asegurar la coherencia de los grupos de versiones de desarrollo a medida que se cambian estas versiones. La gestión de la configuración se aplica en general al desarrollo del hardware y del software.

Descripción: La gestión de la configuración del software es una técnica que se utiliza a lo largo de todo el desarrollo (véase la Norma IEC 61508-3, 6.2.3). Consiste esencialmente en documentar la producción de cada versión significativa y cada relación entre las distintas versiones de los diferentes productos. La documentación resultante permite al desarrollador determinar el efecto sobre otras entregas del cambio de un producto (especialmente uno de sus elementos). En particular, los sistemas o subsistemas se pueden reconstruir de forma segura a partir de conjuntos coherentes de versiones de elementos.

Referencias:

Software engineering: Update. Ian Sommerville, Addison-Wesley Longman, Amsterdam; 8th ed., 2006, ISBN 0321313798, 9780321313799.

Software Engineering. Ian Sommerville, Pearson Studium, 8. Auflage, 2007, ISBN 3827372577, 9783827372574.

Software Configuration Management: Coordination for Team Productivity. W.A. Babich. Addison-Wesley, 1986, ISBN 0201101610, 9780201101614.

CMMI: guidelines for process integration and product improvement, Mary Beth Chrissis, Mike Konrad, Sandy Shrum, Addison-Wesley, 2003, ISBN 0321154967, 9780321154965.

C.5.25 Validación de regresión

NOTA Esta técnica/medida se menciona en la tabla A.8 de la Norma IEC 61508-3.

Objetivo: Asegurar que se extraen conclusiones válidas del ensayo de regresión.

Descripción: El ensayo de regresión completo de un sistema grande o complejo normalmente exigirá mucho esfuerzo y recursos. Cuando sea posible, es deseable restringir el ensayo de regresión para cubrir sólo los aspectos del sistema de interés directo en ese punto del desarrollo del sistema. En este ensayo parcial de regresión es esencial tener un claro entendimiento del alcance del ensayo parcial y extraer sólo conclusiones válidas en relación al estado ensayado del sistema.

Referencia:

Managing the Testing Process: Practical Tools and Techniques for Managing Hardware and Software Testing. R.Black, John Wiley and Sons, 2002, ISBN 0471223980, 9780471223986.

C.5.26 Animación de la especificación y del diseño

NOTA Esta técnica/medida se menciona en la tabla A.9 de la Norma IEC 61508-3.

Objetivo: Guiar la verificación del software con medios de examen sistemático de la especificación.

Descripción: Se examina una representación del software, que es más abstracta que el código ejecutable (es decir, una especificación o un diseño de alto nivel) para determinar el comportamiento del software final ejecutable. El examen se automatiza de alguna forma (dependiendo de las posibilidades que nos podamos permitir en función de la naturaleza y nivel de abstracción de la representación de mayor nivel) de manera que se simule el comportamiento y las salidas del software ejecutable. Una aplicación de esta aproximación es generar ensayos (u “oráculos”) que pueden ser más tarde aplicados al software ejecutable, automatizando hasta cierto grado el proceso de ensayo. Otra aplicación es animar la interfaz de usuario de manera que los usuarios finales no técnicos pueden conseguir cierta apreciación del significado detallado de la especificación sobre la cual trabajarán los desarrolladores del software. Eso suministra un valioso método de comunicación entre los dos grupos.

Referencias:

Supporting the Software Testing Process through Specification Animation. T.Miller, P.Strooper. In Proceedings of the First International Conference on Software Engineering and Formal Methods (SEFM'03), ed. P.Lindsay. IEEE Computer Society, IEEE Computer Society, 2003, ISBN 0769519490, 9780769519494.

B model animation for external verification. H.Waeselynck, S.Behnia, In Proceedings. of the Second International Conference on Formal Engineering Methods, 1998. IEEE Computer Society, 1998, ISBN 0-8186-9198-0.

C.5.27 Ensayo basado en modelo (generación de casos de ensayo)

NOTA Esta técnica/medida se menciona en la tabla A.5 de la Norma IEC 61508-3.

Objetivo: Facilitar la generación automática eficaz del caso de ensayo a partir de los modelos del sistema y generar paquetes de ensayo de alta capacidad de repetición.

Descripción: El Ensayo Basado en Modelo (MBT) es una aproximación de caja negra en la que las tareas comunes de ensayo tales como la generación de caso de ensayo (TCG) y la evaluación de los resultados de ensayo se basan en un modelo de sistema (aplicación) sometido a ensayo (SUT). Típicamente, pero no solamente, los datos del sistema y el comportamiento de usuario se modelan utilizando máquinas de estados finitos, procesos de Markov, tablas de decisión o similares (El-Far, 2001, generalizado). Adicionalmente, el ensayo basado en modelo puede combinarse con una medida de cobertura de ensayo de nivel de código fuente y los modelos funcionales pueden basarse en el código fuente existente.

El ensayo basado en modelo es la generación automática de procedimientos/casos de ensayo eficaces utilizando modelos de los requisitos del sistema y de la funcionalidad específica. (SoftwareTech, 2009).

Puesto que el ensayo es muy caro, hay una gran demanda de herramientas de generación automática de casos de ensayo. En consecuencia, el ensayo basado en modelo es actualmente un campo muy activo de investigación, que da como resultado un gran número de herramientas de Generación de Casos de Ensayo (TCG) disponibles. Estas herramientas típicamente extraen un paquete de ensayo a partir de la parte del modelo relativa al comportamiento, garantizando el cumplimiento de ciertos requisitos de cobertura.

El modelo es una representación abstracta y parcial del comportamiento deseado del sistema sometido a ensayo (SUT). A partir de este modelo, se derivan los modelos de ensayo, construyendo un paquete de ensayo abstracto. Los casos de ensayo se derivan a partir de este paquete de ensayo abstracto y se ejecutan contra el sistema, y los ensayos pueden ejecutarse también contra el modelo del sistema. El MBT con TCG se basa en y está fuertemente ligado a la utilización de métodos formales, por lo que las recomendaciones son similares a las de los niveles de integridad de la seguridad: HR (altamente recomendadas) para SIL más altos y no exigidas para SIL más bajos.

Las actividades específicas son, en general:

- construir el modelo (a partir de los requisitos del sistema);
- generar las entradas esperadas;
- generar las salidas esperadas;
- ejecutar los ensayos;
- comparar las salidas reales con las salidas esperadas;
- decidir sobre otras acciones posteriores (modificación del modelo, generación de más ensayos, estimación de la fiabilidad/calidad del software).

Los ensayos pueden derivarse mediante diferentes métodos y técnicas para la expresión de modelos de comportamiento usuario/sistema, por ejemplo,

- mediante la utilización de tablas de decisión;
- mediante la utilización de máquinas de estados finitos;
- mediante la utilización de gramáticas;
- mediante la utilización de modelos de Cadena de Markov;
- mediante la utilización de mapas de estados;
- mediante la demostración de teoremas;
- mediante la programación lógica de limitaciones;
- mediante el chequeo del modelo;
- mediante ejecución simbólica;
- mediante la utilización del modelo de evento-flujo;
- ensayos reactivos del sistema: autómata finita de jerarquía paralela;
- etc.

Recientemente, el ensayo basado en modelo está centrándose en el dominio crítico de la seguridad. Esto permite la exposición temprana de ambigüedades en la especificación y el diseño, proporciona la capacidad de generar automáticamente muchos ensayos no repetitivos eficaces, permite evaluar los paquetes de ensayo de regresión y valorar la fiabilidad y calidad del software y hace más fácil la actualización de los paquetes de ensayo.

ElFar (2001) y SoftwareTech 2009 (véanse las referencias) suministran una presentación concienzuda, los otros detalles y temas específicos del dominio se discuten en las otras referencias.

Referencias:

T. Bauer, F. Böhr, D. Landmann, T. Beletski, R. Eschbach, Robert and J.H. Poore, *From Requirements to Statistical Testing of Embedded Systems* Software Engineering for Automotive Systems - SEAS 2007, ICSE Workshops, Minneapolis, USA.

Eckard Bringmann, Andreas Krämer; *Model-based Testing of Automotive Systems* In: ICST, pp.485-493, 2008 International Conference on Software Testing, Verification, and Validation, 2008.

Broy M., *Challenges in automotive software engineering*, International conference on Software engineering (ICSE '06), Shanghai, China, 2006.

I. K. El-Far and J. A. Whittaker, *Model-Based Software Testing*. Encyclopedia of Software Engineering (edited by J. J. Marciniak). Wiley, 2001.

Heimdahl, M.P.E.: *Model-based testing: challenges ahead*, Computer Software and Applications Conference (COMPSAC 2005), 25-28 July 2005, Edinburgh, Scotland, UK, 2005.

Jonathan Jacky, Margus Veanes, Colin Campbell, and Wolfram Schulte, *Model-Based Software Testing and Analysis with C#*, ISBN 978-0-521-68761-4, Cambridge University Press 2008.

A. Paradkar, *Case Studies on Fault Detection Effectiveness of Model-based Test Generation Techniques*, in ACM SIGSOFT SW Engineering Notes, Proc. of the first int. workshop on Advances in model-based testing A-MOST '05, Vol. 30 Issue 4. ACM Press 2005.

S. J. Prowell, *Using Markov Chain Usage Models to Test Complex Systems*, HICSS '05: 38th Annual Hawaii, International Conference on System Sciences, 2005.

Mark Utting and Bruno Legeard, *Practical Model-Based Testing: A Tools Approach*, ISBN 978-0-12-372501-1, Morgan-Kaufmann 2007.

Hong Zhu et al. (2008). AST '08: *Proceedings of the 3rd International Workshop on Automation of Software Test*. ACM Press. ISBN 978-1-60558-030-2.

Model-Based Testing of Reactive Systems Advanced Lecture Series, LNCS 3472, Springer-Verlag, 2005, ISBN 978-3-540-26278-7.

Model-based Testing, SoftwareTech July 2009, Vol. 12, No. 2, Software Testing: A Life Cycle Perspective, <http://www.goldpractices.com/practices/mbt/>

C.6 Evaluación de la seguridad funcional

NOTA Las técnicas y medidas relacionadas se describen también en el capítulo B.6.

C.6.1 Tablas de decisión (tablas de verdad)

NOTA Esta técnica/medida se menciona en las tablas A.10 y B.7 de la Norma IEC 61508-3.

Objetivo: Proporcionar una especificación y un análisis claro y coherente de las combinaciones lógicas complejas y de sus relaciones.

Descripción: Este método utiliza unas tablas de dos dimensiones para describir de forma concisa las relaciones entre las variables de programa booleanas.

La precisión y la naturaleza tabular del método lo hacen apropiado como medio de análisis de las combinaciones lógicas complejas expresadas en el código.

El método es potencialmente ejecutable si se utiliza como una especificación.

C.6.2 Estudio del peligro y de la operatividad del software (CHAZOP, AMFE)

Objetivo: Determinar los peligros relacionados con la seguridad de un sistema propuesto o existente, las causas y las consecuencias posibles y las acciones recomendadas para minimizar su aparición.

Descripción: Un equipo de ingenieros cuya experiencia cubra el conjunto del sistema en estudio realiza un examen estructurado de un diseño, durante una serie de reuniones programadas. Estos ingenieros examinan los aspectos funcionales del diseño así como la forma en el que el sistema funcionaría en la práctica (incluyendo las intervenciones humanas y el mantenimiento). El jefe del equipo anima la creatividad de los miembros del equipo en lo relativo a la denuncia de los peligros potenciales y conduce el procedimiento presentando cada parte del sistema en función de varias palabras claves: “ningún”, “más de”, “menos de”, “parte de”, “más que” (o “tan bien como”) y “distinto de”. Cada tipo de condición o modo de disfunción se considera en función de su posibilidad, de la forma en que se puede producir, unas consecuencias posibles (¿hay un peligro?), de la manera en que se puede evitar y el gasto para evitarlo.

Los estudios de peligros se pueden realizar en varias fases del desarrollo del proyecto, pero son más eficaces cuando se realizan suficientemente pronto como para influir las decisiones primordiales relativas al diseño y la operatividad.

La técnica HAZOP se desarrolló en la industria de los procesos y requiere una modificación para la aplicación al software. Se han propuesto diferentes métodos derivados (HAZOP para ordenador, CHAZOP), que en general, utilizan nuevas palabras claves y/o sugieren esquemas para la cobertura sistemática de la arquitectura del sistema y del software.

Referencias:

OF-FMEA: an approach to safety analysis of object-oriented software intensive systems, T. Cichocki, J. Gorski. In Artificial Intelligence and Security in Computing Systems: 9th International Conference, ACS '2002. Ed. J. Soldek. Springer, 2003, ISBN 1402073968, 9781402073960.

Software FMEA techniques. P.L.Goddard. In Proc Annual 2000 Reliability and Maintainability Symposium, IEEE, 2000, ISBN: 0-7803-5848-1.

Software criticality analysis of COTS/SOUP. P.Bishop, T.Clement, S.Guerra. In Reliability Engineering & System Safety, Volume 81, Issue 3, September 2003, Elsevier Ltd., 2003.

C.6.3 Análisis de las disfunciones de causa común

NOTA 1 Esta técnica/medida se menciona en la tabla A.10 de la Norma 61508-3.

NOTA 2 Véase también el anexo D de la Norma 61508-6.

Objetivo: Determinar las disfunciones potenciales de los sistemas múltiples o subsistemas múltiples susceptibles de reducir las ventajas de la redundancia por la aparición simultánea de las mismas disfunciones en las múltiples partes.

Descripción: Los sistemas encargados de la seguridad de una instalación utilizan a menudo la redundancia del hardware y el voto mayoritario. Esta técnica se utiliza con el fin de evitar las disfunciones aleatorias del hardware en los componentes o en los subsistemas que tenderían a impedir el procesamiento correcto de los datos.

Sin embargo, algunas disfunciones pueden ser comunes a varios componentes o subsistemas. Por ejemplo, si un sistema se instala en una sola habitación, los defectos del sistema de aire acondicionado pueden afectar a las ventajas de la redundancia. Esto mismo es cierto para otros efectos externos sobre el sistema tales como el fuego, inundación, las perturbaciones electromagnéticas, los accidentes de avión y terremotos. El sistema también puede ser afectado por los incidentes relacionados con la operación y el mantenimiento. Es esencial que se proporcionen los procedimientos adecuados y bien documentados para la operación y el mantenimiento y que el personal de operación y mantenimiento esté perfectamente entrenado.

Los efectos internos también contribuyen de forma importante a la aparición de disfunciones de causa común. Pueden deberse a los fallos de diseño de los componentes comunes o idénticos y de sus interfaces, así como al envejecimiento de los componentes. El análisis de las disfunciones de causa común tiene por objetivo buscar las disfunciones comunes potenciales del sistema. Los siguientes métodos se utilizan para el análisis de las disfunciones de causa común: control general de la calidad, revisiones de diseño, verificación y ensayo por un equipo independiente y análisis de los incidentes reales con retorno de la experiencia adquirida en sistemas similares. Sin embargo, el objeto del análisis va más allá de los elementos del hardware. Incluso si se utiliza una cierta diversidad del software para los distintos canales de un sistema redundante, puede haber cierta similitud en las aproximaciones del software susceptibles de ocasionar una disfunción de causa común como, por ejemplo, los fallos de la especificación común.

Cuando las disfunciones de causa común no se producen exactamente en el mismo momento, pueden tomarse precauciones utilizando los métodos de comparación entre los canales múltiples, que deberían permitir la detección de una disfunción antes de que esta disfunción sea común a todos los canales. El análisis de las disfunciones de causa común debería tener en cuenta esta técnica.

Referencias:

Reliability analysis of hierarchical computer-based systems subject to common-cause failures. L.Xing, L.Meshkat, S.Donohue. Reliability Engineering & System Safety Volume 92, Issue 3, March 2007.

C.6.4 Diagramas de bloques de fiabilidad

NOTA 1 Esta técnica/medida se menciona en la tabla A.10 de la Norma IEC 61508-3 y se utiliza en el anexo B de la Norma IEC 61508-6.

NOTA 2 Véase también el apartado B.6.6.7 “Diagramas de bloques de fiabilidad”

Objetivo: Modelizar en forma de diagrama, el conjunto de los eventos que se deben producir y las condiciones que se tienen que cumplir para obtener el funcionamiento correcto de un sistema o de una tarea.

Descripción: El objetivo del análisis se representa por una ruta satisfactoria constituida por bloques, líneas y uniones lógicas. La ruta satisfactoria empieza en un lado del diagrama y continúa a través de los bloques y las uniones hacia el otro lado del diagrama. Un bloque representa una condición o un evento y la ruta puede atravesar el bloque si la condición se cumple o el evento se produce. Cuando la ruta llega a una unión, continúa si la lógica de la unión se cumple. Cuando llega a un vértice, la ruta puede continuar a lo largo de todas las líneas salientes. Si al menos una ruta satisfactoria atraviesa el diagrama, el análisis es satisfactorio.

Referencias:

IEC 61025:2006 *Análisis por árbol de disfunción (AAF).*

From safety analysis to software requirements. K.M. Hansen, A.P. Ravn, A.P. V Stavridou. IEEE Trans Software Engineering, Volume 24, Issue 7, Jul 1998.

IEC 61078:2006 *Técnicas de análisis de la confiabilidad. Método del diagrama de bloques de la fiabilidad y métodos booleanos.*

ANEXO D (Informativo)

UNA APROXIMACIÓN PROBABILÍSTICA PARA DETERMINAR LA INTEGRIDAD DE SEGURIDAD DEL SOFTWARE PARA UN SOFTWARE PREDESARROLLADO

D.1 Generalidades

Este anexo proporciona las directrices iniciales relativas a la utilización de una aproximación probabilística para determinar la integridad de seguridad del software para un software predesarrollado a partir de la experiencia en operación. Esta aproximación se considera particularmente apropiada como parte de la cualificación de los sistemas operativos, de los módulos de biblioteca, de los compiladores y otros software del sistema. Este anexo proporciona una indicación sobre lo que es posible, pero las técnicas solo deberían utilizarlas aquellos que sean competentes en análisis estadístico.

NOTA Este anexo utiliza el término nivel de confianza, que se describe en la publicación IEEE 352. Un término equivalente, nivel de significación, se utiliza en la Norma IEC 61164.

Las técnicas también podrían utilizarse para demostrar la mejora del nivel de integridad de seguridad del software en el tiempo. Por ejemplo, se puede mostrar que un software realizado según los requisitos de la Norma IEC 61508-3 de forma que alcance el nivel de integridad de seguridad 1 (SIL1) puede, después de un cierto periodo de utilización satisfactorio en un gran número de aplicaciones, alcanzar el nivel SIL2.

La tabla D.1, a continuación, muestra el número de demandas sin disfunción tratadas o el número de horas de operación sin disfunción necesarias para obtener la calificación correspondiente a un cierto nivel de integridad de seguridad. Esta tabla es un resumen de los resultados dados en los apartados D.2.1 y D.2.3.

La experiencia en operación se puede tratar de forma matemática como se presenta en el capítulo D.2, a continuación, para completar o reemplazar el ensayo estadístico, y se puede combinar la experiencia en operación que viene de varios sitios (es decir, sumando el número de demandas tratadas o las horas de operación), pero solamente si:

- la versión del software a utilizar en el sistema E/E/PE relacionado con la seguridad es idéntica a la versión para la que se declara la experiencia en operación;
- el perfil operacional del espacio de entrada es similar;
- hay un sistema efectivo para informar y documentar las disfunciones; y
- las condiciones previas significativas (véase el capítulo D.2, a continuación) se cumplen.

Tabla D.1 – Histórico necesario para asegurar los niveles de integridad de seguridad

SIL	Modo de funcionamiento de baja demanda	Número de demandas tratadas		Modo de funcionamiento continuo o alta demanda	Número total de horas de funcionamiento	
		$1 - \alpha = 0,99$	$1 - \alpha = 0,95$		$1 - \alpha = 0,99$	$1 - \alpha = 0,95$
	(Probabilidad de disfunción bajo demanda para realizar la función de diseño)			(Probabilidad de disfunción peligrosa por hora)		
4	$\geq 10^{-5}$ a $< 10^{-4}$	$4,6 \times 10^5$	3×10^5	$\geq 10^{-9}$ a $< 10^{-8}$	$4,6 \times 10^9$	3×10^9
3	$\geq 10^{-4}$ a $< 10^{-3}$	$4,6 \times 10^4$	3×10^4	$\geq 10^{-8}$ a $< 10^{-7}$	$4,6 \times 10^8$	3×10^8
2	$\geq 10^{-3}$ a $< 10^{-2}$	$4,6 \times 10^3$	3×10^3	$\geq 10^{-7}$ a $< 10^{-6}$	$4,6 \times 10^7$	3×10^7
1	$\geq 10^{-2}$ a $< 10^{-1}$	$4,6 \times 10^2$	3×10^2	$\geq 10^{-6}$ a $< 10^{-5}$	$4,6 \times 10^6$	3×10^6
NOTA 1 $1 - \alpha$ representa el nivel de confianza.						
NOTA 2 Véanse los apartados D.2.1 y D.2.3 para los requisitos previos y los detalles relativos a la obtención de esta tabla.						

D.2 Fórmulas de ensayos estadísticos y ejemplos de utilización

D.2.1 Ensayo estadístico simple en modo de funcionamiento de baja demanda

D.2.1.1 Requisitos previos

- a) La distribución de los datos de ensayo es igual a la distribución de las demandas durante la operación en línea.
- b) Las ejecuciones de los ensayos son estadísticamente independientes en relación a la causa de disfunción.
- c) Existencia de un mecanismo que permite la detección de disfunciones que puedan producirse.
- d) Número de casos de ensayo $n > 100$.
- e) No ocurre ninguna disfunción durante los n casos de ensayo.

D.2.1.2 Resultados

La probabilidad de disfunción p (por demanda), al nivel de confianza $1 - \alpha$, se obtiene por:

$$p \leq 1 - \sqrt[n]{\alpha} \quad \text{o} \quad n \geq - \frac{\ln \alpha}{p}$$

D.2.1.3 Ejemplo

Tabla D.2 – Probabilidad de disfunción en modo de funcionamiento de baja demanda

$1 - \alpha$	p
0,95	$3/n$
0,99	$4,6/n$

Para una probabilidad de disfunción bajo demanda de SIL3 con un nivel de confianza del 95%, la aplicación de la fórmula da 30 000 casos de ensayo teniendo en cuenta las condiciones previas. La tabla D.1 resume los resultados para cada nivel de integridad de seguridad.

D.2.2 Ensayo de un espacio de entrada (dominio) para un modo de funcionamiento de baja demanda

D.2.2.1 Requisitos previos

La única condición previa es que los datos de ensayo se seleccionen para dar una distribución aleatoria uniforme sobre todo el espacio de entrada (dominio).

D.2.2.2 Resultados

El objetivo es encontrar el número n de ensayos que son necesarios, basándose en el umbral de precisión δ de las entradas para la función de baja demanda a ensayar (por ejemplo, una parada de emergencia de seguridad).

Tabla D.3 – Distancias medias de dos puntos de ensayo

Dimensión del dominio	Distancia media de dos puntos de ensayo en dirección de un eje arbitrario
1	$\delta = 1/n$
2	$\delta = \sqrt[2]{1/n}$
3	$\delta = \sqrt[3]{1/n}$
k	$\delta = \sqrt[k]{1/n}$
NOTA k puede ser cualquier número entero positivo. Los valores 1, 2 y 3 son solo ejemplos.	

D.2.2.3 Ejemplo

Consideremos una parada de emergencia de seguridad que depende solo de dos variables, A y B. Si se ha verificado que los umbrales que dividen el par de entrada de las variables A y B se tratan correctamente con una precisión del 1% del rango de medida de A o de B, el número de casos de ensayo uniformemente distribuidos requerido en el espacio de A y B es

$$n = 1/\delta^2 = 10^4$$

D.2.3 Ensayo estadístico simple en modo de funcionamiento de alta demanda o continuo**D.2.3.1 Requisitos previos**

- La distribución de los datos de ensayo es igual a la distribución durante la operación en línea.
- La reducción relativa de la probabilidad de ausencia de disfunción es proporcional a la longitud del intervalo de tiempo considerado y constante en otro caso.
- Existe un mecanismo adecuado para detectar cualquier disfunción que se pueda producir.
- El ensayo dura un tiempo t .
- No se produce ninguna disfunción durante el tiempo t .

D.2.3.2 Resultados

La relación entre la probabilidad de disfunción λ , el nivel de confianza $1 - \alpha$ y el tiempo de ensayo t es

$$\lambda = -\frac{\ln \alpha}{t}$$

La probabilidad de disfunción es inversamente proporcional al tiempo medio de funcionamiento entre disfunciones:

$$\lambda = \frac{1}{MTBF}$$

NOTA Esta norma no hace distinción entre la probabilidad de disfunción por hora y la tasa de disfunción en 1 h. Estrictamente, la probabilidad de disfunción F , se relaciona con la tasa de disfunción f , por la ecuación $F = 1 - e^{-ft}$, pero el campo de aplicación de esta norma es para tasas de disfunción de menos de 10^{-5} y para estos valores tan pequeños $F \approx ft$.

D.2.3.3 Ejemplo

Tabla D.4 – Probabilidad de disfunción en modo de funcionamiento de alta demanda o continuo

$1 - \alpha$	λ
0,95	$3/t$
0,99	$4,6/t$

Para verificar que el tiempo medio entre disfunciones es al menos de 10^8 h con un nivel de confianza del 95%, se necesita un tiempo de ensayo de 3×10^8 h y cumplir los requisitos previos. La tabla D.1 resume el número de ensayos requeridos para cada nivel de integridad de seguridad.

D.2.4 Ensayo completo

El programa se considera como una urna que contiene un número N conocido de bolas. Cada bola representa una propiedad de interés del programa. Las bolas son sacadas al azar y se reponen después de la inspección. El ensayo completo se realiza cuando todas las bolas se han sacado.

D.2.4.1 Requisitos previos

- La distribución de los datos de ensayo es tal que cada una de las N propiedades del programa sea ensayada con una misma probabilidad.
- Las ejecuciones del ensayo son independientes.
- Cada disfunción que se produce se detecta.
- El número de casos de ensayo $n \gg N$.
- No se produce ninguna disfunción durante los n casos de ensayo.
- Cada ejecución de ensayo se hace sobre una propiedad del programa (una propiedad del programa es la que se puede ensayar durante la ejecución del ensayo).

D.2.4.2 Resultados

La probabilidad p de ensayar todas las propiedades del programa viene dada por

$$p = \sum_{j=0}^{N-1} (-1)^j \binom{N}{j} \left(\frac{N-j}{N} \right)^n \quad \text{o} \quad p = 1 + \sum_{j=1}^N (-1)^j C_{j,N} \left(\frac{N-j}{N} \right)^n$$

donde

$$C_{j,N} = \frac{N(N-1) \dots (N-j+1)}{j!}$$

En lo referente a la evaluación de esta fórmula, en general solo los primeros términos son importantes ya que los casos realistas se caracterizan por $n \gg N$. El último factor hace muy pequeños los términos correspondientes a j grandes. Esto también es visible en la tabla D.5.

D.2.4.3 Ejemplo

Consideremos un programa que se ha utilizado en varias instalaciones durante varios años. En total se ha ejecutado al menos $7,5 \times 10^6$ veces. Se estima que cada cien ejecuciones cumple los requisitos previos. Entonces, para la evaluación estadística se pueden considerar $7,5 \times 10^4$ ejecuciones realizadas. Se estima que 4 000 ejecuciones de ensayo corresponden a un ensayo exhaustivo. Las estimaciones son conservadoras. De acuerdo con la tabla D.5, la probabilidad de no haber ensayado todo es igual a $2,87 \times 10^{-5}$.

Para $N = 4\,000$, los valores de los primeros términos que dependen de n son los siguientes:

Tabla D.5 – Probabilidad de ensayo de todas las propiedades del programa

n	p
5×10^4	$1 - 1,49 \times 10^{-2} + 1,10 \times 10^{-4} - \dots$
$7,5 \times 10^4$	$1 - 2,87 \times 10^{-5} + 4 \times 10^{-10} - \dots$
1×10^5	$1 - 5,54 \times 10^{-8} + 1,52 \times 10^{-15} - \dots$
2×10^5	$1 - 7,67 \times 10^{-19} + 2,9 \times 10^{-37} - \dots$

En la práctica, estas estimaciones deberían hacerse de forma que sean conservadoras.

D.3 Referencias

Informaciones más amplias sobre las técnicas anteriores se pueden encontrar en los documentos siguientes:

IEC 61164:2004 *Crecimiento de la fiabilidad. Ensayos estadísticos y métodos de estimación*.

Verification and Validation of Real-Time Software, Chapter 5. W. J. Quirk (ed.). Springer Verlag, 1985, ISBN 3-540-15102-8.

Combining Probabilistic and Deterministic Verification Efforts. W. D. Ehrenberger, SAFECOMP 92, Pergamon Press, ISBN 0-08-041893-7.

Ingenieurstatistik. Heinhold/Gaede, Oldenburg, 1972, ISBN 3-486-31743-1.

IEEE 352:1987, *IEEE Guide for general principles of reliability analysis of nuclear power generating station safety systems*.

ANEXO E (Informativo)

PRESENTACIÓN DE TÉCNICAS Y MEDIDAS PARA EL DISEÑO DE ASIC

NOTA Presentación de técnicas y medidas contenida en este anexo y mencionada en la Norma IEC 61508-2. Este anexo no debería considerarse ni completo ni exhaustivo.

E.1 Descripción del diseño en (V)HDL

Objetivo: Descripción funcional en alto nivel en un lenguaje de descripción de hardware , por ejemplo VHDL o Verilog.

Descripción: Descripción funcional en alto nivel de abstracción en un lenguaje de descripción de hardware, por ejemplo, VHDL o Verilog. El lenguaje de descripción de hardware aplicado debería permitir la descripción funcional y/o la descripción orientada a la aplicación y debería abstraerse de detalles de implementación posteriores. Los flujos de datos, ramificaciones, operaciones aritméticas y lógicas deberían implementarse mediante asignación y operadores del lenguaje de descripción de hardware, sin conversión en puertas lógicas de la biblioteca aplicada.

NOTA Para simplificar, “descripción funcional en alto nivel de abstracción en un lenguaje de descripción de hardware” será denotado en el resto del documento como (V)HDL.

Referencia:

IEEE VHDL, *Verilog* + *Standard VHDL Design guide*.

E.2 Entrada esquemática

Objetivo: Descripción funcional de la circuitería mediante el dibujo del plan del circuito utilizando puertas y/o macros de la biblioteca del vendedor.

Descripción: Descripción de la funcionalidad del circuito mediante la entrada esquemática del plan de circuito. La función a realizar debería implementarse mediante la instanciación (importación) de los elementos del circuito lógico elementales tales como AND, OR, NOT junto con las macros que consistan en funciones aritméticas y lógicas complejas, que se interconectan después. Deberían dividirse los circuitos complejos, considerando los puntos de vista funcionales y pueden distribuirse en diferentes dibujos, que están jerárquicamente interconectados. Las señales de interconexión deberían estar definidas unívocamente y tener nombres explícitos de señal a lo largo de toda la jerarquía. Debería evitarse la utilización de señales globales (etiquetas) en la medida de lo posible.

E.3 Descripción estructurada

NOTA Véase también el capítulo C.2.7 “Programación Estructurada” y el capítulo E.12 “Modularización”.

Objetivo: La descripción de la funcionalidad del circuito debería estructurarse en un estilo que sea fácilmente legible, es decir, la función del circuito puede entenderse intuitivamente en base a la descripción sin esfuerzos de simulación.

Descripción: Descripción de la funcionalidad del circuito con (V)HDL o mediante entrada esquemática. Se recomienda una estructura fácilmente reconocible y modular. Se debería implementar cada módulo con el mismo estilo y debería describirse de tal manera que sea fácilmente legible con sub funciones definidas claras. Se recomienda una estricta distinción entre la función implementada y la interconexión, es decir, el módulo, que se implementa mediante la instanciación de otros sub módulos, contiene explícitamente interconexiones de los módulos instanciados y no debería contener ningún circuito lógico.

E.4 Herramientas probadas en uso

Objetivo: Aplicación de herramientas probadas en uso para evitar disfunciones sistemáticas mediante una larga práctica de utilización de las herramientas en varios proyectos.

Descripción: La mayoría de la herramientas utilizadas para el diseño de ASIC y FPGA están formadas por software sofisticado, que no puede considerarse que funcione sin ningún fallo en relación a su correcta funcionalidad y que debido a una operación anómala muy probablemente también puede producir fallos. En consecuencia, sólo deberían preferirse las herramientas con el atributo “probada en uso” para diseñar ASIC y FPGA. Esto implica:

- La aplicación de herramientas que han sido utilizadas (en una versión de software comparable) durante un largo periodo de tiempo o por un alto número de usuarios en varios proyectos de complejidad equivalente.
- Una adecuada experiencia de cada diseñador de ASIC/FPGA con la operación de la herramienta durante un largo periodo de tiempo.
- La utilización de herramientas comúnmente utilizadas (adecuado número de usuarios) de manera que la información relativa a las disfunciones conocidas con sus soluciones (control de versión con “lista de disfunciones”) esté disponible.
- El chequeo de la consistencia de la base de datos interna de la herramienta y el chequeo de la verosimilitud para evitar datos de salida anómalos. Las herramientas estándar chequean la consistencia de la base de datos interna, por ejemplo, la consistencia de la base de datos entre la síntesis y la herramienta de ubicación y enrutado, con el fin de operar con datos únicos.

NOTA El chequeo de la consistencia es un atributo inherente de la herramienta utilizada y el diseñador tiene una influencia limitada sobre él. En consecuencia, si se suministra la posibilidad de chequeo manual de la consistencia, el diseñador debería utilizarla adecuadamente.

E.5 Simulación (V)HDL

NOTA Véase también el capítulo E.6 “Ensayo funcional a nivel de módulo”.

Objetivo: Verificación funcional del circuito descrito en (V)HDL mediante simulación.

Descripción: Verificación de la función mediante la simulación del circuito completo o de cada sub módulo. El simulador (V)HDL detecta una secuencia de salidas provocada por un cambio interno de los estados del circuito como resultado de los estímulos de entrada aplicados. La verificación de la secuencia de salida detectada puede realizarse o bien mediante una secuencia pretrazada de las señales de salida (“forma de onda”) o bien mediante un entorno especial conocido como banco de ensayo, que se instala durante el proceso de diseño. El simulador elegido debería tener el atributo de “probado en uso” con el fin de suministrar resultados correctos y enmascarar el comportamiento anómalo de temporizado de las señales (picos, trazado del tri-estado) que pudiera ser provocado por el propio simulador o un modelado anómalo.

E.6 Ensayo funcional a nivel de módulo

NOTA Véanse también los capítulos E.5 “Simulación (V)HDL” y E.13 “Cobertura de los escenarios de verificación”.

Objetivo: Verificación funcional “de abajo hacia arriba”.

Descripción: Verificación de la función implementada – por ejemplo mediante simulación – a nivel de módulo. El módulo sometido a ensayo será instanciado en un entorno de ensayo virtual típico conocido como “banco de ensayo” y estimulado mediante un patrón de ensayo contenido en el código. Se exige al menos una cobertura suficientemente alta de la función especificada, incluyendo todos los casos especiales, si existen. Debería preferirse la verificación automática de la secuencia de salida mediante el código del “banco de ensayo”, en vez de la inspección manual de señales de salida.

E.7 Ensayo funcional al máximo nivel

NOTA Véase también el capítulo E.8 “Ensayo funcional embebido en el entorno del sistema”.

Objetivo: Verificación del ASIC (circuito completo).

Descripción: El objetivo del ensayo es la verificación del circuito completo (ASIC).

E.8 Ensayo funcional embebido en el entorno del sistema

NOTA Véase también el capítulo E.7 “Ensayo funcional al máximo nivel”.

Objetivo: Verificación de la función especificada embebida en el entorno del sistema.

Descripción: El ensayo verificará la funcionalidad completa del circuito (ASIC) en su entorno de sistema, por ejemplo con todos los componentes que están en las placas de circuito o en cualquier otra parte. Se recomienda un modelado de todos los componentes pertinentes de la placa de circuito y la simulación del ASIC junto con el modelo creado para verificar la correcta funcionalidad incluyendo el comportamiento temporal. El ensayo funcional completo también incluye el ensayo de los módulos que están activados sólo durante la presencia de una disfunción.

E.9 Utilización restringida de construcciones asíncronas

Objetivo: Evitar los típicos problemas de temporizado durante la síntesis, evitar la ambigüedad durante la simulación y la síntesis provocada por un insuficiente modelado, diseño para la capacidad de ensayo.

Descripción: Las construcciones asíncronas tales como las señales SET y RESET derivadas de lógica combinatoria son delicadas durante la síntesis y producen circuitos con picos o invierten la secuencia de temporizado y en consecuencia deberían evitarse. Un modelado insuficiente puede no ser interpretado adecuadamente por la herramienta de síntesis, que provoca resultados ambiguos durante la simulación. Adicionalmente las instrucciones asíncronas son poco ensayables o nada ensayables, de manera que la cobertura del ensayo de producción y del ensayo en línea se reduce de manera efectiva. Se recomienda en consecuencia la implementación de un diseño completamente síncrono con un número limitado de señales de reloj. En sistemas con relojes multifase, todos los relojes se deberían derivar de un reloj central. La entrada de reloj de la lógica secuencial debería ser siempre suministrada exclusivamente por la señal de reloj, que no contenga ninguna lógica combinatoria. Las entradas asíncronas SET y RESET de las celdas secuenciales deberían ser siempre suministradas mediante señales síncronas que no contengan ninguna lógica combinatoria. Las señales SET y RESET maestras deberían estar sincronizadas utilizando dos biestables.

E.10 Sincronización de las entradas primarias y control de meta-estabilidades

Objetivo: Evitar el comportamiento ambiguo del circuito como resultado de la violación de la temporización de establecimiento y de mantenimiento.

Descripción: Las señales de entrada de los periféricos externos son generalmente asíncronas y pueden cambiar sus estados arbitrariamente. Un procesamiento directo de tales señales mediante los elementos secuenciales síncronos del circuito del ASIC/FPGA, por ejemplo los biestables, conduce a la violación del tiempo de establecimiento y de mantenimiento, dando como resultado un temporizado y un comportamiento funcional del ASIC/FPGA impredecibles. Al final podría producirse la meta estabilidad del elemento de memoria. Cada señal de entrada asíncrona debería sincronizarse con el circuito ASIC síncrono para evitar la ambigüedad funcional. Se recomiendan las siguientes medidas:

- Las señales de entrada deberían sincronizarse con dos elementos de memoria consecutivos (biestables) o algún circuito equivalente con el fin de lograr un comportamiento funcional predecible.

- Cada señal de entrada asíncrona debería estar fundamentalmente sincronizada de la manera definida anteriormente, es decir, cada señal asíncrona se conecta con exactamente uno de tales circuitos de sincronización. Si es necesario, la salida del circuito de sincronización puede utilizarse para el acceso múltiple.
- El circuito de sincronización debería utilizarse para el ensayo de estabilidad de las señales del bus paralelo y para el control de la consistencia de datos cerca del punto de muestreo.

E.11 Diseño para la capacidad de ensayo

NOTA 1 Véase también el capítulo E.31 “Implementación de las estructuras de ensayo”.

Objetivo: Evitar las estructuras no ensayables o poco ensayables con el fin de lograr una alta cobertura de ensayo para el ensayo de producción o el ensayo en línea.

Descripción: El diseño para la capacidad de ensayo está dirigido a evitar:

- las construcciones asíncronas;
- los latches y las señales tri-estado en el chip;
- la lógica cableada y la lógica redundante.

La profundidad combinatoria de los sub circuitos juega un papel muy importante durante el ensayo. El patrón de ensayo exigido para un ensayo completo crece exponencialmente con la profundidad combinatoria del circuito. En consecuencia, los circuitos con mucha profundidad combinatoria son poco ensayables con los medios adecuados.

Un diseño para una aproximación orientada a la capacidad de ensayo asegura que la cobertura de ensayo deseada se logra. Puesto que la cobertura real de ensayo puede determinarse en un etapa muy tardía del proceso de diseño, una insuficiente consideración de los temas relativos al “diseño para la capacidad de ensayo” podría reducir dramáticamente la cobertura alcanzable de ensayo, conduciendo a un esfuerzo adicional.

NOTA 2 La cobertura de ensayo se determina normalmente mediante el porcentaje de fallos persistentes detectados.

E.12 Modularización

NOTA Véanse también el apartado C.2.8 “Ocultación/encapsulado de la información”, el apartado C.2.9 “Aproximación modular” y el capítulo E.3 “Descripción estructurada”.

Objetivo: Descripción modular de las funciones el circuito.

Descripción: Distinguir particiones de la funcionalidad total en diferentes módulos con funciones limitadas. Se establece así la transparencia de los módulos con la interfaz definida con precisión. Cada subsistema, en todos los niveles del diseño, está claramente definido y es de tamaño restringido (sólo unas pocas funciones). Las interfaces entre los subsistemas se mantienen tan sencillas como sea posible y se minimiza la sección transversal (es decir, los datos compartidos, el intercambio de información). La complejidad de los subsistemas individuales también se restringe.

E.13 Cobertura de los escenarios de verificación (bancos de ensayo)

Objetivo: Evaluación cuantitativa y cualitativa de los escenarios de verificación aplicados durante el ensayo funcional.

Descripción: Se debería documentar cuantitativa y/o cualitativamente la calidad de los escenarios de verificación que se define durante el ensayo funcional, es decir, el patrón de ensayo aplicado (estímulos) para verificar la función especificada incluyendo todos los casos especiales, si existen. Durante una aproximación cuantitativa, la cobertura de ensayo lograda y la profundidad de los ensayos funcionales aplicados, deberían documentarse. La cobertura resultante debería cumplir los niveles establecidos para cada métrica de cobertura. Toda excepción será documentada. En caso de una aproximación cualitativa, se debería estimar el número de líneas de código, las instrucciones o caminos verificados (“cobertura de código”) del código del circuito a ser verificado.

NOTA El análisis exclusivo de “cobertura de código” sólo tiene una relevancia limitada, debido al alto paralelismo de la descripción del hardware, y se justificará mediante chequeos exhaustivos. La “cobertura de código” sirve generalmente para demostrar el código funcional no cubierto.

E.14 Observación de las directrices de codificación

Objetivo: Observación estricta de los resultados del estilo de codificación en un código de circuito sintáctico y semántico correcto.

Descripción: Las reglas sintácticas de codificación ayudan a crear un código fácilmente legible y permiten una mejor documentación incluyendo el control de versión. Típicamente, pueden mencionarse aquí las reglas para organizar y comentar los bloques de instrucciones o módulos.

Las reglas semánticas de codificación ayudan a evitar los típicos problemas de implementación evitando las instrucciones que conducen a una síntesis anómala con ambigüedades de implementación de la función del circuito. Las reglas típicas son por ejemplo, evitar las construcciones asíncronas o construcciones que producen una secuencia de temporizado impredecible. En general, la utilización de latches o el acoplamiento de datos con señales de reloj conducen a tales ambigüedades.

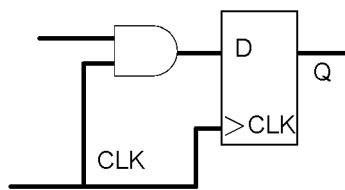
Se recomiendan directrices de diseño para evitar las disfunciones sistemáticas de diseño durante el proceso de desarrollo del ASIC. Un estilo de codificación en cierto modo limita la eficacia del diseño, pero sin embargo ofrece la ventaja de evitar disfunciones durante el proceso de desarrollo del ASIC. En particular:

- evitar la típica falta de consistencia o disfunción de codificación;
- utilización restringida de construcciones problemáticas que producen resultados de síntesis ambiguos;
- diseño para la capacidad de ensayo;
- código transparente y fácil de utilizar.

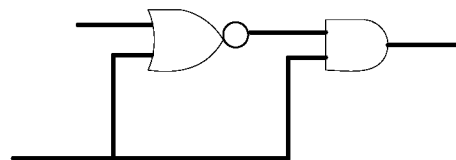
Ejemplo de Estilo de Codificación

- 1 El código debería contener cuantos comentarios sean necesarios para entender los detalles de la función e implementación. Las convenciones utilizadas tienen que definirse antes de empezar el diseño. El cumplimiento de las convenciones debería chequearse durante la fase de diseño.
 - 1.1 Encabezados normalizados que incluyan la historia, referencias cruzadas a la especificación, datos de acompañamiento de responsabilidad y diseño tales como número de versión, solicitud de cambio, etc.
 - 1.2 Plantillas fácilmente legibles: los procesos equivalentes deberían describirse con el mismo procedimiento, es decir, utilización de plantillas predefinidas para procesos recurrentes (si-entonces-en otro caso, para, etc.).
 - 1.3 Convención de nombrado precisa y legible, es decir, letra mayúscula/minúscula, prefijos y sufijos, diferenciación precisa entre nombre de los puertos, señales internas, constantes, variables, nivel activo bajo (xxx_n), etc.
 - 1.4 El tamaño del módulo y el número de puertos por módulo deberían limitarse para aumentar la legibilidad del código.

- 1.5 Desarrollo de código estructural y defensivo, por ejemplo, la información de estado debería estar encapsulada en FSM (ocultación de la información) para suministrar una fácil alteración del código.
- 1.6 Se deberían implementar chequeos de verosimilitud tales como el chequeo de rangos, etc.
- 1.7 Evitar las siguientes construcciones/instrucciones:
 - uso de rango ascendente (x a y) para señales de bus;
 - instrucción “disable” en Verilog (corresponde a la instrucción goto);
 - matrices multidimensionales (>2), registros;
 - combinación de tipos de datos con y sin signo.
- 2 Diseño síncrono completo (se permiten los relojes derivados a partir de los relojes centrales)
 - 2.1 Las salidas de los módulos deberían estar sincronizadas, esto también apoya la capacidad de ensayo y el análisis estático de temporizado.
 - 2.2 Relojes con puerta deberían manejarse con especial precaución.
- 3 Evitar el acoplamiento de datos con el reloj aumenta la capacidad de ensayo, la reproducibilidad entre los datos de pre y post trazado y el cumplimiento del comportamiento RTL (nivel de transferencia de registro).
- 4 La lógica redundante se debería evitar pues no es ensayable:



Acoplamiento de señales de datos y de reloj



Lógica redundante

- 5 Los bucles de realimentación en lógica combinatoria se deberían evitar puesto que producen un diseño inestable y no serán ensayables.
- 6 Se recomienda el diseño de barrido completo.
- 7 Evitar los latches aumenta la capacidad de ensayo y reduce las limitaciones de temporizado durante la síntesis.
- 8 El reset maestro y todas las entradas asíncronas deberían sincronizarse con dos elementos de memoria consecutivos (biestables) o circuito equivalente (meta-estabilidad).
- 9 Se recomienda evitar el set/reset asíncrono excepto para el reset maestro.
- 10 Las señales a nivel de puerto del módulo deberían ser del tipo std_logic o std_logic_vector.

E.15 Aplicación de la herramienta de revisión de código

Objetivo: Verificación automática de las reglas de codificación (“Estilo de codificación”) mediante una herramienta de chequeo del código.

Descripción: La aplicación de la herramienta de revisión del código ayuda en gran medida a respetar el estilo de codificación y a generar la documentación en línea. Sin embargo, la herramienta automática de revisión de código puede generalmente verificar la sintaxis y semántica del código. La aplicación de tales herramientas debería estar, en consecuencia, acompañada por una extensión de las reglas generales de codificación (“específica de la herramienta”) con las reglas de codificación específicas del proyecto que el diseñador tiene que implementar y evaluar separadamente.

E.16 Programación defensiva

Véase el apartado C.2.5.

E.17 Documentación de los resultados de simulación

Objetivo: Documentación de todos los datos necesarios para una simulación exitosa con el fin de verificar la función específica del circuito.

Descripción: Se deberían documentar bien y archivar todos los datos necesarios para la simulación funcional a nivel de módulo, chip o sistema, con los siguientes propósitos:

- Repetir la simulación en alguna etapa posterior con estilo llave en mano.
- Demostrar la corrección y completitud de todas las funciones especificadas.

Se debería archivar la siguiente base de datos con dicho propósito:

- Configuración de la simulación incluyendo el software completo de las herramientas aplicadas, por ejemplo simulador, sintetizador con su correspondiente versión y la biblioteca de simulación necesaria.
- Fichero de registro de la simulación con detalles completos relativos al tiempo de simulación, herramientas aplicadas con su versión e informe completo de las soluciones de los problemas si fuera necesario.
- Todos los resultados pertinentes de la simulación incluso el flujo de señal, especialmente en caso de inspección manual y la documentación de los resultados adquiridos.

E.18 Inspección del código

NOTA 1 Véase también el apartado C.5.14 “Inspecciones formales”.

Objetivo: Revisar la descripción del circuito.

Descripción: La revisión de la descripción del circuito debería realizarse mediante:

- Chequeo del estilo de codificación.
- Verificación de la funcionalidad descrita contra la especificación.
- Chequeo de la codificación defensiva, manejo de errores y excepciones.

NOTA 2 Si no se realiza la simulación (V)HDL, la completitud de la inspección del código y los resultados obtenidos deberían tener una calidad equivalente a la que se obtendría mediante la simulación (V)HDL.

E.19 Sondeo

NOTA 1 Véase también el apartado C.5.15 “Sondeo”.

Objetivo: Revisar la descripción del circuito mediante sondeo.

Descripción: El sondeo del código consiste en un equipo humano que selecciona un grupo pequeño de casos de ensayo, grupos representativos de entradas y las correspondientes salidas esperadas para el programa. Los datos de ensayo se siguen a continuación de manera manual a través de la lógica del programa.

NOTA 2 Como medida aislada se debería aplicar sólo a circuitos de muy baja complejidad. En el caso de que falle la simulación (V)HDL, la completitud de los sondeos y la calidad de los resultados obtenidos deberían tener una calidad equivalente a la que se obtendría mediante simulación (V)HDL.

Referencia:

IEC 61160:2005 *Revisión de diseño*.

E.20 Aplicación de funciones lógicas validadas

Objetivo: Evitar la disfunción durante la operación de las funciones lógicas mediante la aplicación de funciones lógicas validadas.

Descripción: Si el vendedor valida las funciones lógicas, se deberían cumplir los siguientes requisitos:

- La validación de las funciones lógicas debería realizarse para la operación del sistema relacionado con la seguridad, y debería tener al menos un nivel de integridad de seguridad equivalente o mayor al del sistema en planificación.
- Deberían cumplirse todas las hipótesis y restricciones que sean necesarias para la validación de las funciones lógicas.
- Deberían estar fácilmente disponibles todos los documentos necesarios para la validación de las funciones lógicas, véase también el capítulo E.17 “Documentación de los resultados de simulación”.
- La especificación del vendedor debería respetarse estrictamente y debería documentarse la evidencia de su cumplimiento.

E.21 Validación de la función lógica

NOTA Véase también el capítulo E.6 “Ensayo funcional a nivel de módulo”

Objetivo: Evitar la disfunción durante la operación de la función lógica mediante la validación de la misma durante el ciclo de vida del diseño.

Descripción: Si la función lógica no está explícitamente desarrollada para la operación en un sistema relacionado con la seguridad, el código generado debería validarse con las mismas premisas que se aplican para la validación de cualquier código fuente. Esto significa que se deberían definir e implementar todos los posibles casos de ensayo. La verificación funcional debería derivarse de la simulación posterior.

E.22 Simulación de la lista de red de puertas para chequear las limitaciones de temporizado

Objetivo: Verificación independiente de la limitación de temporizado alcanzada durante la síntesis.

Descripción: Simulación de la lista de red de puertas generada por la síntesis que incluye la retro- anotación de los retardos de línea y los retardos de puerta. Los estímulos deberían derivarse para estimular el circuito de forma que se cubra un alto porcentaje de limitaciones de temporizado y se incluyan todos los caminos de temporizado del peor caso. En general, los estímulos necesarios para realizar el capítulo E.6 “Ensayo funcional a nivel de módulo” o el capítulo E.7 “Ensayo funcional” suministran unos criterios adecuados para la selección de los estímulos, siempre que pueda declararse la suficiente cobertura de ensayo durante el ensayo funcional. El circuito debería ensayarse en las condiciones del mejor y del peor caso a la máxima frecuencia de reloj especificada.

La verificación del temporizado puede realizarse mediante un chequeo automático del tiempo de establecimiento y de mantenimiento de los elementos de memoria (biestables) de la biblioteca objetivo así como mediante la verificación funcional del circuito. La verificación funcional debería realizarse principalmente mediante la observación de las salidas del chip. Esto puede automatizarse mediante la comparación de las señales de salida del circuito con un modelo de referencia adecuado o con el código fuente (V)HDL del circuito. Este ensayo se conoce como un “ensayo de regresión” y debería preferirse al chequeo manual de las señales de salida.

NOTA Aplicando esta medida, sólo puede verificarse el comportamiento de temporizado de aquellos caminos que son realmente estimulados durante la simulación y en consecuencia, la medida diseñada especialmente no puede suministrar un análisis completo del temporizado del circuito en general.

E.23 Análisis estático del retardo de propagación (STA)

Objetivo: Verificación independiente de la limitación de temporizado realizada durante la síntesis.

Descripción: El Análisis Estático de Temporizado (STA) analiza todos los caminos de una lista de red (circuito) generada mediante la herramienta de síntesis considerando la retro-anotación, es decir, los retardos de línea estimados mediante la herramienta de síntesis, así como los retardos de puerta sin realizar la simulación real. En consecuencia, permite en general, un análisis completo de la limitación de temporizado del circuito entero. El circuito a ser ensayado debería analizarse en las condiciones del mejor y del peor caso funcionando a la máxima frecuencia de reloj especificada y contabilizando el jitter de reloj aplicable y el desvío del ciclo de trabajo. El número de caminos de temporizado no pertinentes puede limitarse a un cierto mínimo mediante la adopción de una técnica de diseño adecuada. Se recomienda investigar, analizar y definir la técnica utilizada que permita resultados fácilmente legibles antes de empezar con el diseño.

NOTA Puede asumirse que el STA cubre explícitamente todos los caminos de temporizado existente si:

- a) Las limitaciones de temporizado están especificadas adecuadamente.
- b) El circuito sometido a ensayo contiene sólo caminos de temporizado que pueden analizarse mediante herramientas STA, es decir, generalmente el caso de circuitos completamente síncronos.

E.24 Verificación de la lista de red de puertas contra el modelo de referencia mediante simulación

Objetivo: Chequear la equivalencia funcional de la lista de red de puertas sintetizada.

Descripción: Simulación de la lista de red de puertas generada mediante la herramienta de síntesis. Los estímulos aplicados para la verificación del circuito mediante simulación corresponden exactamente a los estímulos aplicados durante el capítulo E.6 “Ensayo funcional a nivel de módulo” y el capítulo E.7 “Ensayo funcional a máximo nivel” para la verificación de la función a nivel de módulo y a máximo nivel respectivamente. La verificación funcional debería realizarse principalmente mediante la observación de las salidas del chip. Esto puede ser automatizado mediante la comparación de las señales de salida del circuito con el modelo de referencia adecuado o con el código fuente (V)HDL del circuito. Este ensayo se conoce como “ensayo de regresión” y debería preferirse al chequeo manual de las señales de salida.

NOTA Aplicando esta medida, sólo puede verificarse el comportamiento funcional de aquellos caminos que son realmente estimulados durante la simulación. La cobertura de ensayo, en consecuencia, sólo puede ser tan buena como durante el ensayo funcional original a nivel de módulo o a máximo nivel, respectivamente. Es posible complementar esta medida con un ensayo de equivalencia formal. En todos los casos debería realizarse una verificación funcional del código fuente (V)HDL con la lista de red final generada por la herramienta de síntesis.

E.25 Comparación de la lista de red de puertas con el modelo de referencia (ensayo formal de equivalencia)

Objetivo: Chequear que la equivalencia funcional que es independiente de la simulación.

Descripción: Comparación de la funcionalidad del circuito descrita mediante el código fuente (V)HDL con la funcionalidad del circuito de la lista de red de puertas generada mediante síntesis. Las herramientas basadas en el principio de equivalencia formal son capaces de verificar la equivalencia funcional de una forma de representación diferente del circuito, por ejemplo la descripción (V)HDL o la descripción de la lista de red. Mediante la aplicación de esta medida no es necesaria una simulación funcional y es viable un chequeo funcional independiente. La aplicación exitosa de esta medida sólo puede garantizarse si la herramienta aplicada es capaz de probar una equivalencia completa y todas las discrepancias indicadas se evalúan mediante inspección manual o automáticamente.

NOTA Es ventajoso combinar esta medida con el capítulo E.24 “Verificación de la lista de red de puertas contra el modelo de referencia mediante simulación”. En todos los casos debería realizarse una verificación funcional del código fuente (V)HDL con la lista de red final generada por la herramienta de síntesis.

E.26 Chequeo de los requisitos y limitaciones del vendedor

Objetivo: Evitar la disfunción durante la producción mediante el chequeo de los requisitos del vendedor.

Descripción: Un cuidadoso chequeo de los requisitos y limitaciones del vendedor, por ejemplo fan-in y fan-out máximos y mínimos, máxima longitud de cableado (retardo de línea), tasa máxima de variación rápida de las señales, desviación del reloj y similares, mediante la herramienta de síntesis mejora la fiabilidad del producto. Además de la importancia de los requisitos para el proceso de producción, su violación tiene un gran impacto en la validez de los modelos aplicados que se utilizan para la simulación. Así, cualquier violación de los requisitos y limitaciones del vendedor conduce a resultados de simulación anómalos que provocan una funcionalidad no deseada.

E.27 Documentación de las limitaciones, resultados y herramientas de síntesis

Objetivo: Documentación de todas las limitaciones definidas que son necesarias para una síntesis óptima que genere la lista final de red de puertas.

Descripción: La documentación de todas las limitaciones y resultados de la síntesis es indispensable debido a las siguientes razones:

- reproducir la síntesis en cualquier etapa posterior;
- generar unos resultados de síntesis independientes para la verificación.

Los documentos esenciales son:

- Configuración de la síntesis incluyendo las herramientas y software de síntesis aplicados con la versión real, la biblioteca de síntesis aplicada y las limitaciones y scripts definidos.
- Fichero de registro de síntesis con los comentarios de tiempo, herramienta aplicada con versión y la documentación completa de la síntesis.
- La lista de red generada con retardos de tiempo estimados (Fichero normalizado (SDF) de formato de retardo).

E.28 Aplicación de herramienta de síntesis probada en uso

Objetivo: Herramienta basada en la conversión de la descripción (V)HDL de un circuito en la lista de red de puertas.

Descripción: Asignación realizada mediante una herramienta del código fuente (V)HDL de la funcionalidad del circuito mediante la conexión de las puertas adecuadas y de las primitivas del circuito de la biblioteca objetivo de ASIC. La implementación seleccionada de toda la variedad posible de implementaciones que cumple la funcionalidad deseada depende del resultado más óptimo que se derive mediante las limitaciones de síntesis tales como el temporizado (frecuencia de reloj) y área del chip.

E.29 Aplicación de biblioteca objetivo probada en uso

NOTA Véase también el capítulo E.4 “Herramientas probadas en uso”.

Objetivo: Evitar las disfunciones sistemáticas causados por una biblioteca objetivo defectuosa.

Descripción: Las bibliotecas objetivo de síntesis y simulación para el desarrollo de un ASIC se derivan de una base de datos común y en consecuencia no son independientes. Una disfunción sistemática tal como:

- ambigüedad entre el comportamiento real y el modelado de los elementos del circuito;
- modelado insuficiente por ejemplo del tiempo de establecimiento y de mantenimiento;

son ejemplos típicos.

En consecuencia, sólo las tecnologías y las bibliotecas objetivo probadas en uso deberían utilizarse para el diseño de ASIC que realizan funciones de seguridad. Esto significa:

- La aplicación de bibliotecas objetivo que han sido utilizadas a lo largo de un tiempo significativo en proyectos comparables en complejidad y frecuencia de reloj.
- Disponibilidad de la tecnología y biblioteca objetivo correspondiente a lo largo de un periodo de tiempo suficientemente largo de manera que pueda esperarse suficiente precisión de modelado de la biblioteca.

E.30 Procedimientos basados en script

Objetivo: Reproducibilidad de resultados y automatización de ciclos de síntesis.

Descripción: Control automático y basado en script de los ciclos de síntesis que incluyen la definición de las limitaciones aplicadas. Además de una documentación precisa de una limitación completa de síntesis, ayuda a reproducir la lista de red tras la alteración del código fuente (V)HDL en idénticas condiciones.

E.31 Implementación de las estructuras de ensayo

Objetivo: Diseño de ASIC que garantice el ensayo final de producción.

Descripción: El diseño para la capacidad de ensayo permite circuitos fácilmente ensayables mediante la implementación de diferentes estructuras de ensayo, por ejemplo:

- Camino de barrido: en una técnica de barrido, de todos (diseño de barrido completo) o parte de los biestables (diseño de barrido parcial) que se conectan en una cadena sencilla o en múltiples cadenas construyendo una cadena de registros de desplazamiento. El camino de barrido permite una generación automática del patrón de ensayo de la lógica entera de un circuito. La herramienta que genera el patrón se llama ATPG “Generador Automático de Patrón de Ensayo”. La implementación del camino de barrido mejora tremendamente la capacidad de ensayo de un circuito y permite más del 98% de cobertura de ensayo con un esfuerzo razonable. En consecuencia, se recomienda implementar un camino de barrido completo, si es posible.
- Árbol- NAND: En un árbol NAND, todas las entradas primarias de un circuito se conectan en cascada para construir una cadena. Aplicando un patrón de ensayo adecuado (“bit deslizante”) es posible ensayar el comportamiento de conmutación (nivel de temporizado y de disparo) de las entradas. El árbol NAND ofrece medios directos para la caracterización de las entradas primarias. Se recomienda su implementación si el comportamiento de conmutación del circuito no puede ensayarse de otra manera.

- Auto-ensayo incorporado (BIST): El auto-ensayo de un circuito y en particular el auto-ensayo de la memoria embebida puede realizarse muy eficazmente mediante la implementación de un generador de patrón de ensayo en el chip. El BIST permite una verificación automática de la estructura del circuito mediante la aplicación de un patrón de ensayo pseudoaleatorio y evaluando de la firma de la estructura del circuito implementada. El BIST se recomienda como medida aditiva particularmente para el ensayo de memoria. El camino de barrido puede reemplazarse por un BIST.
- Ensayo de corriente en reposo (ensayo IDDQ): un circuito estático CMOS consume corriente principalmente durante el evento de conmutación. Un circuito absolutamente libre de defecto consume en consecuencia una cantidad muy pequeña de corriente ($< 1 \mu A$, corriente de fuga) mientras el patrón de ensayo se mantiene estacionario. El ensayo IDDQ es muy eficaz y suministra más del 50% de la cobertura de ensayo justo después de la aplicación de un par de patrones de ensayo. El ensayo IDDQ puede aplicarse sobre patrones de ensayo funcionales así como sobre patrones de ensayo sintetizados generados por el ATPG. Este método de ensayo se ha probado como de mucha ayuda en la práctica y es capaz de detectar disfunciones que otros ensayos raramente detectan o no pueden detectar. Esta medida debería en consecuencia aplicarse de manera aditiva a los ensayos normales de producción.
- Barrido de la frontera: La arquitectura de ensayo implementada para la verificación de la interconexión de los componentes de una placa de circuito impreso de acuerdo a la norma JTAG. La misma filosofía podría aplicarse para verificar la interconexión de módulos a nivel de chip. El barrido de frontera se recomienda primariamente para mejorar la capacidad de ensayo de las placas de circuito impreso.

E.32 Estimación de la cobertura de ensayo mediante simulación

Objetivo: Determinación de la cobertura de ensayo alcanzada por la arquitectura de ensayo implementada durante el ensayo de producción.

Descripción: La cobertura de ensayo alcanzada mediante el ensayo de camino de barrido, BIST, patrón de ensayo funcional o cualesquiera otras medidas puede determinarse mediante simulación de fallo. Durante la simulación de fallo se aplica un patrón de ensayo al circuito en el que se insertan los fallos. Una respuesta anómala del circuito ante los estímulos aplicados corresponde a los fallos insertados y así contribuye a la cobertura de ensayo. La simulación de fallo permite la detección de los fallos persistentes “persistente en 1” y “persistente en 0” y la cobertura de ensayo lograda representa la calidad del patrón de ensayo aplicado. La simulación de fallo puede en general utilizarse muy eficazmente para detectar los fallos asociados a la lógica que no es parte del camino de barrido, por ejemplo en el caso de caminos de barrido parciales.

E.33 Estimación de la cobertura de ensayo mediante la aplicación de la herramienta ATPG

Objetivo: Determinación de la cobertura de ensayo que puede esperarse mediante el patrón de ensayo sintetizado (camino de barrido, BIST) durante el ensayo de producción.

Descripción: Actualmente, hay una variedad de procedimientos que generan patrones de ensayo pseudoaleatorios o algorítmicos para un circuito implementado con camino de barrido. La herramienta de síntesis tal como el ATPG crea durante la síntesis un catálogo de fallos no detectados. La cobertura de ensayo puede así estimarse y define el límite inferior de la cobertura de ensayo alcanzada con el patrón de ensayo aplicado. Es importante notar que la cobertura de ensayo se limita a la lógica del circuito que se cubre mediante el camino de barrido. Los módulos tales como la memoria, BIST o parte de los circuitos que no está integrada en el camino de barrido no se consideran en la estimación de la cobertura de ensayo.

E.34 Justificación de probado en uso para núcleos hardware aplicados

Objetivo: Evitar la disfunción sistemática durante la aplicación de núcleos hardware.

Descripción: Un núcleo hardware se ve generalmente como una caja negra que representa la funcionalidad deseada y se compone de una base de datos de trazado en la tecnología objetivo que suministra el componente de circuito deseado. La posible disfunción funcional puede tratarse como en los componentes discretos como los microprocesadores normales, memorias, etc. La operación de tales núcleos sin la verificación de la correcta funcionalidad es posible, si para la tecnología objetivo aplicada el núcleo utilizado puede considerarse como componente probado en uso. El resto del circuito debería, a continuación, verificarse de manera intensiva.

E.35 Aplicación de núcleos hardware validados

NOTA Véase también el capítulo E.6 “Ensayo funcional a nivel de módulo”.

Objetivo: Evitar la disfunción sistemática durante la aplicación de núcleos hardware.

Descripción: La validación núcleo debería realizarse por los vendedores, debido a la naturaleza compleja del mismo y las limitaciones asumidas durante la fase de diseño en base al código fuente (V)HDL. La validación puede justificarse sólo para la configuración y la tecnología objetivo del componente aplicado.

E.36 Ensayo en línea de núcleos hardware

NOTA Véase también el capítulo E.13 “Cobertura de los escenarios de verificación (bancos de ensayo)”.

Objetivo: Evitar la disfunción sistemática durante la aplicación de núcleos hardware.

Descripción: Verificación de la función e implementación correctas de los núcleos hardware utilizados mediante la aplicación de ensayos en línea. En la aplicación de esta medida es necesario un concepto de ensayo eficaz y la evaluación del concepto aplicado debería documentarse.

E.37 Chequeo de la regla de diseño (DRC)

Objetivo: Verificación de las reglas de diseño del vendedor.

Descripción: Verificación del trazado generado en relación a las reglas de diseño del vendedor, por ejemplo longitudes mínimas y máximas de cableado y varias reglas relativas a la ubicación de las estructuras del trazado. Debería documentarse en detalle una ejecución completa y correcta del DRC.

E.38 Verificación del trazado versus el esquema (LVS)

Objetivo: Verificación independiente del trazado.

Descripción: El LVS extrae la funcionalidad del circuito de la base de datos del trazado y compara los elementos del circuito extraídos que incluyen las interconexiones con la lista de red de entrada. Esto asegura la equivalencia del trazado del circuito con la lista de red que especifica la funcionalidad del circuito. Se debería documentar en detalle una ejecución completa y correcta del LVS.

E.39 Margen adicional (>20%) para tecnologías de proceso en uso durante menos de 3 años

Objetivo: Asegurar la robustez de la funcionalidad del circuito implementado incluso bajo una fuerte fluctuación del proceso y del parámetro.

Descripción: El comportamiento real del circuito se define mediante un número de efectos físicos que se solapan, particularmente para pequeñas estructuras (por ejemplo por debajo de 0,5 μm). De hecho, debido a la falta de conocimiento detallado y a las necesarias simplificaciones, no puede derivarse un modelo exacto de los elementos del circuito. Con la disminución de las estructuras geométricas los retardos de línea juegan un papel más y más dominante. Los retardos de señal a lo largo de los hilos y las capacidades de acoplamiento cruzado entre los hilos crecen más que proporcionalmente. Los retardos de señal no siguen siendo despreciables comparados con los retardos de puerta. Los retardos de línea muestran un riesgo creciente con el decrecimiento de las estructuras geométricas.

En consecuencia, se recomienda planear un margen adecuado (>20%) en relación a las limitaciones de temporizado mínima y máxima para los circuitos diseñados utilizando procesos en uso desde hace menos de 3 años, con el fin de garantizar la correcta operación de la funcionalidad del circuito en presencia de parámetros fuertemente fluctuantes durante la producción o debido a la falta de un modelado preciso.

E.40 Ensayo de quemado

Objetivo: Asegurar la robustez del chip fabricado. Eliminar las disfunciones tempranas. Los chips sin encapsular no tienen que probar su robustez con el ensayo de quemado si no, por ejemplo, mediante métodos de esfuerzo a nivel de oblea.

Descripción: Se debería realizar el ensayo de quemado a la temperatura de funcionamiento tolerable más alta posible (generalmente 125 °C). La duración del ensayo depende del nivel de SIL objetivo o de las recomendaciones de quemado por ejemplo del fabricante del ASIC. El quemado puede utilizarse para:

- eliminar las disfunciones tempranas (principio de la curva de bañera con tasa de disfunción decreciente);
- probar que las disfunciones tempranas han sido ya eliminadas durante la fabricación y el ensayo (es decir, que los dispositivos fuera de la línea de producción ya están en la región de tasa de disfunción constante de la curva de bañera).

E.41 Aplicación de series de dispositivos probados en uso

Objetivo: Asegurar la fiabilidad de los chips fabricados.

Descripción: el fabricante del diseño de seguridad debería tener suficiente experiencia con la tecnología del dispositivo programable utilizado y con las herramientas de desarrollo implicadas.

E.42 Proceso de producción probado en uso

Objetivo: Asegurar la fiabilidad de los chips fabricados.

Descripción: Un proceso de producción probado en uso se caracteriza por una experiencia suficiente en la producción en serie.

E.43 Control de calidad del proceso de producción

Las medidas de calidad y los mecanismos de control durante el proceso de producción del dispositivo aseguran un control continuo del proceso. Por ejemplo el control óptico o eléctrico de las estructuras de ensayo, los ensayos de temperatura y humedad o el ensayo de ciclo de temperatura (véanse las Normas IEC 60068-2-1, IEC 60068-2-2, etc.).

E.44 Aprobación de la calidad de fabricación del dispositivo

La calidad del dispositivo se probará mediante los ensayos de esfuerzo de partes seleccionadas, por ejemplo, los ensayos de temperatura y humedad o los ensayos de cambio de temperatura (véanse las Normas IEC 60068-2-1, IEC 60068-2-2, etc.). El fabricante del dispositivo dará prueba de ello.

E.45 Aprobación de la calidad funcional del dispositivo

Todos los dispositivos serán ensayados funcionalmente. El fabricante del dispositivo dará prueba de ello.

E.46 Normas de calidad

El fabricante del ASIC debería suministrar una gestión de la calidad suficiente, por ejemplo, documentada con un Manual de Calidad y Fiabilidad: Certificación ISO 9000 o SSQA - Evaluación Estándar de la Calidad del Suministrador.

ANEXO F (Informativo)**DEFINICIONES DE LAS PROPIEDADES DE LAS FASES DEL CICLO DE VIDA DEL SOFTWARE**

Tabla F.1 – Especificación de los requisitos de seguridad del software
(Véase la Norma IEC 61508-3, apartado 7.2 y la Norma IEC 61508-3 tabla C.1)

	Propiedad	Definición
1.1	Completitud en relación a las necesidades de seguridad a ser gestionadas mediante el software	La especificación de requisitos de seguridad del software trata todas las necesidades y limitaciones de seguridad que resultan de las fases tempranas del ciclo de vida de seguridad y asignadas al software. Las necesidades y limitaciones de seguridad son normalmente establecidas en las entradas de la actividad de especificación de requisitos de seguridad del software. Esto puede incluir la especificación de qué no debe hacer el software o qué debe evitar.
1.2	Corrección en relación a las necesidades de seguridad a ser gestionadas mediante el software	La especificación de requisitos de seguridad del software que suministra una respuesta apropiada a las necesidades y limitaciones de seguridad asignadas al software. El objetivo es asegurar que lo que se ha especificado garantizará realmente la seguridad en todas las condiciones necesarias
1.3	Exención de fallos intrínsecos de especificación, incluyendo exención de la ambigüedad	La completitud y la consistencia internas de la especificación de requisitos de seguridad del software: que suministra toda la información necesaria para todas las funciones y situaciones que pueden derivarse de sus enunciados; que se expresan sin contradicciones o enunciados inconsistentes. Al contrario que para la completitud y la consistencia en relación a las necesidades de seguridad, la completitud y la consistencia internas pueden evaluarse basándose sólo en la especificación de requisitos de seguridad del software
1.4	Capacidad de comprensión de los requisitos de seguridad	La especificación de requisitos de seguridad del software es totalmente entendible sin un esfuerzo excesivo por todos aquellos que necesitan leerla, incluso si no han estado involucrados al principio del proyecto, siempre que tengan el conocimiento exigido. Un objetivo importante es facilitar la verificación y, posiblemente, las modificaciones.
1.5	Exención de interferencia adversa de las funciones de no-seguridad sobre las necesidades de seguridad gestionadas mediante el software	La especificación de requisitos de seguridad del software evita los requisitos que no son necesarios para la seguridad del EUC. El objetivo es evitar una complejidad innecesaria en el diseño y la implementación del software, para reducir el riesgo de que los fallos y las funciones no importantes para la seguridad interfieran con, o comprometan a, aquellas que son importantes para la seguridad
1.6	Capacidad de suministrar una base para la verificación y la validación	La especificación de requisitos de seguridad del software provee los ensayos y exámenes que generan la evidencia objetiva de que el software satisface la especificación de requisitos de seguridad del software.

Tabla F.2 – Diseño y desarrollo del software: diseño de la arquitectura del software
(Véase la Norma IEC 61508-3, apartado 7.4.3 y la Norma IEC 61508-3 tabla C.2)

	Propiedad	Definición
2.1	Complejidad en relación a la especificación de los requisitos de seguridad del software	El diseño de la arquitectura del software trata todas las necesidades y limitaciones invocadas por la especificación de requisitos de seguridad del software.
2.2	Corrección en relación a la especificación de los requisitos de seguridad del software	El diseño de la arquitectura del software suministra una respuesta apropiada a los requisitos de seguridad del software especificados.
2.3	Exención de fallos intrínsecos de diseño	El diseño de la arquitectura del software y la documentación del diseño están libres de fallos que pueden identificarse independientemente de cualquier requisito de seguridad del software. Ejemplos: bloqueos, acceso a recursos no autorizados, fugas de recursos, no completitud intrínseca (es decir, disfunción al tratar todas las situaciones que se deriven del propio diseño).
2.4	Simplicidad y capacidad de comprensión Capacidad de predecir el comportamiento	El diseño de la arquitectura del software que permite una predicción correcta y precisa, en todas las situaciones especificadas, del funcionamiento del software. En particular, estas situaciones incluyen situaciones erróneas y de disfunción. La capacidad de predicción implica en particular que el funcionamiento no depende de elementos que no pueden ser controlados por los diseñadores o los usuarios.
2.5	Diseño verificable y ensayable	El diseño de la arquitectura del software y la documentación de diseño permiten y facilitan la elaboración de la evidencia creíble de que todos los requisitos de seguridad del software especificados son correctamente tenidos en cuenta por el diseño y de que el diseño está exento de fallos intrínsecos. La capacidad de verificación puede implicar propiedades derivadas tales como modularidad, claridad, capacidad de ensayo, capacidad de prueba, etc., dependiendo de las técnicas de verificación utilizadas.
2.6	Tolerancia al fallo	El diseño de la arquitectura del software asegura que el software tendrá un comportamiento seguro en presencia de errores (errores internos, errores de los operadores o de los sistemas externos). El diseño defensivo puede ser activo o pasivo. Los diseños defensivos activos pueden incluir características tales como la detección, informe y contención de errores, degradación escalonada y limpieza de todo efecto colateral no deseado antes de reiniciar la operación normal. Los diseños defensivos pasivos incluyen características que garantizan la impermeabilidad a tipos particulares de errores o condiciones particulares (avalanchas de entradas, horas y fechas concretas) sin que el software tome ninguna acción específica.
2.7	Defensa frente a la disfunción de causa común de sucesos externos	El diseño de la arquitectura del software facilita la identificación de modos de disfunción de causa común y las precauciones eficaces contra la disfunción.

Tabla F.3 – Diseño y desarrollo del software: herramientas de soporte y lenguaje de programación
(Véase la Norma IEC 61508-3, apartado 7.4.4 y la Norma IEC 61508-3 tabla C.3)

	Propiedad	Definición
3.1	Soporta la producción del software con las propiedades software exigidas	Medios para suministrar la detección de errores o la eliminación de construcciones proclives al error
3.2	Claridad de la operación y de la funcionalidad de la herramienta	La provisión de la cobertura y realimentación exhaustivas de todos los aspectos de operación de la herramienta.
3.3	Corrección y repetibilidad de la salida	La consistencia y la precisión de la salida de la herramienta para toda entrada dada

Tabla F.4 – Diseño y desarrollo del software: diseño detallado
(Véase la Norma IEC 61508-3, apartado 7.4.5, la Norma IEC 61508-3 apartado 7.4.6 y la Norma IEC 61508-3 tabla C.4)

	Propiedad	Definición
4.1	Completitud en relación a la especificación de los requisitos de seguridad del software	Se adoptan métodos de diseño detallado del software y de la producción que aseguran que el software resultante trata todas las necesidades y limitaciones asignadas al software.
4.2	Corrección en relación a la especificación de los requisitos de seguridad del software	Existe evidencia específica para declarar que los requisitos de seguridad asignados al software han sido cumplidos por el software desarrollado.
4.3	Exención de fallos intrínsecos de diseño	El software desarrollado está libre de fallos intrínsecos. Ejemplos: bloqueos, acceso a recursos no autorizados, fugas de recursos.
4.4	Simplicidad y capacidad de comprensión Capacidad de predecir el comportamiento	El comportamiento del software desarrollado es predecible mediante ensayo y análisis objetivos y convincentes.
4.5	Diseño verificable y ensayable	El software desarrollado es verificable y puede ensayarse.
4.6	Tolerancia al fallo/Detección del fallo	Las técnicas y diseños aseguran que el software desarrollado se comportará de manera segura en presencia de errores.
4.7	Exención de disfunción de causa común	Las técnicas y diseños identifican los modos de disfunción de causa común y suministran precauciones eficaces contra la disfunción del software.

Tabla F.5 – Diseño y desarrollo del software: ensayo e integración del módulo de software
(Véase la Norma IEC 61508-3, apartado 7.4.7, la Norma IEC 61508-3 apartado 7.4.8 y la Norma IEC 61508-3 tabla C.5)

	Propiedad	Definición
5.1	Compleitud del ensayo y la integración en relación a las especificaciones de diseño	En ensayo del software examina el comportamiento del software de manera suficientemente concienzuda como para asegurar que todos los requisitos de la especificación de diseño del software han sido tratados.
5.2	Corrección del ensayo y la integración en relación a las especificaciones de diseño del software (terminación exitosa)	La tarea de ensayo del software se ha completado y existe evidencia específica para declarar que se han cumplido los requisitos de seguridad.
5.3	Repetibilidad	Se producen resultados consistentes al repetir las evaluaciones individuales realizadas como parte del ensayo e integración del módulo.
5.4	Configuración de ensayo definida de manera precisa	El ensayo e integración del módulo han sido aplicados a la versión correcta de los elementos y del software, con los resultados declarados, y permiten que los resultados se ligen a la configuración específica del software incorporado.

Tabla F.6 – Integración de la electrónica programable (hardware y software)
(Véase la Norma IEC 61508-3, apartado 7.5 y la Norma IEC 61508-3 tabla C.6)

	Propiedad	Definición
6.1	Compleitud de la integración en relación a las especificaciones de diseño	La integración suministra la profundidad y cobertura apropiadas de los elementos del sistema para demostrar que puede realizar las funciones previstas y que no realiza funciones no previstas en las condiciones previstas y con disfunción del sistema. Esto cubre los principios utilizados para la verificación, los niveles objetivo de diseño y los aspectos de integración (por ejemplo, la verificación de la completitud de la interacción entre módulos)
6.2	Corrección de la integración en relación a las especificaciones de diseño (terminación exitosa)	La integración se basa en hipótesis correctas. Por ejemplo, corrección de los resultados esperados, de las condiciones de utilización consideradas, representatividad de los entornos de ensayo. La tarea de integración se ha completado y existe evidencia específica para declarar que los requisitos de seguridad se han cumplido.
6.3	Repetibilidad	Se producen resultados consistentes al repetir las evaluaciones individuales realizadas como parte de la integración.
6.4	Configuración de integración definida de manera precisa	La integración asegura de forma apropiada que ha sido efectivamente aplicada como se documentó para la versión correcta de los elementos y del software, con los resultados declarados, y permite que los resultados se ligen a la configuración específica del software incorporado.

Tabla F.7 – Aspectos software de la validación de la seguridad del sistema
(Véase la Norma IEC 61508-3, apartado 7.7 y la Norma IEC 61508-3 tabla C.7)

	Propiedad	Definición
7.1	Compleitud de la validación en relación a la especificación de diseño del software	La validación del software trata todos los requisitos de la especificación de diseño del software.
7.2	Corrección de la validación en relación a la especificación de diseño del software (terminación exitosa)	La tarea de validación software ha sido completada y existe evidencia específica para declarar que los requisitos de seguridad se han cumplido.
7.3	Repetibilidad	Se producen resultados consistentes al repetir las evaluaciones individuales realizadas como parte de la validación del software.
7.4	Configuración de validación definida de manera precisa	La definición clara y concisa de: el sistema los requisitos el entorno

Tabla F.8 – Modificación del software
(Véase la Norma IEC 61508-3, apartado 7.8 y la Norma IEC 61508-3 tabla C.8)

	Propiedad	Definición
8.1	Compleitud de la modificación en relación a sus requisitos	La modificación ha sido adecuadamente aprobada por el personal autorizado, con un entendimiento adecuado de sus consecuencias funcionales, de seguridad, técnicas y operacionales.
8.2	Corrección de la modificación en relación a sus requisitos	La modificación logra sus objetivos especificados.
8.3	Exención de la introducción de fallos intrínsecos de diseño	La modificación no introduce nuevos fallos sistemáticos. Ejemplos: división por cero, índices o punteros fuera de límite, utilización de variables no inicializadas.
8.4	Capacidad de evitar el comportamiento no deseado	La modificación no introduce ningún comportamiento que, de acuerdo con las limitaciones establecidas en la especificación de requisitos de seguridad del software, deba ser evitado.
8.5	Diseño verificable y ensayable	El diseño del software es tal que el efecto de la modificación puede ser examinado concienzudamente.
8.6	Cobertura del ensayo de regresión y la verificación	El diseño del software es tal que es posible el ensayo de regresión eficaz y concienzudo para demostrar que el software después de la modificación sigue satisfaciendo la especificación de requisitos de seguridad del software.

Tabla F.9 – Verificación del software
(Véase la Norma IEC 61508-3, apartado 7.9 y la Norma IEC 61508-3 tabla C.9)

	Propiedad	Definición
9.1	Compleitud de la verificación en relación a la fase previa	La verificación es capaz de establecer que el software satisface todos los requisitos pertinentes de la especificación de requisitos de seguridad del software.
9.2	Corrección de la verificación en relación a la fase previa (terminación exitosa)	La tarea de verificación ha sido completada, y existe evidencia específica para declarar que los requisitos de seguridad se han cumplido.
9.3	Repetibilidad	Se producen resultados consistentes al repetir las evaluaciones individuales realizadas como parte de la verificación.
9.4	Configuración de verificación definida de manera precisa	La verificación ha sido aplicada para la versión correcta de los elementos y del software, con los resultados declarados, y permite que los resultados se ligen a la configuración específica del software incorporado.

Tabla F.10 – Evaluación de la seguridad funcional
(Véase la Norma IEC 61508-3, capítulo 8 y la Norma IEC 61508-3 tabla C.10)

	Propiedad	Definición
10.1	Compleitud de la evaluación de la seguridad funcional en relación a esta norma	La evaluación de la seguridad funcional del software produce una declaración clara del alcance del cumplimiento hallado, de los juicios realizados, de las acciones de remedio y de las escalas de tiempo recomendadas, de las conclusiones alcanzadas y de las recomendaciones que surgen para la aceptación, la aceptación condicionada o el rechazo y para toda limitación de tiempo dada para dichas recomendaciones.
10.2	Corrección de la evaluación de la seguridad funcional en relación a las especificaciones de diseño (terminación exitosa)	La tarea de evaluación de la seguridad funcional del software ha sido completada, y existe evidencia específica para declarar que los requisitos de seguridad se han cumplido.
10.3	Cierre trazable de todos los problemas identificados	Hay una clara declaración del alcance para el que los temas que surgen durante la evaluación de la seguridad funcional del software han sido tratados.
10.4	Capacidad para modificar la evaluación de la seguridad funcional después de los cambios sin necesidad de rehacer extensivamente la evaluación	La evaluación de la seguridad funcional del software es capaz de ser rehecha para permitir que las partes de la evaluación de la seguridad funcional del software sean reevaluadas después del cambio del software y para que se logren las conclusiones revisadas, sin necesidad de rehacer extensivamente la evaluación completa de la seguridad funcional del software.
10.5	Repetibilidad	La evaluación de la seguridad funcional se realiza según un proceso consistente, planificado y abierto sobre los individuos identificados y se documenta, lo que permite el escrutinio de la base para las evaluaciones y el juicio logrado para todos aquellos afectados sus juicios, incluyendo suministradores del sistema, usuarios, mantenedores y reguladores. La evaluación de la seguridad funcional permite al personal competente independiente repetir las evaluaciones individuales realizadas como parte de la evaluación.

	Propiedad	Definición
10.6	Que es oportuna	La evaluación de la seguridad funcional se realiza con una frecuencia apropiada ligada a las fases del ciclo de vida de la seguridad del software y al menos antes de que determinados peligros estén presentes, y suministra oportunamente un informe de deficiencias. Los resultados de los ensayos, inspecciones, análisis, etc. están realmente disponibles cuando se exigen como entrada para una decisión de la evaluación.
10.7	Configuración definida de manera precisa	La evaluación de la seguridad funcional del software permite que los resultados sean ligados a la configuración específica del sistema que va a ser fundamentada mediante los resultados de la evaluación de la seguridad funcional.

ANEXO G (Informativo)**GUÍA PARA EL DESARROLLO DE SOFTWARE RELACIONADO CON LA SEGURIDAD
ORIENTADO A OBJETOS**

Todas las recomendaciones de esta norma para el diseño del software se aplican al software orientado a objetos. Puesto que la aproximación orientada a objetos presenta la información de manera diferente de las aproximaciones de procedimiento o funcionales, la siguiente lista contiene aquellas recomendaciones que necesitan una consideración específica:

- comprensión de las jerarquías de clases, e identificación de la(s) función(es) de software que será ejecutada tras la invocación de un método dado (incluyendo cuando se utilice una biblioteca de clases existente);
- ensayo basado en estructura (Norma IEC 61508-3, tabla B.2 y Norma IEC 61508-7, apartado C.5.8).

Las tablas G.1 y G.2 suministran guías informativas para la utilización del software orientado a objetos que suplementan las directrices normativas más generales dadas en la Norma IEC 61508-3, tablas A.2 y A.4.

Tabla G.1 – Arquitectura del Software Orientado a objetos

	Recomendación	Detalles	SIL1	SIL2	SIL3	SIL4
G1.1	Trazabilidad del concepto del dominio de aplicación a las clases de la arquitectura.	Nota 1	R	HR	HR	HR
G1.2	Utilización de marcos adecuados, combinaciones de clases y diseño de patrones comúnmente utilizados. NOTA Cuando se utilizan marcos existentes y patrones de diseño, los requisitos del software predesarrollado se aplican a dichos marcos y patrones	Nota 2	R	HR	HR	HR
NOTA 1 La trazabilidad del dominio de aplicación a la arquitectura de clases es menos importante. NOTA 2 EJEMPLO 1: Para una parte del proyecto previsto relacionado con la seguridad podría existir un marco de un proyecto no relacionado con la seguridad que haya resuelto una tarea similar de manera exitosa y que sea bien conocido por los participantes del proyecto. En tal caso se recomienda la utilización de dicho marco. EJEMPLO 2: Puede ocurrir que se necesiten diferentes algoritmos para resolver sub-tareas muy conectadas del proyecto relacionado con la seguridad. El patrón de la estrategia puede elegirse para acceder a los algoritmos. EJEMPLO 3: Parte del proyecto relacionado con la seguridad puede consistir en la entrega de avisos internos a las personas internas y externas interesadas. El patrón del observador puede elegirse para organizar estos avisos. El requisito no se aplica a las bibliotecas. NOTA 3 Normalmente es una clase básica abstracta que suministra acceso a las clases derivadas concretas.						

Tabla G.2 – Diseño detallado orientado a objetos

	Recomendación	SIL1	SIL2	SIL3	SIL4
G2.1	Las clases deberían tener sólo un objetivo	R	R	HR	HR
G2.2	Herencia utilizada sólo si la clase derivada es un refinamiento de su clase básica	HR	HR	HR	HR
G2.3	Profundidad de la herencia limitada por las normas de codificación	R	R	HR	HR
G2.4	Sobrecarga de operaciones (métodos) bajo control estricto	R	HR	HR	HR
G2.5	Herencia múltiple utilizada sólo para las clases de interfaz	HR	HR	HR	HR
G2.6	Herencia de clases desconocidas			NR	NR
G2.7	Verificación de que la biblioteca orientada a objetos reutilizada cumple las recomendaciones de esta tabla	HR	HR	HR	HR
NOTA 1 En otras palabras: Una clase se caracteriza por tener una responsabilidad, es decir cuida de los datos muy conectados y de las operaciones sobre dichos datos.					
NOTA 2 Se requiere tener cuidado para evitar dependencias circulares entre los objetos.					

Los siguientes términos utilizados anteriormente se definen informalmente aquí.

Tabla G.3 – Algunos Términos Orientados a Objeto

Término	Definición informal
Clase básica	Clase que ha derivado clases. Una clase básica es algunas veces llamada clase superior o clase padre
Clase derivada	Clase (conjunto de atributos y operaciones) que hereda atributos y/o operaciones de otra clase (clase básica). Una clase derivada es a veces llamada subclase o clase hijo.
Marco	Estructura de un programa, en muchos casos predesarrollada con el fin de ser rellenada para una aplicación específica.
Sobrecarga	Reemplazar una operación (método, subrutina) por otra operación (método, subrutina) de la misma firma y jerarquía de herencia durante el tiempo de ejecución; propiedad de los lenguajes y programas orientados a objetos; implementa el polimorfismo.
Firma de una operación	Nombre de una operación (subrutina, método), junto con sus parámetros (argumentos) y sus tipos, ocasionalmente también sus tipos de retorno. Dos firmas son iguales si tienen los mismos nombres, número y tipos de parámetros; en algunos lenguajes también tienen que ser iguales los tipos de retorno.

BIBLIOGRAFÍA

- [1] IEC 60068-1:1988, *Environmental testing. Part 1: General and guidance*.
| NOTA Armonizada como Norma EN 60068-1:1994 (sin ninguna modificación).
- [2] IEC 60529:1989, *Degrees of protection provided by enclosures (IP Code)*.
| NOTA Armonizada como Norma EN 60529:1991 (sin ninguna modificación).
- [3] IEC 60812:2006, *Analysis techniques for system reliability. Procedure for failure mode and effects analysis (FMEA)*.
| NOTA Armonizada como Norma EN 60812:2006 (sin ninguna modificación).
- [4] IEC 60880:2006, *Nuclear power plants. Instrumentation and control systems important to safety. Software aspects for computer-based systems performing category A functions*.
| NOTA Armonizada como Norma EN 60880:2009 (sin ninguna modificación).
- [5] IEC 61000-4-1:2006, *Electromagnetic compatibility (EMC). Part 4-1: Testing and measurement techniques. Overview of IEC 61000-4 series*.
| NOTA Armonizada como Norma EN 61000-4-1:2007 (sin ninguna modificación).
- [6] IEC 61000-4-5:2005, *Electromagnetic compatibility (EMC). Part 4-5: Testing and measurement techniques. Surge immunity test*.
| NOTA Armonizada como Norma EN 61000-4-5:2006 (sin ninguna modificación).
- [7] IEC/TR 61000-5-2:1997, *Electromagnetic compatibility (EMC). Part 5: Installation and mitigation guidelines. Section 2: Earthing and cabling*.
- [8] IEC 61025:2006, *Fault tree analysis (FTA)*.
| NOTA Armonizada como Norma EN 61025:2007 (sin ninguna modificación).
- [9] IEC 61069-5:1994, *Industrial-process measurement and control. Evaluation of system properties for the purpose of system assessment. Part 5: Assessment of system dependability*.
- [10] IEC 61078:2006, *Analysis techniques for dependability. Reliability block diagram and boolean methods*.
| NOTA Armonizada como Norma EN 61078:2006 (sin ninguna modificación).
- [11] IEC 61131-3:2003, *Programmable controllers. Part 3: Programming languages*.
| NOTA Armonizada como Norma EN 61131-3:2003 (sin ninguna modificación).
- [12] IEC 61160:2005, *Design review*.
| NOTA Armonizada como Norma EN 61160:2005 (sin ninguna modificación).
- [13] IEC 61163-1:2006, *Reliability stress screening. Part 1: Repairable assemblies manufactured in lots*.
| NOTA Armonizada como Norma EN 61163-1:2006 (sin ninguna modificación).

- [14] IEC 61164:2004, *Reliability growth. Statistical test and estimation methods*.
| NOTA Armonizada como Norma EN 61164:2004 (sin ninguna modificación).
- [15] IEC 61165:2006, *Application of Markov techniques*.
| NOTA Armonizada como Norma EN 61165:2006 (sin ninguna modificación).
- [16] IEC 61326-3-1:2008, *Electrical equipment for measurement, control and laboratory use. EMC requirements. Part 3-1: Immunity requirements for safety-related systems and for equipment intended to perform safety-related functions (functional safety). General industrial applications*.
| NOTA Armonizada como Norma EN 61326-3-1:2008 (sin ninguna modificación).
- [17] IEC 61326-3-2:2008, *Electrical equipment for measurement, control and laboratory use. MC requirements. Part 3-2: Immunity requirements for safety-related systems and for equipment intended to perform safety-related functions (functional safety). Industrial applications with specified electromagnetic environment*.
| NOTA Armonizada como Norma EN 61326-3-2:2008 (sin ninguna modificación).
- [18] IEC 81346-1:2009, *Industrial systems, installations and equipment and industrial products. Structuring principles and reference designations. Part 1: Basic rules*.
| NOTA Armonizada como Norma EN 81346-1:2009 (sin ninguna modificación).
- [19] IEC 61506:1997, *Industrial-process measurement and control. Documentation of application software*.
- [20] IEC/TR 61508-0:2005, *Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 0: Functional safety and IEC 61508*.
- [21] IEC 61511 (todas las partes), *Functional safety. Safety instrumented systems for the process industry sector*.
| NOTA Armonizada como serie de Normas EN 61511 (sin ninguna modificación).
- [22] IEC 62061:2005, *Safety of machinery. Functional safety of safety-related electrical, electronic and programmable electronic control systems*.
| NOTA Armonizada como Norma EN 62061:2005 (sin ninguna modificación).
- [23] IEC 62308:2006, *Equipment reliability. Reliability assessment methods*.
| NOTA Armonizada como Norma EN 62308:2006 (sin ninguna modificación).
- [24] ISO/IEC 1539-1:2004, *Information technology. Programming languages. Fortran. Part 1: Base language*.
- [25] ISO 5807:1985, *Information processing. Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts*.
- [26] ISO/IEC 7185:1990, *Information technology. Programming languages. Pascal*.
- [27] ISO/IEC 8631:1989, *Information technology. Program constructs and conventions for their representation*.
- [28] ISO/IEC 8652:1995, *Information technology. Programming languages. Ada*.
- [29] ISO 8807:1989, *Information processing systems. Open Systems Interconnection. LOTOS. A formal description technique based on the temporal ordering of observational behaviour*.
- [30] ISO/IEC 9899:1999, *Programming languages. C*.

- [31] ISO/IEC 10206:1991, *Information technology. Programming languages. Extended Pascal.*
- [32] ISO/IEC 10514-1:1996, *Information technology. Programming languages. Part 1: Modula-2, Base Language.*
- [33] ISO/IEC 10514-3:1998, *Information technology. Programming languages. Part 3: Object Oriented Modula-2.*
- [34] ISO/IEC 13817-1:1996, *Information technology. Programming languages, their environments and system software interfaces. Vienna Development Method. Specification Language. Part 1: Base language.*
- [35] ISO/IEC 14882:2003, *Programming languages. C++.*
- [36] ISO/IEC/TR 15942:2000, *Information technology. Programming languages. Guide for the use of the Ada programming language in high integrity systems.*
- [37] IEC 61800-5-2, *Electromagnetic compatibility (EMC). Part 5: Installation and mitigation guidelines. Section 2: Earthing and cabling.*
- | NOTA Armonizada como Norma EN 61800-5-2.
- [38] IEC 60601 (all parts), *Medical electrical equipment.*
- | NOTA Armonizada como serie de Normas EN 60601 (parcialmente modificada).
- [39] IEC 60068-2-1, *Environmental testing. Part 2-1: Tests. Test A: Cold.*
- | NOTA Armonizada como Norma EN 60068-2-1.
- [40] IEC 60068-2-2, *Environmental testing. Part 2-2: Tests. Test B: Dry heat.*
- | NOTA Armonizada como Norma EN 60068-2-2.
- [41] ISO 9000, *Quality management systems. Fundamentals and vocabulary.*
- | NOTA Armonizada como Norma EN ISO 9000.
- [42] IEC 61508-1:2010, *Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 1: General requirements.*
- | NOTA Armonizada como Norma EN 61508-1:2010 (sin ninguna modificación).
- [43] IEC 61508-2:2010, *Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 2: Requirements for electrical/electronic/programmable electronic safety-related systems.*
- | NOTA Armonizada como Norma EN 61508-2:2010 (sin ninguna modificación).
- [44] IEC 61508-3:2010, *Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 3: Software requirements.*
- | NOTA Armonizada como Norma EN 61508-3:2010 (sin ninguna modificación).
- [45] IEC 61508-6: 2010, *Functional safety of electrical/electronic/programmable electronic safety-related systems. Part 6: Guidelines on the application of IEC 61508-2 and IEC 61508-3.*
- | NOTA Armonizada como Norma EN 61508-6:2010 (sin ninguna modificación).

ÍNDICE

A

Accionamiento de la parada de seguridad por medio de un fusible térmico	A.10.3
Acuse de recibo de las entradas	B.4.9
Alimentación	A.8
Análisis de disfunciones	B.6.6
Análisis de flujo de control	C.5.9
Análisis de flujo de datos	C.5.10
Análisis de impacto	C.5.23
Análisis de las disfunciones de causa común	C.6.3
Análisis de los modos de disfunción y de sus efectos (AMFE)	B.6.6.1
Análisis de los modos de disfunción, de sus efectos y de su criticidad (FMECA)	B.6.6.4
Análisis de los valores límites	C.5.4
Análisis del caso más desfavorable	B.6.7
Análisis estático	B.6.4
Análisis estático del retardo de propagación (STA)	E.23
Análisis numérico fuera de línea	C.2.13
Análisis por árbol de eventos (ETA)	B.6.6.3
Análisis por árbol de disfunciones (AAF)	B.6.6.5
Análisis y ensayo dinámicos	B.6.5
Animación de la especificación y del diseño	C.5.26
Aplicación de biblioteca objetivo probada en uso	E.29
Aplicación de funciones lógicas validadas	E.20
Aplicación de herramienta de síntesis probada en uso	E.28
Aplicación de la herramienta de revisión de código	E.15
Aplicación de núcleos hardware validados	E.35
Aplicación de series de dispositivos probados en uso	E.41
Aprobación de la calidad de fabricación del dispositivo	E.44
Aprobación de la calidad funcional del dispositivo	E.45
Aproximación modular	C.2.9
Aumento de la inmunidad a las interferencias	A.11.3
Autotest soportado por el hardware (un canal)	A.3.3
Autotests mediante el software: bit deslizante (un canal)	A.3.2
Autotests mediante el software: Número limitado de configuraciones (un canal)	A.3.1

C

Casos de utilización	C.3.12.2
CCS – Calculus of Communicating Systems	C.2.4.2
Chequeo de la regla de diseño (DRC)	E.37
Chequeo de los requisitos y limitaciones del vendedor	E.26
Chequeo del modelo	C.5.12.1
Clases de equivalencia y ensayo de las particiones de entrada	C.5.7
Cobertura de los escenarios de verificación (bancos de ensayo)	E.13
Códigos de detección y corrección de errores	C.3.2
Combinación de supervisión temporal y lógica de las secuencias del programa	A.9.4
Comparación de la lista de red de puertas con el modelo de referencia (ensayo formal de equivalencia)	E.25

Comparación del programa fuente y del código ejecutable	C.4.4.1
Comparación recíproca mediante el software	A.3.5
Comparación/voto mayoritario sobre las entradas	A.6.5
Comparadores	A.1.3
Componentes eléctricos/electrónicos con chequeo automático	A.2.6
Comunicación y almacenamiento masivo	A.11
Conexión de la refrigeración por aire forzado e indicación de estado	A.10.5
Conmutador de acción directa	A.12.2
Control de calidad del proceso de producción	E.43
Control de los ventiladores	A.10.2
Control en línea durante la creación de variables dinámicas u objetos dinámicos	C.2.6.4
Corrección de fallos utilizando las técnicas de inteligencia artificial	C.3.9
CSP – Communicating Sequential Processes	C.2.4.3

D

Degradación “escalonada”	C.3.8
Desarrollo de Jackson del sistema (JSD, Jackson System Development)	C.2.1.3
Descripción del diseño en (V)HDL	E.1
Descripción estructurada	E.3
Detección de disfunciones por supervisión en línea	A.1.1
Detección y diagnóstico de fallos	C.3.1
Devaluación	A.2.8
Diagramas de actividad	C.3.12.3
Diagramas de bloques de fiabilidad	C.6.4
Diagramas de Bloques de Fiabilidad (RBD)	B.6.6.7
Diagramas de causa-consecuencia	B.6.6.2
Diagramas de clase	C.3.12.1
Diagramas de estructuras	C.2.3
Diagramas de flujos de datos	C.2.2
Diseño de la arquitectura	C.3
Diseño de software sin estados (o diseño de estados limitados)	C.2.12
Diseño estructurado	B.3.2
Diseño para la capacidad de ensayo	E.11
Diseño y desarrollo de los sistemas E/E/PE	B.3
Dispositivo de voto mayoritario	A.1.4
Diversidad del hardware	B.1.4
Diversidad del software (programación diversificada)	C.3.5
Doble RAM con comparación por hardware o software y ensayo de lectura/escritura	A.5.7
Documentación	B.1.2
Documentación de las limitaciones, resultados y herramientas de síntesis	E.27
Documentación de los resultados de simulación	E.17

E

Ejecución simbólica	C.5.11
Eléctricos	A.1
Electrónicos	A.2
Elementos finales (accionadores)	A.13
Ensayo basado en modelo (generación de casos de ensayo)	C.5.27

Ensayo completo	D.2.4
Ensayo de “caja negra”	B.5.2
Ensayo de inserción de fallos	B.6.10
Ensayo de interfaz	C.5.3
Ensayo de quemado	E.40
Ensayo de RAM “Abraham”	A.5.4
Ensayo de RAM “damero” o “marcha”	A.5.1
Ensayo de RAM “galpat” o “galpat transparente”	A.5.3
Ensayo de RAM “walkpath”	A.5.2
Ensayo de un espacio de entrada (dominio) para un modo de funcionamiento de baja demanda	D.2.2
Ensayo del caso más desfavorable	B.6.9
Ensayo en línea de núcleos hardware	E.36
Ensayo estadístico	B.5.3
Ensayo estadístico simple en modo de funcionamiento de alta demanda o continuo	D.2.3
Ensayo estadístico simple en modo de funcionamiento de baja demanda	D.2.1
Ensayo funcional	B.5.1
Ensayo funcional a nivel de módulo	E.6
Ensayo funcional al máximo nivel	E.7
Ensayo funcional embebido en el entorno del sistema	E.8
Ensayo funcional extendido	B.6.8
Ensayo probabilístico	C.5.1
Ensayos basados en la estructura	C.5.8
Ensayos de avalancha/estrés	C.5.21
Ensayos de inmunidad a las ondas de choque	B.6.2
Ensayos funcionales en las condiciones del entorno	B.6.1
Ensayos por el hardware redundante	A.2.1
Entrada esquemática	E.2
Especificación de los requisitos de diseño del sistema E/E/PE	B.2
Especificación estructurada	B.2.1
Estimación de la cobertura de ensayo mediante la aplicación de la herramienta ATPG	E.33
Estimación de la cobertura de ensayo mediante simulación	E.32
Estímulo y respuesta	B.2.4.5
Estudio del peligro y de la operatividad del software (CHAZOP, AMFE)	C.6.2
Evaluación de la seguridad funcional	C.6
Evaluación de un cuerpo de evidencia de verificación	C.2.10.2
Experiencia en campo	B.5.4
Expresión controlada de requisitos (CORE, Controlled Requirements Expression)	C.2.1.2

F

Facilidad de mantenimiento	B.4.3
Facilidad de utilización	B.4.2
Firma de una palabra doble (16 bits)	A.4.4
Firma de una sola palabra (8 bits)	A.4.3
Fórmulas de ensayos estadísticos y ejemplos de utilización	D.2

G

Generación automática de software	C.4.6
Gestión de la configuración del software	C.5.24

Gestión de proyecto	B.1.1
Gestión del ensayo y herramientas de automatización	C.4.7

H

Herramientas certificadas y traductores certificados	C.4.3
Herramientas de desarrollo y lenguajes de programación	C.4
Herramientas de diseño asistido por ordenador	B.3.5
Herramientas de especificación asistidas por ordenador	B.2.4
Herramientas no orientadas a ningún método específico	B.2.4.2
Herramientas probadas en uso	E.4
Herramientas y traductores: aumento de confianza como resultado de su utilización	C.4.4
HOL – Higher Order Logic	C.2.4.4

I

Implementación de errores	C.5.6
Implementación de las estructuras de ensayo	E.31
Inspección (revisión y análisis)	B.3.7
Inspección de la especificación	B.2.6
Inspección del código	E.18
Inspección utilizando los patrones de ensayo	A.7.4
Inspecciones formales	C.5.14
Instrucciones de operación y de mantenimiento	B.4.1
Integración de los sistemas E/E/PE	B.5

J

Justificación de probado en uso para núcleos hardware aplicados	E.34
---	------

L

Lenguajes de programación adecuados	C.4.5
Lenguajes de programación fuertemente tipificados	C.4.1
Listas de control	B.2.5
Lógica temporal	C.2.4.7
LOTOS	C.2.4.5

M

Mapas de secuencia de mensaje	C.2.14
Máquinas de estados finitos/diagramas de cambios de estado	B.2.3.2
Margen adicional (>20%) para tecnologías de proceso en uso durante menos de 3 años	E.39
Mecanismos de recuperación de fallos por relanzamiento	C.3.7
Medidas contra el entorno físico	A.14
Medidas y técnicas generales	B.1
Mensaje escalonado de los sensores térmicos y alarma condicional	A.10.4
Métodos estructurados de diagrama	C.2.1
Métodos formales	B.2.2
Métodos formales	C.2.4

Métodos semiformales	B.2.3
Métricas de complejidad	C.5.13
Modelización del funcionamiento	C.5.20
Modelos de árbol de disfunciones	B.6.6.9
Modelos de datos de entidad-relación-atributo	B.2.4.4
Modelos de Markov	B.6.6.6
Modelos estocásticos generalizados de red de Petri (GSPN)	B.6.6.10
Modularización	B.3.4
Modularización	E.12
Ni variables dinámicas ni objetos dinámicos	C.2.6.3

N

Normas de calidad	E.46
-------------------	------

O

OBJ	C.2.4.6
Observación de las directrices de codificación	E.14
Ocultación/encapsulado de la información	C.2.8
Operación únicamente por operadores cualificados	B.4.5

P

Patrón de ensayo	A.6.1
Perro guardián con base de tiempo separada sin ventana temporal	A.9.1
Perro guardián con base de tiempo separada y ventana temporal	A.9.2
Posibilidades de operación limitada	B.4.4
Principio de corriente en reposo (sin energía para iniciar la activación)	A.1.5
Principios dinámicos	A.2.2
Probado en uso	C.2.10.1
Procedimiento orientado al modelo con un análisis jerárquico	B.2.4.3
Procedimientos basados en script	E.30
Procedimientos de operación y de mantenimiento de los sistemas E/E/PE	B.4
Procesamiento codificado (un canal)	A.3.4
Proceso de producción probado en uso	E.42
Programación defensiva	C.2.5
Programación defensiva	E.16
Programación estructurada	C.2.7
Programación por aserción de las disfunciones	C.3.3
Protección contra las modificaciones	B.4.8
Protección contra las sobretensiones con parada de seguridad	A.8.1
Protección contra los errores humanos	B.4.6
Protección por código	A.6.2
Prototipo/animación	C.5.17
Prueba formal (verificación)	C.5.12
Puerto de acceso de ensayo normalizado y arquitectura de ensayo de barrido del conjunto de las uniones	A.2.3
Puesta fuera de tensión con parada de seguridad	A.8.3

R

Rangos de memoria invariable	A.4
Rangos de memoria variable	A.5
Reconfiguración dinámica	C.3.10
Recuperación hacia atrás	C.3.6
Redes de Petri temporales	B.2.3.3
Redundancia completa del hardware	A.7.3
Redundancia de información	A.7.6
Redundancia de transmisión	A.7.5
Redundancia de un bit (por ejemplo, supervisión de la RAM con un bit de paridad)	A.5.5
Redundancia del hardware sobre un bit	A.7.1
Redundancia del hardware sobre varios bits	A.7.2
Redundancia multibit con protección de palabra (por ejemplo, supervisión de la ROM con un código de Hamming modificado)	A.4.1
Redundancia supervisada	A.2.5
Referencias	D.3
Registro y análisis de datos	C.5.2
Reglas de codificación	C.2.6.2
Reglas de diseño y de codificación	C.2.6
Réplica de bloque (por ejemplo, doble ROM con comparación por hardware o software)	A.4.5
Requisitos de funcionamiento	C.5.19
Requisitos y diseño detallado	C.2
Revisión del diseño	C.5.16
Rutas de datos (comunicación interna)	A.7

S

Salida paralela multicanal	A.6.3
Salidas supervisadas	A.6.4
Seguimiento de las directrices y de las normas	B.3.1
Seguridad y funcionamiento en tiempo real: Arquitectura Activada por Tiempo	C.3.11
Sensor de referencia	A.12.1
Sensor de temperatura	A.10.1
Sensores	A.12
Separación de las funciones de seguridad de las funciones de no-seguridad del sistema E/E/PE	B.1.3
Separación entre las líneas de alimentación eléctrica y las líneas de información	A.11.1
Separación espacial de las líneas múltiples	A.11.2
Simulación	B.3.6
Simulación (V)HDL	E.5
Simulación de la lista de red de puertas para chequear las limitaciones de temporizado	E.22
Simulación de Monte Carlo	B.6.6.8
Simulación del proceso	C.5.18
Sincronización de las entradas primarias y control de meta-estabilidades	E.10
Sondeo	B.3.8
Sondeo	E.19
Sondeo (software)	C.5.15
Subconjuntos de lenguajes	C.4.2
Suma de control modificada	A.4.2
Supervisión	A.13.1
Supervisión cruzada de varios accionadores	A.13.2

Supervisión de la RAM con un código de Hamming modificado, o detección de las disfunciones de los datos con los códigos de detección/corrección de errores (EDC)	A.5.6
Supervisión de la señal analógica	A.2.7
Supervisión de la tensión (secundaria)	A.8.2
Supervisión de los contactos de relés	A.1.2
Supervisión lógica de la secuencia del programa	A.9.3
Supervisión temporal con control en línea	A.9.5
Supervisión temporal y lógica de la secuencia del programa	A.9
Supervisor diversificado	C.3.4
Suposición de los errores	C.5.5

T

Tablas de decisión (tablas de verdad)	C.6.1
Tiempo de respuesta y limitaciones de memoria	C.5.22
Transmisión de señales complementarias	A.11.4
Trazabilidad	C.2.11

U

UML	C.3.12
Unidades de procesamiento	A.3
Unidades e interfaces de E/S (comunicación externa)	A.6
Utilización de componentes probados	B.3.3
Utilización de los elementos del software probados/verificados	C.2.10
Utilización limitada de la recursividad	C.2.6.7
Utilización limitada de las interrupciones	C.2.6.5
Utilización limitada de los punteros	C.2.6.6
Utilización restringida de construcciones asíncronas	E.9

V

Validación de la función lógica	E.21
Validación de la seguridad de los sistemas E/E/PE	B.6
Validación de regresión	C.5.25
VDM, VDM++ – Vienna Development Method	C.2.4.8
Ventilación y calentamiento	A.10
Verificación de la lista de red de puertas contra el modelo de referencia mediante simulación	E.24
Verificación del trazado versus el esquema (LVS)	E.38
Verificación y modificación	C.5

Y

Yourdon en tiempo real	C.2.1.4
------------------------	---------

Z

Z	C.2.4.9
---	---------

ANEXO ZA (Normativo)**OTRAS NORMAS INTERNACIONALES CITADAS EN ESTA NORMA
CON LAS REFERENCIAS DE LAS NORMAS EUROPEAS CORRESPONDIENTES**

Las normas que a continuación se indican son indispensables para la aplicación de esta norma. Para las referencias con fecha, sólo se aplica la edición citada. Para las referencias sin fecha se aplica la última edición de la norma (incluyendo cualquier modificación de ésta).

NOTA Cuando una norma internacional haya sido modificada por modificaciones comunes CENELEC, indicado por (mod), se aplica la EN/HD correspondiente.

Norma Internacional	Fecha	Título	EN/HD	Fecha
IEC 61508-4	2010	Seguridad funcional de los sistemas eléctricos/electrónicos/electrónicos programables relacionados con la seguridad. Parte 4: Definiciones y abreviaturas	EN 61508-4	2010



Génova, 6
28004 MADRID-España

info@aenor.es
www.aenor.es

Tel.: 902 102 201
Fax: 913 104 032